

SUPSI

Rilevamento dati personali in documenti non strutturati aziendali

Studente/i

Algisi Gabriele

Relatore

Gay Mario

Correlatore

Consoli Angelo

Committente

Gay Mario

Corso di laurea

Ingegneria Informatica

Modulo

C11303

Anno

2025-2026

Data

26 agosto 2026

Indice

Dichiarazione sull'uso dell'intelligenza artificiale	vii
1 Introduzione	1
1.1 Abstract	1
1.2 Progetto assegnato	1
1.3 Contesto	2
1.3.1 Il quadro normativo	2
1.3.2 Dalla <i>data anonymization</i> alla <i>data governance</i>	3
1.4 Motivazione	4
1.5 Obiettivi del progetto	4
1.6 Struttura della tesi	5
2 Analisi dei requisiti	7
2.1 Requisiti funzionali	7
2.1.1 Acquisizione dell'input	7
2.1.2 Estrazione del testo	7
2.1.3 Rilevamento delle PII	8
2.1.4 Ingestione del ROPA	8
2.1.5 Confronto dei risultati di analisi con il ROPA	9
2.1.6 Rilevamento delle anomalie di conformità	9
2.1.7 Generazione report per DPO	9
2.1.8 Persistenza e aggiornamento del registro delle PII rilevate	10
2.1.9 Feedback loop	10
2.1.10 Analisi contestuale della struttura del filesystem	10
2.1.11 Impieghi selettivi dell'AI generativa	10
2.2 Requisiti normativi	11
2.2.1 Conformità ai principi di limitazione delle finalità e della conservazione	11
2.2.2 Riferimento al registro delle attività di trattamento come fonte dichiarativa	12
2.2.3 Copertura delle categorie di dati degni di particolare attenzione	13
2.2.4 Posizionamento rispetto all'obbligo di verifica periodica delle misure	13

2.2.5	Esclusione delle basi giuridiche dal perimetro di analisi automatica . . .	13
2.3	Requisiti operativi	14
2.3.1	Elaborazione locale dei dati	14
2.3.2	Vincolo sui modelli AI impiegabili	14
2.3.3	Facilità di installazione e aggiornamento	15
2.3.4	Estensibilità delle categorie e dei detector	16
2.3.5	Minimizzazione dei dati trattati dal sistema stesso	16
2.3.6	Efficienza delle rianalisi periodiche	16
2.3.7	Operatività senza addestramento iniziale (zero-training)	17
2.4	Requisiti di qualità	17
2.4.1	Prestazioni su larga scala	18
2.4.2	Precisione del rilevamento	18
2.4.3	Tracciabilità delle anomalie	18
2.5	Requisiti di interfaccia	19
2.5.1	Interfaccia web come modalità primaria	19
2.5.2	Interfaccia a riga di comando (CLI)	19
2.5.3	Fruizione del report	20
3	Stato dell'arte	21
3.1	Background tecnologico e terminologia	21
3.1.1	Supervised vs unsupervised	21
3.1.2	Pre-tuning vs fine-tuning	22
3.1.3	Zero-shot learning	23
3.1.4	Metriche di valutazione: precisione, recall e F1	24
3.2	Ambiguità contestuale e frammentazione dei domini	25
3.3	La verifica automatizzata della conformità	25
3.4	Approcci tecnici di PII detection	25
3.4.1	Recognition rule-based	26
3.4.2	Recognition con NLP - NER	26
3.4.3	Recognition con AI generativa	26
3.4.4	Limitazioni nella soluzione	27
3.4.5	Soluzioni ibride	27
3.4.6	Confronto empirico degli approcci	27
3.5	Panoramica dei sistemi esistenti	30
3.5.1	Sistemi open-source	30
3.5.2	Sistemi commerciali	31
3.6	Sintesi del gap e posizionamento del progetto	32
3.6.1	Quadro comparativo	33

4	Analisi dei rischi	37
4.1	Rischi di progetto	37
4.1.1	Disponibilità e qualità dei dataset di valutazione	37
4.1.2	Complessità dell'integrazione multi-approccio	38
4.1.3	Costo computazionale e tempi di elaborazione	38
4.1.4	Dipendenza da modelli e librerie di terze parti	39
4.1.5	Dilatazione del perimetro (<i>scope creep</i>)	39
4.1.6	Competenze richieste e curva di apprendimento	40
4.2	Rischi di prodotto	40
4.2.1	Accuratezza insufficiente nella detection delle PII	40
4.2.2	Falsi positivi e impatto sull'usabilità	41
4.2.3	Falsi negativi e mancata rilevazione di PII critiche	41
4.2.4	Generalizzazione cross-domain (<i>domain-silo problem</i>)	42
4.2.5	Supporto multilingue e multilocalizzazione	43
4.2.6	Scalabilità su filesystem di grandi dimensioni	43
4.2.7	Qualità e completezza del ROPA in input	44
4.2.8	Conformità normativa e aggiornamento dei framework legali	45
4.2.9	Sicurezza e protezione dei dati durante l'analisi	45
4.3	Valutazione dei rischi	46
4.3.1	Criteri di classificazione	46
4.3.2	Matrice probabilità–impatto	46
4.4	Strategie di mitigazione	47
5	Studi e valutazioni	51
5.1	Uso dell'AI in rapporto al suo impatto ambientale ed economico	51
5.1.1	Il divario energetico fra modelli discriminativi e generativi	52
5.1.2	Il quadro comparativo	52
5.1.3	Conseguenza per l'architettura del sistema	53
5.2	Costo di addestramento e costo di inferenza	54
5.3	Riutilizzo di Presidio	54
5.4	Scelta dello Small Language Model per gli impieghi selettivi dell'AI	56
5.4.1	Criteri di selezione	56
5.4.2	Fattibilità dell'inferenza su CPU	56
5.4.3	Modelli candidati	57
5.4.4	Analisi per task	59
5.4.5	Confronto e conclusione	59
5.4.6	Energia, efficienza e limiti	60

6 Architettura del sistema	63
6.1 Input del sistema	66
6.1.1 ROPA - Registro dei Trattamenti	66
6.1.2 File system	66
6.2 Normalizzazione ROPA - B1	66
6.2.1 Template registro CNIL	67
6.2.2 Ruolo AI in ROPA	68
6.3 Analisi struttura File System - B2	69
6.4 Estrazione e normalizzazione del testo - B3	69
6.5 Pipeline ibrida di detection di PII - B4	70
6.5.1 Due sorgenti complementari dietro un contratto unico	70
6.5.2 Estendere il rilevamento senza scrivere codice	70
6.5.3 Il merge: unificazione e grado di certezza	71
6.5.4 Impostazione <i>recall-oriented</i>	71
6.5.5 Il ruolo della passata generativa	71
6.5.6 AI generativa come secondo parere	72
6.5.7 Quando usare l'AI	72
6.6 Registro PII rilevate - B5	72
6.7 Associazione documento-trattamento - B6	73
6.8 Verifica conformità PII presenti - B7	74
6.8.1 Verifica della conservazione	75
6.9 Dashboard per DPO - B8	75
6.9.1 Una sola applicazione per più scopi	76
6.9.2 Visualizzazione e minimizzazione	76
7 Implementazione	77
7.1 Ingestione del ROPA - B1	77
7.1.1 Uso del template CNIL	78
7.1.2 Lettura: un'unica forma per ODS ed Excel	78
7.1.3 Interpretazione: guidata dalle etichette, non dalle posizioni	78
7.1.4 Persistenza e mappatura assistita	79
7.2 Analisi struttura file system - B2	80
7.3 Estrazione del testo - B3	80
7.4 Tecnologia di persistenza	81
7.5 Livello di rilevamento - B4	82
7.5.1 Estensibilità config-driven: aggiungere categorie e pattern	83
7.5.2 Presidio come engine	83
7.5.3 Il detector generativo	84
7.5.4 Il merge: realizzazione	84

7.5.5	La politica di campionamento	85
7.5.6	Il benchmark multi-modello	85
7.6	Registro PII rilevate - B5	85
7.6.1	Scansione incrementale	88
7.7	Associazione documento-trattamento - B6	89
7.7.1	Associazione diretta e regole cartella	89
7.7.2	Riconciliazione e precedenza del manuale	90
7.7.3	Applicazione automatica a fine scansione	90
7.7.4	Il collegamento fra i due database	90
7.8	Verifica di conformità - B7	91
7.8.1	Il controllo di conservazione	92
7.9	Dashboard DPO - B8	92
7.9.1	Un'unica applicazione su una porta	93
7.9.2	Dati mostrati sempre per riferimento	93
7.9.3	La scansione asincrona e l'analisi AI	93
7.10	Deployment containerizzato e portabilità	94
7.10.1	Portabilità e isolamento delle dipendenze	94
7.10.2	Architettura di deployment	94
7.10.3	Contratti e direzione	95
8	Test e validazione	97
8.1	Verifica funzionale: i test di unità	97
8.2	Generazione dei corpus sintetici	98
8.3	Metodo di valutazione	99
8.3.1	Ambiente e condizioni di prova	102
8.4	Il sistema di rilevamento nel suo insieme	103
8.4.1	Contributo dei singoli rilevatori	103
8.4.2	Rilevamento tradizionale contro agentico	105
8.4.3	Il sistema completo	106
8.4.4	Validazione end-to-end e degrado da estrazione	107
8.5	Studio sui modelli generativi: qualità contro costo	108
8.5.1	Verifica dell'impatto dell'inferenza su sola CPU	113
9	Risultati e discussione	115
9.1	Il sistema realizzato	115
9.2	Costo e apporto dell'AI generativa	116
9.3	Considerazioni conclusive	118

10 Sviluppi futuri	121
10.1 Analisi contestuale della struttura del file system	121
10.2 Canale di segnalazione dai dipendenti	122
10.3 Estensione oltre i documenti nativi digitali	123
10.4 Tracciamento temporale delle PII rilevate	123
10.5 Limiti dichiarati e direzioni residue	124

Dichiarazione sull'uso dell'intelligenza artificiale

In questa sezione viene esplicitato come è stata utilizzata l'AI generativa durante lo sviluppo del progetto. L'utilizzo di strumenti di AI generativa è stato limitato a **Claude Code**, utilizzato da terminale, con i modelli della famiglia **Claude Opus**.

L'AI è stata utilizzata per

- **Documentazione:** riscrivere frasi intere per impiegare termini più specifici e migliorare la fluidità generale del testo, senza alterarne la sostanza; usato sia per i file `.md` della *codebase*, sia per il documento di tesi. È stata utilizzata anche per convertire rapidamente le tabelle in formato $\text{\LaTeX}$ e per mantenere coerenti i rimandi interni (`\label/\ref`) a fronte di spostamenti e rinumerazioni dei capitoli. Nel Capitolo 4 è stata usata per la stesura delle singole voci a partire dalla traccia definita manualmente.
- **Ricerca di fonti:** supporto parziale al reperimento di letteratura e riferimenti per il Capitolo 3; ogni fonte così individuata è stata poi verificata alla sorgente e consultata prima di essere citata.
- **Commenti:** scrivere e riscrivere i commenti e i *docstring* all'interno della *codebase*, per dare una spiegazione chiara di che cosa fanno le funzioni; da essi è generata la *reference* tecnica del codice.
- **Codice:** scrivere e riscrivere porzioni di codice per correggere *bug*, semplificare le funzioni, alleggerire e migliorare la qualità generale, correggere *typo* nei nomi di variabili e funzioni e in alcuni casi facendo *spec-driven development*; l'uso delle librerie di terze parti e la sintassi sono stati in larga parte delegati. Usato intensivamente nella parte di *frontend* web e nella scrittura dei test, per velocizzare processi meccanici che avrebbero sottratto tempo alla scrittura delle parti più importanti ai fini del progetto.
- **Apparato sperimentale:** scrivere gli script che eseguono le campagne di misura del Capitolo 8 e ne raccolgono i risultati. L'AI ha scritto il **codice che misura**, non i **numeri**

misurati: ogni valore riportato nelle tabelle è l'esito dell'esecuzione di quegli script sul corpus, ed è rigenerabile ri-eseguendo la campagna.

- **Consultazione:** dividere a livello concettuale il carico di lavoro in passi più specifici e focalizzati; in alcuni casi è stata delegata la stesura dei piani di dettaglio, poi riscritti e riformulati manualmente. È stata consultata anche per quanto riguarda la separazione delle sezioni nei vari capitoli della documentazione, chiedendo consigli su materiale da tenere, rivedere o scartare, e per l'impiego di nuove tecnologie o processi.

L'AI NON è stata utilizzata per

- Gestire il progetto a livello organizzativo, comprese le attività relative a Git e GitHub, come gestione del *repository*, *branch*, *issue*, *commit*, *merge* e versionamento del codice;
- Definire autonomamente gli obiettivi, i requisiti e il perimetro del progetto;
- Progettare da zero l'architettura generale, la struttura del progetto e la sua suddivisione in blocchi funzionali;
- Prendere autonomamente decisioni progettuali e implementative, o decidere da zero come realizzare una nuova funzionalità;
- Scegliere autonomamente quali *pattern* di programmazione adottare e come integrarli nell'architettura del progetto;
- Prendere le decisioni finali riguardo a tecnologie, modelli e strategie di rilevamento, fusione dei risultati e verifica di conformità;
- Produrre i valori sperimentali riportati nelle tabelle del Capitolo 8, che provengono unicamente dall'esecuzione delle campagne di misura;
- Prendere le decisioni finali nell'interpretazione dei risultati sperimentali;
- Eseguire operazioni di pubblicazione, *merge* o *deployment* senza una verifica personale;
- Sostituire la revisione e la verifica personale del codice generato o modificato con il supporto dell'AI;
- Generare *asset* grafici o artistici, tra cui logo, sfondi e grafiche utilizzati nel *frontend* o nella documentazione.

Capitolo 1

Introduzione

1.1 Abstract

Le organizzazioni tenute a rispettare le normative sulla protezione dei dati personali devono dichiarare in un registro dei trattamenti quali dati raccolgono, per quale scopo e per quanto tempo li conservano. Buona parte di quei dati vive però dentro documenti ordinari — contratti, moduli, fogli di calcolo — sparsi nelle cartelle aziendali, dove nessuno verifica se ciò che il registro dichiara corrisponda a ciò che è realmente conservato.

Questa tesi affronta tale distanza fra dichiarato e reale. Il lavoro si articola in due parti. La prima è la realizzazione di un prototipo che analizza i documenti di un'organizzazione, vi individua i dati personali presenti, li confronta con quanto il registro dichiara e segnala al responsabile della protezione dei dati ciò che non torna: dati che nessun trattamento giustifica e documenti conservati oltre il termine previsto. La seconda parte raccoglie gli studi che hanno guidato quella realizzazione e ne verificano l'esito, con particolare attenzione a una domanda oggi ricorrente: se e quando convenga affidare all'intelligenza artificiale generativa un compito che tecniche più tradizionali svolgono da tempo.

1.2 Progetto assegnato

Le aziende e le organizzazioni soggette a normative sulla protezione dei dati personali, inclusi GDPR e nLPD (nuova Legge federale sulla protezione dei dati), sono tenute a sapere dove si trovano i dati personali che trattano, a quale trattamento riportato nel registro dei trattamenti sono riconducibili e se sono ancora entro i termini di conservazione dichiarati nel registro. Nella pratica però, gran parte di questi dati è dispersa in documenti non strutturati — ad esempio PDF, Word, Excel, scansioni — senza che l'organizzazione ne abbia visibilità sistematica [1].

Gli strumenti disponibili a supporto della conformità alle normative europee (GDPR) o statunitensi (es. CCPA), sia quelli più abordabili (es. PII Crawler, PII Tools) sia quelli

enterprise (es. BigID, Varonis) [2, 3, 4], non necessariamente gestiscono le categorie di dati specifiche della nLPD svizzera senza configurazioni e validazioni dedicate. Oltre a ciò, solo alcuni prodotti enterprise più avanzati integrano il registro dei trattamenti come riferimento dichiarativo configurabile rispetto al quale valutare i dati rilevati. Di conseguenza la verifica del superamento del periodo di retention è spesso possibile solo tramite una categorizzazione molto precisa all'origine.

Il progetto propone lo sviluppo di un sistema in grado di evidenziare il divario tra le attività di trattamento dichiarate nel registro e l'effettiva gestione dei documenti aziendali, anche utilizzando tecniche NLP e AI. Per ciascun documento analizzato, il sistema deve identificare eventuali dati personali presenti (Personally Identifiable Information, PII), associare il documento alla corrispondente attività di trattamento combinando diversi indicatori (es. posizione nel file system, contenuto, PII identificati) e verificare la conformità al periodo di conservazione dichiarato nel registro. I documenti che hanno superato il loro periodo di conservazione e le PII [5] che non possono essere collegate ad alcuna operazione di trattamento registrata devono essere segnalati come anomalie.

1.3 Contesto

1.3.1 Il quadro normativo

La protezione dei dati personali costituisce oggi uno dei nodi centrali della governance organizzativa, tanto sul piano giuridico quanto su quello operativo. In Europa, il regolamento (UE) 2016/679 (GDPR) [6] ha introdotto un corpus di obblighi sostanziali per i titolari del trattamento, tra cui il principio di limitazione della conservazione sancito dall'art. 5(1)(e), secondo cui i dati personali devono essere conservati "in una forma che consenta l'identificazione degli interessati per un arco di tempo non superiore al conseguimento delle finalità per le quali sono trattati". In Svizzera, la nuova Legge federale sulla protezione dei dati (nLPD, RS 235.1, in vigore dal 1° settembre 2023) ha recepito principi analoghi: l'art 6(4) dispone che "i dati personali sono distrutti o resi anonimi appena non sono più necessari per lo scopo del trattamento", stabilendo un legame diretto tra la finalità dichiarata e il ciclo di vita del dato. Entrambe le normative impongono inoltre la tenuta di un registro delle attività di trattamento (ROPA, Record of Processing Activities). Il GDPR, all'art. 30(1)(f), richiede che il registro contenga, "ove possibile, i termini ultimi previsti per la cancellazione delle diverse categorie di dati"; la nLPD [7], all'art. 12(2)(e), prevede analogamente l'indicazione della durata di conservazione dei dati personali o dei criteri per determinarne la durata. Il ROPA costituisce dunque il documento di riferimento per qualsiasi verifica di conformità: è in esso che l'organizzazione dichiara cosa tratta, perché, con quale base giuridica e per quanto tempo. I due ordinamenti divergono tuttavia su un punto strutturalmente rilevante per qualsiasi sistema di verifica automatica. Il GDPR elenca le basi giuridiche lecite all'art. 6(1), subor-

dinando la liceità del trattamento alla sussistenza di almeno una di esse. La nLPD adotta invece un approccio invertito: il trattamento da parte di privati — termine con cui la legge designa i soggetti di diritto privato, imprese incluse, in contrapposizione agli organi federali, ai quali l'art. 34 impone una base legale — non richiede alcuna giustificazione preventiva ed è quindi lecito per default, ma diventa illecito se lede ingiustificatamente la personalità dell'interessato ai sensi dell'art. 30. La giustificazione può provenire dal consenso, da un interesse preponderante o dalla legge, secondo il catalogo dell'art. 31. Questa differenza strutturale impone che un sistema di analisi della conformità tenga conto del framework giuridico applicabile, poiché il criterio con cui si valuta l'esistenza di una base legittima non è identico nei due contesti.

Nella pratica organizzativa, la conformità dichiarata nel registro tende a divergere dalla situazione reale dei dati. Gran parte dell'informazione trattata da un'organizzazione è incorporata in documenti non strutturati (file PDF, Word, Excel, messaggi di posta elettronica. . .) distribuiti su file system locali, server condivisi e servizi cloud, categoria che secondo la letteratura costituisce la quota nettamente maggioritaria dei dati aziendali [1]. Questi documenti non recano metadati normalizzati che consentono di associarli automaticamente a un'attività di trattamento specifica, né di verificarne la conformità al periodo di retention dichiarato. Il risultato è un divario strutturale tra ciò che il registro descrive e ciò che esiste effettivamente nei sistemi informativi dell'organizzazione.

Questo divario assume rilevanza giuridica diretta. Il GDPR all'art. 32(1)(d) qualifica come misura di sicurezza obbligatoria una procedura per testare, verificare e valutare regolarmente l'efficacia delle misure tecniche e organizzative al fine di garantire la sicurezza del trattamento. La nLPD, pur con formulazione più sintetica, impone all'art. 8(1) che il titolare e il responsabile del trattamento garantiscano, mediante appropriati provvedimenti tecnici e organizzativi, che la sicurezza dei dati personali sia adeguata al rischio. Un sistema in grado di rilevare automaticamente i dati personali presenti nei documenti aziendali e di verificarne la coerenza con il registro può essere inteso come implementazione tecnica di quest'obbligo.

1.3.2 Dalla *data anonymization* alla *data governance*

Gran parte degli strumenti che trattano dati personali in testo libero nasce per rispondere a una domanda operativa: *come rimuovo questo dato?* La rassegna dello stato dell'arte del settore [8] lo conferma: le famiglie di tecniche censite — e con esse i corpus di riferimento su cui vengono valutate [9] — sono tutte orientate alla rimozione o alla sostituzione del dato. L'anonimizzazione, la pseudonimizzazione e il mascheramento condividono infatti il medesimo presupposto — il dato personale è un ostacolo da neutralizzare prima che il documento venga condiviso, pubblicato o impiegato per addestrare un modello. È un presupposto legittimo, ma risponde a un problema diverso da quello affrontato in questa tesi.

La domanda della *data governance* [10] non è come rimuovere un dato, bensì se quel dato **possa legittimamente trovarsi** dove si trova: se rientri fra le categorie che l'organizzazione ha dichiarato di trattare per quella finalità, e se sia conservato entro il termine di tempo dichiarato. Il dato non va quindi eliminato ma **messo in relazione** con ciò che l'organizzazione afferma di sé nel proprio registro dei trattamenti. Ne discende una conseguenza che orienta l'intero lavoro: il rilevamento delle PII smette di essere il fine del sistema e ne diventa un **presupposto**. Un rilevatore, per quanto accurato, non risponde ancora ad alcuna domanda di conformità finché ciò che trova non viene confrontato con ciò che è stato dichiarato.

1.4 Motivazione

Il divario descritto nella sezione precedente tra le attività di trattamento dichiarate nel registro e la gestione effettiva dei documenti non è soltanto un problema teorico: ha conseguenze dirette sulla capacità di un'organizzazione di dimostrare conformità. Un Data Protection Officer che debba rispondere a una richiesta dell'autorità di controllo, o semplicemente verificare se un'attività di trattamento è ancora coperta da una base giuridica valida, si trova nella pratica privo di uno strumento che colleghi sistematicamente il contenuto dei documenti aziendali al registro. La verifica, quando viene fatta, è perlopiù manuale, parziale e non ripetibile nel tempo [11].

Come verrà approfondito nell'analisi degli strumenti esistenti (cfr. 3.5), il panorama commerciale attuale non risolve in modo sufficiente questo problema per il contesto svizzero, lasciando aperto uno spazio applicativo concreto. Allo stesso tempo, come discusso negli studi effettuati (cfr. 5), la maturità raggiunta dalle tecniche di elaborazione del linguaggio naturale rende oggi percorribile, a un costo computazionale sostenibile, un approccio che pochi anni fa avrebbe richiesto un investimento sproporzionato [12, 13].

Il progetto nasce dunque dall'intersezione di tre fattori: un obbligo normativo concreto, non pienamente automatizzabile con gli strumenti esistenti; uno spazio applicativo non coperto in modo adeguato dal mercato; e una maturità tecnologica che rende oggi realistico costruire un sistema di verifica della conformità basato sul confronto tra registro dichiarato e contenuto documentale effettivo.

1.5 Obiettivi del progetto

Il progetto intende produrre un sistema che renda visibile lo scarto tra le attività di trattamento dichiarate nel registro e la gestione effettiva dei dati nei documenti aziendali. In pratica, il prototipo dovrà:

- rilevare le categorie di dati personali presenti in documenti non strutturati, sulla base delle categorie canoniche della nLPD e di quelle dichiarate nel registro dei trattamenti;

- associare i documenti al trattamento corrispondente combinando più segnali;
- verificare la conformità ai periodi di retention dichiarati nel registro;
- segnalare categorie di dati personali non dichiarate nel registro e trattamenti non censiti;
- produrre un report leggibile dal Data Protection Officer dell'organizzazione.

1.6 Struttura della tesi

In questa sezione viene fornita la struttura con cui viene presentato il lavoro. Le argomentazioni discusse sono suddivise come segue:

1. **Introduzione** (1): inquadra il progetto assegnato, il contesto normativo (GDPR e nLPD) e il divario fra registro dichiarato e gestione effettiva dei documenti; ne motiva l'attualità e ne fissa gli obiettivi.
2. **Analisi dei requisiti** (2): raccoglie e classifica ciò che il sistema deve fare — requisiti funzionali, normativi, operativi e di qualità — descrivendo il problema senza anticipare le scelte realizzative.
3. **Stato dell'arte** (3): ricostruisce il background tecnologico, gli approcci esistenti al rilevamento delle PII e i sistemi disponibili sul mercato, per individuare il gap che il progetto intende colmare.
4. **Analisi dei rischi** (4): individua i rischi di progetto e di prodotto, li valuta secondo una matrice probabilità–impatto e definisce le strategie di mitigazione previste.
5. **Studi e valutazioni effettuate** (5): raccoglie gli studi comparativi a supporto delle decisioni progettuali — impatto ambientale ed economico dell'AI, riuso di Presidio e scelta dello Small Language Model per gli impieghi generativi.
6. **Architettura** (6): presenta la struttura del sistema a blocchi (B1–B8), dagli input (ROPA e file system) fino alla dashboard, motivando le scelte di progettazione di ciascuno.
7. **Implementazione** (7): descrive la realizzazione concreta dei blocchi architetturali, le tecnologie adottate, la pipeline di detection e il deployment containerizzato e portabile.
8. **Test e validazione** (8): illustra i test di unità, la generazione dei corpus sintetici e il metodo di valutazione, misurando le prestazioni del rilevamento e il costo dell'AI generativa.
9. **Risultati e discussione** (9): tira le somme sul sistema realizzato e sull'effettiva convenienza dell'AI generativa, con le considerazioni conclusive sul lavoro svolto.

10. **Sviluppi futuri** (10): documenta le funzionalità concepite ma non realizzate — con valore atteso, costo ed esito — e le direzioni residue di estensione del sistema.

Capitolo 2

Analisi dei requisiti

La struttura di questo capitolo si ispira a un modello di specifica dei requisiti conforme allo standard ISO/IEC/IEEE 29148:2018 — *Systems and software engineering — Life cycle processes — Requirements engineering* [14], nella resa fornita da un modello pubblico [15]. Ogni requisito è enunciato in forma verificabile e, dove non risulti immediata, ne è esplicitata la sorgente: un obbligo di legge, un vincolo del contesto d'impiego o un obiettivo assegnato al progetto. Le soglie quantitative e i criteri di accettazione sperimentali sono rimandati al Capitolo 8.

2.1 Requisiti funzionali

2.1.1 Acquisizione dell'input

Il sistema deve poter analizzare come input un insieme di file selezionati, una cartella oppure un intero filesystem. L'acquisizione deve avvenire senza presupporre una struttura predefinita dell'albero di directory e deve supportare l'elaborazione ricorsiva delle sottocartelle.

L'utente deve poter specificare percorsi multipli e pattern di esclusione, in modo da limitare la scansione ai soli percorsi rilevanti.

L'input è costituito esclusivamente da file residenti su un file system. Le ulteriori sorgenti in cui un'organizzazione conserva dati personali — server di posta elettronica, sistemi di gestione di basi di dati, applicativi gestionali e servizi che non espongano i propri contenuti come file — restano fuori dal perimetro del progetto: la loro acquisizione richiederebbe un connettore dedicato per ciascuna tecnologia, senza modificare la natura del problema affrontato, che riguarda il testo non strutturato.

2.1.2 Estrazione del testo

Il sistema deve estrarre il contenuto testuale dai documenti non strutturati indipendentemente dal formato di partenza, supportando almeno i documenti impaginati in formato PDF, i fogli di

calcolo e il testo semplice.

L'estrazione deve preservare, dove il formato di partenza lo consente, i metadati strutturali necessari a contestualizzare le PII rilevate e a tracciarne la posizione nel report finale (cfr. 2.4.3): almeno la suddivisione in pagine, l'organizzazione tabellare e le intestazioni di sezione.

2.1.3 Rilevamento delle PII

Il sistema deve identificare le occorrenze di dati personali nel testo estratto tramite una combinazione di meccanismi complementari — pattern deterministici, modelli statistici e tecniche di elaborazione del linguaggio naturale [16, 17, 18] — la cui composizione esatta è oggetto di definizione architetturale.

Il rilevamento deve coprire almeno i dati identificativi, i dati di contatto, gli identificatori numerici nazionali e i dati degni di particolare protezione ai sensi dell'art. 5 lett. c nLPD [7], corrispondenti alle categorie particolari di dati personali dell'art. 9 GDPR [6].

2.1.4 Ingestione del ROPA

Il sistema deve accettare in input il registro delle attività di trattamento (ROPA) dell'organizzazione in formato strutturato — eventualmente articolato in più sezioni o fogli — da utilizzare come riferimento dichiarativo autoritativo per il confronto con le PII rilevate nei documenti. Il ROPA costituisce, ai sensi dell'art. 30 GDPR [6] e dell'art. 12 nLPD [7], il documento in cui l'organizzazione dichiara le proprie attività di trattamento: senza la sua ingestione il sistema non potrebbe effettuare alcuna verifica di conformità.

Il sistema deve supportare almeno i formati di foglio di calcolo ODS e XLSX, nei quali i registri sono comunemente redatti e distribuiti dalle autorità di controllo [19]. Per ogni attività di trattamento, il formato di ingestione deve includere almeno i seguenti campi:

- categorie di dati personali previste;
- finalità del trattamento;
- periodo di conservazione dichiarato, o i criteri per la sua determinazione;
- categorie di interessati;
- eventuali categorie di destinatari.

Il sistema deve segnalare all'utente, in fase di ingestione, i campi obbligatori mancanti o i valori non interpretabili, senza interrompere l'elaborazione per i record validi. Il requisito si considera soddisfatto quando il sistema importa correttamente un registro di riferimento in ciascuno dei formati supportati, estraendone tutti i campi previsti e segnalando i record incompleti.

2.1.5 Confronto dei risultati di analisi con il ROPA

Il sistema deve confrontare le entità e le categorie di dati personali effettivamente rilevate nei documenti con quanto dichiarato nel ROPA, verificando la coerenza tra dati trovati e dati dichiarati.

Il confronto deve produrre, per ciascun documento analizzato, un'associazione — anche parziale o nulla — con una o più attività di trattamento del ROPA, secondo criteri ed euristiche definiti in fase di progettazione architettuale.

2.1.6 Rilevamento delle anomalie di conformità

Il sistema deve identificare e classificare, sulla base del confronto di cui alla sezione 2.1.5, le seguenti tipologie di anomalie:

- categorie di PII rilevate ma non dichiarate nel ROPA per l'attività di trattamento associata;
- dati personali privi di una base giuridica o di un'attività di trattamento associabile ("PII orfane");
- violazioni dei periodi di conservazione dichiarati nel ROPA, ossia documenti contenenti PII oltre il termine di retention dichiarato.

Ogni anomalia rilevata deve essere classificata secondo una tipologia predefinita e associata a un livello di gravità.

2.1.7 Generazione report per DPO

Il sistema deve produrre, come esito dell'analisi, un report di conformità destinato al Data Protection Officer (DPO), che raccolga le anomalie rilevate e sia consultabile senza competenze tecniche.

Il report deve includere, per ciascuna anomalia, la tipologia, i documenti coinvolti, l'attività di trattamento di riferimento ove presente e la motivazione del rilevamento. Deve inoltre essere organizzato **per documento**, in modo che il DPO possa passare dal quadro d'insieme del corpus documentale al dettaglio del singolo file, delle sue PII e del suo verdetto.

Il report deve descrivere le PII rilevate come *riferimenti* — categoria, posizione, provenienza del rilevamento — e non deve in nessun caso riportare il valore del dato personale rilevato (cfr. 2.3.5).

2.1.8 Persistenza e aggiornamento del registro delle PII rilevate

Il sistema deve mantenere un registro interno persistente delle PII rilevate, che associ a ciascuna occorrenza il file sorgente, la posizione all'interno del documento, la categoria di PII e un identificatore temporale dell'analisi.

Il registro deve consentire il confronto tra esecuzioni successive sullo stesso corpus documentale, al fine di rilevare:

- PII precedentemente segnalate che risultano rimosse o modificate nel documento sorgente;
- nuove PII introdotte da documenti aggiunti o modificati dopo l'ultima analisi;
- documenti eliminati dal filesystem che contenevano PII precedentemente censite.

Il formato del registro deve essere strutturato e interrogabile meccanicamente, così da supportare sia la consultazione diretta sia l'integrazione con il report di conformità.

2.1.9 Feedback loop

Il sistema deve mettere a disposizione un canale di segnalazione che consenta al personale dell'organizzazione di notificare al DPO le PII presenti nei documenti aziendali e sfuggite al rilevamento automatico.

Le segnalazioni raccolte devono confluire nel registro interno (cfr. 2.1.8) mantenendo distinta la provenienza manuale da quella automatica, in modo che il DPO possa valutarle e, ove pertinenti, utilizzarle per estendere le regole di rilevamento (cfr. 2.3.4).

2.1.10 Analisi contestuale della struttura del filesystem

Il sistema deve considerare la posizione di ciascun documento all'interno della gerarchia del filesystem come segnale contestuale per l'associazione all'attività di trattamento. La struttura delle cartelle — nomi, profondità, organizzazione logica — può fornire indicazioni sulla finalità del trattamento e sulla legittimità della presenza di determinati dati personali in una posizione specifica.

Il sistema deve segnalare nel report come anomalia posizionale il documento le cui categorie di PII non risultano coerenti con la finalità della collocazione che lo ospita, in quanto indicativo di una possibile violazione del principio di limitazione delle finalità.

2.1.11 Impieghi selettivi dell'AI generativa

Il sistema può impiegare modelli di AI generativa per attività mirate che beneficiano della comprensione semantica profonda, senza richiederne l'applicazione sistematica a ogni documento del corpus. Tali impieghi sono circoscritti a:

- **arricchimento del ROPA:** qualora il registro fornito in input contenga descrizioni sintetiche o generiche delle categorie di dati, il sistema può utilizzare un modello linguistico per espanderle e normalizzarle [20], migliorando la granularità del confronto con le PII rilevate;
- **analisi campionata:** il sistema può sottoporre a un modello linguistico un sottoinsieme rappresentativo di documenti per ottenerne un'analisi contestuale approfondita, identificando PII deducibili dal contesto che sfuggirebbero ai metodi deterministici e statistici [20];
- **analisi della struttura del filesystem:** il sistema può utilizzare un modello linguistico per interpretare la semantica dei percorsi e dei nomi di cartella, al fine di rilevare incoerenze tra la posizione del documento e il suo contenuto (cfr. 2.1.10).

Questi impieghi devono essere documentati e giustificati da studi comparativi (cfr. 5) che ne attestino il valore aggiunto rispetto al costo computazionale sostenuto.

2.2 Requisiti normativi

2.2.1 Conformità ai principi di limitazione delle finalità e della conservazione

Il sistema deve supportare la verifica di conformità ai seguenti principi normativi:

- **limitazione della finalità** (art. 5(1)(b) GDPR [6]; art. 6 cpv. 3 nLPD [7]): i dati personali devono essere raccolti per finalità determinate, esplicite e legittime, e non ulteriormente trattati in modo incompatibile con tali finalità;
- **limitazione della conservazione** (art. 5(1)(e) GDPR [6]; art. 6 cpv. 4 nLPD [7]): i dati personali devono essere conservati per un arco di tempo non superiore al conseguimento delle finalità per le quali sono trattati.

Questi due principi costituiscono il fondamento giuridico dell'intera logica di verifica del sistema: il confronto tra le PII rilevate nei documenti e quanto dichiarato nel ROPA è, nella sua essenza, una verifica che i dati trattati siano coerenti con le finalità dichiarate e che non siano conservati oltre i termini previsti.

La verifica deve operare su due assi:

1. **asse delle finalità:** per ciascun documento analizzato, il sistema deve verificare che le categorie di PII rilevate rientrino tra quelle dichiarate per l'attività di trattamento associata;
2. **asse della conservazione:** per ciascun documento contenente PII, il sistema deve confrontare la data del documento, o quella del suo ultimo aggiornamento, con il periodo di retention dichiarato nel ROPA per l'attività di trattamento associata.

Sull'asse della conservazione va precisato che cosa il sistema può realmente garantire. La data di un documento non strutturato non è un dato accertabile ma una **stima**, ricavata dalle sole informazioni che il file rende disponibili: i segnali utilizzabili — i metadati interni al formato e la data di ultima modifica registrata dal file system — hanno affidabilità diversa fra loro e sono entrambi alterabili da eventi estranei al contenuto, come una copia massiva o un ripristino da backup. Il sistema deve pertanto **registrare e mostrare la provenienza** della data impiegata, affinché il DPO possa distinguere una segnalazione fondata da un indizio da verificare. Deve inoltre **segnalare esplicitamente** i casi in cui il confronto non è possibile — tipicamente quando il registro esprime la conservazione come criterio anziché come durata — anziché considerarli conformi: l'assenza di verifica non deve essere indistinguibile da un esito favorevole.

Il requisito si considera soddisfatto quando, dato un corpus documentale di test con anomalie note, il sistema le identifica correttamente su entrambi gli assi, senza segnalare come non conformi i documenti che non lo sono.

2.2.2 Riferimento al registro delle attività di trattamento come fonte dichiarativa

Il sistema deve trattare il ROPA fornito in input come riferimento dichiarativo autoritativo per ogni valutazione di conformità, e non deve generare, inferire o modificare autonomamente le attività di trattamento: il ROPA rappresenta la *ground truth* dichiarativa dell'organizzazione [11].

L'art. 30 GDPR [6] e l'art. 12 nLPD [7] impongono al titolare del trattamento la tenuta di un registro che documenti, tra l'altro, le categorie di interessati, le categorie di dati personali trattati, le finalità, la durata di conservazione e le categorie di destinatari. Il sistema si fonda su questo documento come base di confronto, senza sindacarne la correttezza né sostituirsi alla responsabilità del titolare nella sua compilazione.

Qualora il ROPA fornito risulti incompleto o incoerente, il sistema deve:

- segnalare le carenze nel report, classificandole come anomalie del ROPA stesso;
- proseguire l'analisi limitatamente ai campi disponibili, senza interrompere l'elaborazione.

Il requisito si considera soddisfatto quando il sistema, ricevendo un ROPA con campi mancanti, produce le segnalazioni appropriate senza errori di esecuzione e senza alterare i dati del registro fornito.

2.2.3 Copertura delle categorie di dati degni di particolare attenzione

Il sistema deve riconoscere le categorie di dati personali degni di particolare protezione secondo l'art. 5 lett. c nLPD [7] e le categorie particolari di dati personali secondo l'art. 9(1) GDPR [6], segnalandone la presenza con priorità elevata nel report delle anomalie, in considerazione del regime di consenso espresso richiesto (art. 6 cpv. 7 lett. a nLPD [7]; art. 9(2) GDPR [6]).

2.2.4 Posizionamento rispetto all'obbligo di verifica periodica delle misure

Il sistema, nella sua esecuzione periodica, deve costituire un'implementazione tecnica della procedura di *test, verifica e valutazione regolare dell'efficacia delle misure tecniche e organizzative* prevista dall'art. 32(1)(d) GDPR [6] e, analogamente, dall'art. 8 nLPD [7].

L'art. 32(1)(d) GDPR [6] qualifica come misura di sicurezza obbligatoria una procedura per testare e valutare regolarmente l'efficacia delle misure adottate. Un sistema che rileva periodicamente i dati personali nei documenti aziendali e ne verifica la coerenza con il registro dei trattamenti implementa, di fatto, questa procedura con riferimento alla conformità documentale.

Il perimetro del sistema è tuttavia limitato alla verifica di conformità documentale. Restano esplicitamente esclusi:

- le funzioni di anonimizzazione o pseudonimizzazione dei dati;
- la gestione degli incidenti di sicurezza (*data breach*);
- la valutazione d'impatto sulla protezione dei dati (DPIA).

Il requisito si considera soddisfatto quando il sistema può essere eseguito periodicamente in modo non presidiato, producendo report comparabili tra esecuzioni successive.

2.2.5 Esclusione delle basi giuridiche dal perimetro di analisi automatica

Il sistema non deve determinare autonomamente la base giuridica applicabile a un trattamento (art. 6 GDPR [6]; art. 30-31 nLPD [7]): tale valutazione resta di competenza esclusiva del DPO o del consulente giuridico dell'organizzazione.

La determinazione della base giuridica richiede una valutazione contestuale di natura giuridica che non è riducibile a un'analisi automatica del contenuto documentale. Come evidenziato nella sezione 2.2.1, il GDPR e la nLPD adottano inoltre approcci strutturalmente diversi alla liceità del trattamento [6, 7], rendendo impossibile un'automazione generalizzata senza introdurre rischi di errore significativi.

Il sistema si limita pertanto a:

- segnalare la presenza di PII non riconducibili ad alcuna attività di trattamento dichiarata nel ROPA (“PII orfane”);
- fornire al DPO gli elementi contestuali — documento, posizione, categoria di PII — necessari per una valutazione umana.

Il requisito si considera soddisfatto quando il sistema non associa in nessun caso autonomamente una base giuridica a un trattamento e le PII non associabili ad alcuna attività di trattamento risultano segnalate come orfane.

2.3 Requisiti operativi

2.3.1 Elaborazione locale dei dati

Il sistema deve garantire che ogni dato analizzato rimanga nell'infrastruttura in cui si trova inizialmente. Nessun contenuto documentale, testo estratto, PII rilevata o dato intermedio deve essere trasmesso a servizi esterni, API cloud o endpoint di inferenza remoti, in nessuna fase dell'elaborazione.

I documenti analizzati possono contenere dati degni di particolare protezione ai sensi dell'art. 5 lett. c nLPD [7] e categorie particolari ai sensi dell'art. 9 GDPR [6]. La trasmissione di tali dati a servizi terzi introdurrebbe un trattamento ulteriore che richiederebbe una base giuridica propria, un'analisi dei rischi specifica e garanzie contrattuali (art. 28 GDPR [6]), vanificando la finalità stessa del sistema.

Il vincolo si applica a tutte le componenti del sistema, incluse:

- i modelli di AI e NLP (cfr. 2.3.2);
- i moduli di estrazione del testo (cfr. 2.1.2);
- il registro interno delle PII (cfr. 2.1.8);
- la generazione e la presentazione del report di conformità (cfr. 2.1.7).

Il requisito si considera soddisfatto quando, durante l'esecuzione del sistema, non viene effettuata alcuna connessione di rete in uscita per la trasmissione di dati documentali o da essi derivati; la verifica può essere condotta tramite analisi del traffico di rete o ispezione del codice sorgente.

2.3.2 Vincolo sui modelli AI impiegabili

Il sistema deve impiegare esclusivamente modelli di intelligenza artificiale eseguibili interamente in locale, senza richiedere la trasmissione dei dati analizzati a servizi di inferenza

esterni. Tale vincolo discende direttamente dal requisito di elaborazione locale (cfr. 2.3.1) e dalla natura potenzialmente sensibile dei dati trattati.

La scelta dei modelli deve bilanciare accuratezza e sostenibilità computazionale:

- per il rilevamento sistematico delle PII sull'intero corpus documentale, il sistema deve privilegiare approcci computazionalmente leggeri, eseguibili su hardware aziendale standard [21, 22];
- per le attività selettive che richiedono comprensione semantica profonda (cfr. 2.1.11), il sistema può impiegare modelli generativi di dimensioni compatibili con l'esecuzione locale [20], limitandone l'applicazione a sottoinsiemi mirati del corpus.

La giustificazione della scelta architetturale dei modelli deve essere documentata attraverso studi comparativi che confrontino i risultati ottenuti con il consumo di risorse sostenuto (cfr. Capitolo 5).

2.3.3 Facilità di installazione e aggiornamento

Il sistema deve essere installabile e operativo in qualsiasi infrastruttura aziendale con un numero minimo di passaggi di configurazione, senza richiedere personalizzazioni specifiche per l'ambiente di destinazione. Il sistema è destinato a organizzazioni con livelli di maturità IT eterogenei: un processo di installazione complesso o dipendente da configurazioni ad-hoc ne ridurrebbe l'adozione e renderebbe difficoltosa la manutenzione nel tempo.

In particolare, il sistema deve soddisfare i seguenti vincoli:

- l'installazione deve avvenire tramite un meccanismo standardizzato e riproducibile, che integri le dipendenze necessarie all'esecuzione;
- la configurazione iniziale deve limitarsi alla specifica dei percorsi di input — documenti e ROPA — e delle preferenze di output;
- l'aggiornamento del sistema deve poter avvenire senza perdita dei dati del registro interno (cfr. 2.1.8) e senza richiedere una reinstallazione completa;
- in caso di malfunzionamento, il sistema deve poter essere riportato a una configurazione funzionante nota senza interventi specialistici.

Il requisito si considera soddisfatto quando l'installazione su un ambiente pulito può essere completata seguendo la documentazione fornita, senza interventi manuali non documentati.

2.3.4 Estensibilità delle categorie e dei detector

Il sistema deve adottare un'architettura modulare che consenta l'aggiunta, la rimozione e la modifica delle categorie di PII rilevabili e dei relativi detector senza richiedere modifiche al codice sorgente. Le categorie di dati personali rilevanti variano infatti in funzione del framework normativo applicabile, del settore di attività dell'organizzazione e delle specifiche attività di trattamento dichiarate nel ROPA: un sistema non estensibile costringerebbe l'organizzazione ad attendere un aggiornamento del prodotto per coprire le categorie specifiche del proprio contesto.

L'estensibilità deve essere garantita su due livelli:

- **categorie di PII:** l'operatore deve poter definire nuove categorie di dati personali specificandone il nome, la descrizione e le regole di rilevamento associate;
- **detector:** il sistema deve supportare l'aggiunta di nuovi moduli di rilevamento tramite un'interfaccia di configurazione dichiarativa, senza richiedere competenze di sviluppo software.

Il requisito si considera soddisfatto quando un operatore non sviluppatore è in grado di aggiungere una nuova categoria di PII con il relativo detector seguendo la documentazione fornita, e il sistema la rileva correttamente al successivo ciclo di analisi.

2.3.5 Minimizzazione dei dati trattati dal sistema stesso

Il sistema, nel corso della propria esecuzione, deve rispettare il principio di minimizzazione dei dati (art. 5(1)(c) GDPR [6]; art. 6 cpv. 2 nLPD [7]), limitando la quantità di informazioni personali che tratta e conserva internamente a quanto strettamente necessario per produrre il report di conformità.

In particolare:

- il sistema non deve duplicare integralmente i documenti analizzati; il contenuto estratto deve essere mantenuto in memoria per la sola durata dell'elaborazione;
- le PII rilevate devono essere riportate nel registro interno (cfr. 2.1.8) e nel report finale attraverso riferimenti posizionali, e non tramite la riproduzione del dato personale;
- i dati intermedi devono essere eliminati al termine dell'esecuzione.

2.3.6 Efficienza delle rianalisi periodiche

Il sistema è destinato a un impiego **ricorrente** sullo stesso corpus documentale: la conformità non è una fotografia ma uno stato da mantenere nel tempo, e il registro interno (cfr. 2.1.8) esiste proprio per confrontare esecuzioni successive. Un'analisi che rielabori integralmente

l'intero corpus a ogni esecuzione rende però l'esecuzione periodica impraticabile su condivisioni aziendali reali, e il costo sostenuto è interamente superfluo per i documenti che nel frattempo non sono stati toccati [3].

Il sistema deve pertanto:

- rianalizzare i soli documenti **modificati** dall'ultima esecuzione, riconoscendoli da informazioni ottenibili senza leggerne il contenuto;
- **dichiarare** all'operatore quali documenti sono stati analizzati e quali omessi in quanto invariati, sia prima dell'esecuzione sia nel resoconto finale;
- consentire di **forzare** una rianalisi completa, e provvedervi autonomamente quando muta la configurazione del rilevamento, poiché in tal caso i risultati conservati non descrivono più ciò che il sistema troverebbe;
- garantire che l'omissione di un documento invariato non alteri in alcun modo il registro, e in particolare non lo faccia apparire rimosso.

Il requisito si considera soddisfatto quando una seconda esecuzione su un corpus immutato non rianalizza alcun documento, lascia il registro identico a quello prodotto dalla prima e lo dichiara nel proprio resoconto.

2.3.7 Operatività senza addestramento iniziale (zero-training)

Il sistema deve essere operativo immediatamente dopo l'installazione, senza richiedere una fase di addestramento su dati specifici dell'organizzazione. Il rilevamento delle PII deve basarsi su conoscenze preconfigurate — modelli pre-addestrati, regole predefinite, dizionari standard — sufficienti a garantire un livello di detection accettabile per le categorie canoniche della nLPD e del GDPR.

Il sistema deve tuttavia supportare meccanismi di personalizzazione facoltativa, attivabili dopo il primo impiego:

- dizionari specifici per il framework normativo di riferimento;
- specializzazioni basate su documenti rappresentativi dell'organizzazione, per migliorare la precisione del rilevamento sulla terminologia e sui formati propri del dominio aziendale.

2.4 Requisiti di qualità

I requisiti di qualità definiscono le proprietà non funzionali che il sistema deve soddisfare per essere considerato adeguato al contesto operativo previsto. Essi riguardano in particolare le prestazioni su volumi documentali realistici, l'affidabilità del rilevamento e la capacità di giustificare ogni anomalia segnalata.

2.4.1 Prestazioni su larga scala

Il sistema deve essere in grado di analizzare volumi documentali rappresentativi di un'organizzazione di medie dimensioni, nell'ordine delle migliaia di documenti, in tempi compatibili con un'esecuzione periodica non presidiata.

L'architettura deve prevedere meccanismi di parallelizzazione dell'elaborazione e, ove necessario, di analisi incrementale, in modo da evitare la rielaborazione integrale del corpus documentale a ogni esecuzione. Il tempo di elaborazione deve scalare in modo al più lineare rispetto al numero di documenti analizzati.

2.4.2 Precisione del rilevamento

Il sistema deve privilegiare un elevato valore di *recall*, al fine di minimizzare il rischio che dati personali effettivamente presenti nei documenti non vengano segnalati: un falso negativo equivale, in questo contesto, a una non-conformità non rilevata, con potenziali conseguenze sanzionatorie per l'organizzazione.

Al contempo, il tasso di falsi positivi deve essere mantenuto a un livello tale da non compromettere l'usabilità del report per il DPO, poiché un numero eccessivo di segnalazioni errate riduce la fiducia nel sistema e ne vanifica l'utilità operativa.

Le soglie di *precision* e *recall* accettabili per ciascuna categoria di PII sono definite in fase di validazione sperimentale (cfr. Capitolo 8).

2.4.3 Tracciabilità delle anomalie

Il sistema deve rendere ogni anomalia segnalata nel report tracciabile fino alla sua origine, associandole:

- il documento sorgente, identificato da percorso, nome del file ed eventuale impronta crittografica;
- la posizione specifica all'interno del documento, ove applicabile;
- la regola o il modello che ha generato il rilevamento, con la categoria di PII attribuita;
- l'attività di trattamento del ROPA a cui l'anomalia è stata associata, o l'indicazione esplicita di assenza di associazione.

La tracciabilità è condizione necessaria per consentire al DPO di verificare ciascuna segnalazione e, ove necessario, di utilizzare il report come evidenza documentale nell'ambito delle procedure di *accountability* richieste dal GDPR (art. 5(2)) [6] e dalla nLPD [7].

2.5 Requisiti di interfaccia

2.5.1 Interfaccia web come modalità primaria

Il sistema deve esporre un'interfaccia web come modalità primaria di interazione, servita dal sistema stesso su una porta di rete e raggiungibile da browser senza installazione lato client. Il destinatario è il DPO, figura non necessariamente tecnica, e il sistema è installato come unità autocontenuta e avviabile con un numero minimo di passaggi (cfr. 2.3.3): un'interfaccia servita dall'unità stessa è operabile senza accesso alla shell della macchina che la ospita e senza conoscenza dei comandi, a differenza di una CLI.

L'interfaccia deve consentire almeno le seguenti operazioni:

- avviare l'analisi di una cartella, indicandone il percorso o selezionandola dal browser, con anteprima dei file che verranno analizzati e visualizzazione dello stato di avanzamento;
- fornire il ROPA in input e revisionare le categorie di dati riconosciute (cfr. 2.1.4);
- consultare il registro delle PII rilevate e il verdetto di conformità per ciascun documento (cfr. 2.1.8);
- gestire l'associazione tra documenti e attività di trattamento (cfr. 2.1.5).

Il requisito si considera soddisfatto quando, a partire dal solo avvio del sistema, un utente è in grado di importare un ROPA, avviare una scansione e consultare il verdetto di conformità senza ricorrere alla riga di comando.

2.5.2 Interfaccia a riga di comando (CLI)

Il sistema deve esporre, come modalità secondaria, interfacce a riga di comando destinate all'automazione e alla diagnosi. La CLI è il punto di aggancio per l'esecuzione periodica non presidiata (cfr. 2.2.4) e per l'ispezione dei singoli stadi dell'elaborazione, e deve consentire almeno:

- l'ingestione di un ROPA e la mappatura delle sue categorie;
- la scansione di un singolo documento o di un albero di cartelle, con aggiornamento del registro interno;
- la verifica di conformità e la stampa del relativo verdetto.

Non è richiesto che tali operazioni siano esposte da un unico comando orchestratore: è ammessa una CLI per stadio dell'elaborazione, purché ciascuna sia componibile in uno script di schedulazione. L'interfaccia deve produrre messaggi di log progressivi che consentano di monitorare l'avanzamento e di diagnosticare gli errori.

2.5.3 Fruizione del report

Il sistema deve rendere il report di conformità (cfr. 2.1.7) fruibile in due modi complementari:

- **consultazione interattiva:** l'interfaccia web (cfr. 2.5.1) deve presentare il report al DPO in forma navigabile — elenco dei documenti analizzati, dettaglio delle PII rilevate e verdetto per documento — senza richiedere un passo di esportazione né strumenti esterni;
- **lettura meccanica:** il contenuto del report deve poggiare sul registro interno persistente (cfr. 2.1.8), in un formato strutturato e interrogabile, così da restare accessibile agli strumenti di governance, risk e compliance già in uso nell'organizzazione.

Il report non è un artefatto separato ma una *vista* sui dati che il sistema già mantiene: il registro è la fonte di verità, l'interfaccia ne è la presentazione. Un file di report generato a fine analisi sarebbe invece una fotografia che invecchia — disallineata alla prima ri-scansione — e duplicherebbe in un secondo formato informazioni già persistite, con l'onere di mantenerli coerenti. La consultazione dal browser garantisce inoltre che il DPO veda sempre lo stato corrente del corpus documentale, e l'assenza di file di report esportati è coerente con il vincolo di minimizzazione (cfr. 2.3.5): non si moltiplicano le copie del quadro delle PII dell'organizzazione.

L'esportazione del report in file autonomi — strutturati per l'integrazione con sistemi di governance, o impaginati per la presentazione in sede di audit — non rientra nel perimetro del prototipo: è una resa aggiuntiva sopra i medesimi dati, che non aggiunge capacità di analisi né di verifica.

Il requisito si considera soddisfatto quando, al termine di un'analisi di test, il DPO può consultare dal browser l'elenco dei documenti analizzati e il verdetto di ciascuno, e il medesimo contenuto informativo è ricavabile dal registro interno mediante interrogazione diretta, senza passare dall'interfaccia.

Capitolo 3

Stato dell'arte

Questo capitolo ricostruisce il panorama in cui il lavoro si colloca. Si apre con il vocabolario tecnico necessario alla lettura dei capitoli successivi; prosegue con le due difficoltà che caratterizzano il problema, l'ambiguità del testo non strutturato e la verifica automatizzata della conformità; passa quindi in rassegna gli approcci al rilevamento delle PII e i sistemi, open-source e commerciali, oggi disponibili; si chiude individuando il divario che il progetto intende colmare.

3.1 Background tecnologico e terminologia

Il rilevamento delle PII attinge a famiglie di tecniche di apprendimento automatico che ricorrono in tutto il documento. Questa sezione ne fissa il vocabolario — i paradigmi di addestramento, il rapporto fra pre-addestramento e specializzazione, la capacità di operare su categorie mai osservate — e le metriche con cui i risultati vengono confrontati, così che i capitoli successivi possano impiegarli senza doverli ridefinire.

3.1.1 Supervised vs unsupervised

Come spiegato in uno dei testi cardine del machine learning, redatto da Goodfellow et al. [23, § 5.1], queste due metodologie di apprendimento si distinguono in base al tipo di *esperienza* che l'algoritmo acquisisce a partire dai dati durante l'addestramento. La medesima distinzione è alla base della letteratura di riferimento del settore [24, 25]. Le due definizioni che seguono ne riprendono la trattazione:

- per **Supervised learning** si intende quel paradigma di apprendimento automatico in cui l'algoritmo viene addestrato su un insieme di dati in cui ogni esempio, oltre alle sue caratteristiche, è associato a un'etichetta o a un valore target noto a priori. Il modello apprende quindi una funzione che mappa le caratteristiche di input all'etichetta corrispondente, sulla base di esempi già classificati o annotati. Un caso tipico è quello

di un classificatore addestrato a riconoscere la specie di appartenenza di un fiore a partire da misurazioni morfologiche etichettate [23, § 5.7].

- per **Unsupervised learning** si intende quel paradigma in cui l'algoritmo opera su un insieme di dati privo di etichette, con l'obiettivo di individuare autonomamente strutture, regolarità o proprietà latenti presenti nei dati stessi. Nel contesto del deep learning, questo si traduce spesso nel tentativo di apprendere — in modo esplicito (ad esempio tramite stima di densità) o implicito (ad esempio per compiti di generazione o denoising) — la distribuzione di probabilità sottostante ai dati [23, § 5.8]. Altri algoritmi non supervisionati svolgono invece compiti diversi, come il clustering, che consiste nel raggruppare gli esempi del dataset in base alla loro similarità [24, § 9.1].

Storicamente i modelli impiegati per l'estrazione di informazioni si sono divisi lungo questa linea, e il confronto fra i due paradigmi è documentato in termini di scalabilità: i modelli supervisionati garantiscono prestazioni superiori sui pattern noti, ma richiedono onerose fasi di etichettatura manuale per adattarsi a nuovi scenari [26].

Nel contesto specifico della NER, l'approccio supervisionato conserva un vantaggio di accuratezza sulle alternative prive di annotazione, ma a un costo che non è di calcolo bensì di **etichettatura**: la letteratura di rassegna individua nella disponibilità di un corpus annotato di dimensioni adeguate, e non nella potenza di inferenza, l'ostacolo principale all'adozione [27, 28]. Il guadagno è inoltre distribuito in modo diseguale fra le categorie: è massimo proprio su quelle che gli approcci privi di addestramento coprono peggio — le entità sprovviste di un formato riconoscibile — e trascurabile su quelle già ben servite da regole deterministiche. I valori misurati che sostengono questa lettura sono riportati nel confronto empirico (cfr. 3.4.6).

3.1.2 Pre-training vs fine-tuning

Il paradigma pre-training/fine-tuning è stato reso centrale nell'NLP moderno dal lavoro di Devlin et al. [29], che lo applicano al modello BERT (Bidirectional Encoder Representations from Transformers). Le due fasi si articolano come segue:

- per **pre-training** si intende la fase iniziale di addestramento in cui il modello apprende rappresentazioni linguistiche generali a partire da grandi quantità di testo non etichettato, tramite obiettivi di tipo *self-supervised* (ad esempio indovinare delle parole mascherate casualmente all'interno di una frase). In questa fase il modello non è ancora specializzato su alcun compito specifico: l'obiettivo è costruire una rappresentazione del linguaggio sufficientemente generale per poter essere riutilizzata per molteplici task.
- per **fine-tuning** si intende la fase successiva, in cui i parametri del modello pre-addestrato vengono ulteriormente ottimizzati su un dataset etichettato e di dimensioni

ridotte, specifico per il compito che si intende risolvere. A differenza del pre-training, che richiede risorse computazionali ingenti, il fine-tuning può avvenire con un costo computazionale contenuto, poiché parte da una rappresentazione linguistica già appresa e la adatta al task target, spesso aggiungendo un solo livello di output specifico per il compito.

Applicato al rilevamento di PII, questo paradigma presenta due criticità che ne condizionano l'adozione. La prima è il **costo di annotazione**: l'adattamento richiede dataset etichettati di dimensioni rilevanti, il che nel caso delle PII comporta l'intervento manuale di esperti e solleva problemi di riservatezza intrinseci alla manipolazione di dati reali [28]. La seconda è la **perdita di generalizzazione** (*catastrophic forgetting*): una specializzazione spinta su un singolo formato documentale può compromettere la capacità del modello di operare su testi imprevisti [30].

Entrambi i vincoli si stringono negli ambienti operativi ristretti, dove l'inferenza va eseguita in locale su hardware limitato e i costi di esecuzione in memoria e latenza diventano essi stessi un vincolo di progetto [12]: in tali condizioni ogni ciclo di riaddestramento è particolarmente oneroso, e coprire una nuova categoria di dati non dovrebbe richiederne uno.

3.1.3 Zero-shot learning

Il concetto di zero-shot learning è stato formalizzato in modo sistematico da Xian et al. [31], che lo definiscono come quel paradigma di apprendimento in cui un modello viene valutato su classi (o, nel caso della Named Entity Recognition, su tipologie di entità) che non erano presenti nel suo insieme di addestramento.

A differenza dei paradigmi supervised e unsupervised descritti in 3.1.1, lo zero-shot learning non riguarda la presenza o assenza di etichette nei dati di training, bensì la capacità del modello di generalizzare a categorie mai osservate, sfruttando informazioni semantiche ausiliarie (ad esempio descrizioni testuali delle classi target) anziché esempi annotati specifici per quelle categorie. Tale capacità si distingue dal *one-shot* e dal *few-shot learning*, in cui il modello dispone rispettivamente di uno o di un numero ridotto di esempi annotati per le classi target [32].

Il meccanismo con cui questo paradigma viene realizzato nella NER è lo *span-matching*: architetture come GLiNER [33] ricevono in ingresso sia il documento da analizzare sia le descrizioni testuali delle entità da individuare (ad esempio «numero di previdenza sociale»), e calcolano la similarità semantica fra le porzioni di testo e la descrizione dell'etichetta cercata. Estendere le categorie riconosciute non richiede quindi un riaddestramento, ma l'aggiornamento della lista di istruzioni testuali fornite in input — proprietà che risponde direttamente ai vincoli di flessibilità e di riservatezza locale richiamati sopra. Le prestazioni effettivamente misurate su questa famiglia di modelli sono discusse nel confronto empirico (cfr. 3.4.6).

3.1.4 Metriche di valutazione: precisione, recall e F1

Valutare le prestazioni di un detector richiede un termine di paragone: un corpus in cui ogni occorrenza di dato personale sia già stata individuata e classificata, documento per documento. Questo insieme di annotazioni è detto **gold standard** — o, per abbreviazione, *gold* — e corrisponde all'esito che produrrebbe un rilevatore perfetto: è la risposta esatta contro cui viene confrontata quella automatica. Può essere ottenuto annotando manualmente testi reali secondo linee guida esplicite, come nel *Text Anonymization Benchmark* [9], oppure derivandolo dalla procedura che genera un corpus sintetico [34]. In entrambi i casi la sua qualità limita superiormente ciò che si può misurare: un rilevatore non può risultare più accurato dell'annotazione contro cui viene confrontato. Che il limite non sia teorico lo mostrano i dati dello stesso benchmark, dove l'accordo fra annotatori umani — corretto per il caso — vale $\kappa = 0.67$ sulla categoria semantica dell'entità, ma scende a $\kappa = 0.46$ sul tipo di identificatore e a $\kappa = 0.30$ sugli attributi confidenziali [9].

Il confronto fra detector — in questo capitolo come nei capitoli sperimentali — poggia su tre grandezze standard nella valutazione dei sistemi di estrazione di informazione. Dato l'insieme delle occorrenze registrate nel gold e l'insieme di quelle restituite dal rilevatore, ogni esito ricade in una di tre classi: i **veri positivi** (TP) sono le occorrenze segnalate correttamente; i **falsi positivi** (FP) sono le segnalazioni cui non corrisponde alcuna occorrenza attesa; i **falsi negativi** (FN) sono le occorrenze attese che il rilevatore non ha segnalato.

Da queste tre quantità discendono le metriche impiegate nel seguito:

- la **precisione** (*precision*), $P = TP / (TP + FP)$, è la quota di segnalazioni corrette fra quelle prodotte: misura quanto ci si possa fidare di ciò che il sistema riporta;
- il **recall**, $R = TP / (TP + FN)$, è la quota di occorrenze attese effettivamente individuate: misura quanto, di ciò che esiste nel documento, il sistema riesce a trovare;
- la **misura F1**, $F_1 = 2PR / (P + R)$, è la media armonica delle due e viene impiegata quando serve un unico valore di confronto. La media armonica penalizza gli squilibri assai più della media aritmetica: un rilevatore con precisione 1.00 e recall 0.10 ottiene $F_1 = 0.18$, non 0.55.

Le due grandezze sono in tensione fra loro: un rilevatore diventa più preciso segnalando solo ciò di cui è certo, al prezzo di lasciarsi sfuggire i casi dubbi, e aumenta il recall segnalando in modo più generoso, al prezzo di falsi positivi. Il punto di equilibrio non è però universale, ma dipende dal costo relativo dei due errori nel dominio applicativo. Nella verifica di conformità l'asimmetria è marcata: un dato personale non rilevato è una violazione che nessuno vedrà, mentre una segnalazione errata costa a chi la esamina il tempo di scartarla.

3.2 Ambiguità contestuale e frammentazione dei domini

Sul piano tecnico, la difficoltà del testo non strutturato non consiste nel *leggere* il documento, ma nel disambiguare le informazioni in base al contesto in cui compaiono. Una medesima sequenza di caratteri può essere un identificativo personale o un codice gestionale a seconda di ciò che la circonda; un nome proprio può designare una persona fisica, una via o una ragione sociale. La formattazione, che rende affidabile il riconoscimento di un IBAN o di un codice fiscale, non offre alcun appiglio per queste categorie.

A questa difficoltà si somma quella che la letteratura descrive come problema del *domain-silo* [35]: i sistemi di rilevamento vengono addestrati e valutati su dati provenienti da un dominio specifico e degradano sensibilmente su testi eterogenei. È precisamente la condizione in cui si trova una condivisione documentale aziendale reale, dove contratti, verbali, corrispondenza, fogli di calcolo e manualistica convivono nella stessa cartella: un corpus che nessun singolo dominio di addestramento rappresenta.

3.3 La verifica automatizzata della conformità

Il confronto fra quanto un'organizzazione dichiara e quanto effettivamente tratta è, di per sé, un problema di ricerca autonomo. Amaral et al. [11] affrontano il caso duale a quello qui considerato — la verifica automatizzata, mediante tecniche di NLP, della conformità di un accordo sul trattamento dei dati (DPA) rispetto agli obblighi del GDPR — mostrando come un requisito normativo espresso in prosa possa essere reso in criteri verificabili meccanicamente.

Il presente lavoro sposta il termine di confronto: non l'informativa rispetto alla norma, ma il **contenuto documentale effettivo** rispetto al registro dei trattamenti dell'organizzazione. Il registro fornisce l'atteso — quali categorie di dati personali, per quali finalità, per quanto tempo — e il rilevamento fornisce l'osservato; la verifica consiste nel differenziale fra i due insiemi. Questa impostazione richiede che il registro, tipicamente redatto in prosa libera, sia reso confrontabile in maniera algoritmica: un'esigenza che ricade sull'ingestione del registro e sulla riconduzione delle categorie dichiarate a un vocabolario condiviso con il rilevatore.

3.4 Approcci tecnici di PII detection

Per rilevare le PII, esistono più metodi e strategie, tutte ampiamente discusse in studi accademici. Ognuna di queste ha dei punti di forza e di debolezza, infatti esiste una area di problemi dove ogni singola opzione diventa la migliore, riducendo o addirittura annullando gli effetti negativi.

Le soluzioni in questione sono:

3.4.1 Recognition rule-based

Approccio più semplice e rule-based. La detection delle PII è solamente affidata a regole di pattern-matching e meccanismi simili. Come si può già intuire, questo approccio, per alcune categorie di dati sensibili, è particolarmente efficace: la massima performance si ottiene quando si trattano dati strettamente formattati in determinati modi (IBAN, AVS, codice fiscale etc...) [27]. Non appena i testi diventano liberi e non strutturati, allora l'accuratezza crolla. [35, Sezione 5.2] Questo approccio tuttavia, ha dei problemi seri che non possono essere trascurati: categorie nettamente più difficili da trovare (i nomi propri, per esempio, si baserebbero su regole facilmente mal-interpretabili e con un alto numero di falsi positivi, come la sola lettera maiuscola) [36].

Vantaggio: più veloce in assoluto tra tutte le soluzioni [37].

3.4.2 Recognition con NLP - NER

I modelli di Natural Language Processing (NLP), tramite la Named Entity Recognition (NER), classificano le entità testuali analizzando la struttura logica della frase. Architetture basate su Transformers (es. BERT) superano i limiti delle regole, comprendono il contesto [29] e raggiungono accuratèzze fino al 99.55% [38].

Vantaggio: eccellente bilanciamento tra comprensione del testo ed efficienza computazionale [39].

Svantaggio: Rischio di calo prestazionale su dati aziendali particolarmente specifici senza fare fine-tuning su dataset simili [35].

3.4.3 Recognition con AI generativa

Ultima soluzione "pura" è l'utilizzo di LLM con AI generativa per l'individuazione di PII, approccio ampiamente documentato in letteratura [20, 35]. Il suo punto di forza è il *recall*: i modelli generativi raggiungono valori di recall superiori a quelli degli altri approcci, in particolare sulle PII che dipendono dal contesto [40, 35]. Questo vantaggio ha però un costo computazionale e di risorse superiore di ordini di grandezza rispetto ai modelli discriminativi, e va sostenuto a ogni interrogazione [39, 41, 42], spesso incompatibile con l'hardware ordinario. L'unico modo per contenere i tempi di esecuzione è ridurre la taglia dei modelli — ricorrendo a modelli più piccoli e alla quantizzazione [12, 13] — ma tale riduzione comporta un decadimento di accuratezza [43]: più si scende con la dimensione, più il vantaggio degli LLM si assottiglia rispetto agli approcci NLP-NER. Se il confronto regga ancora nella fascia di taglia sostenibile su CPU è una domanda empirica, affrontata sul corpus di questo lavoro nel Capitolo 8.

Tuttavia, gli LLM hanno un vantaggio unico per quanto riguarda un aspetto fondamentale nel nostro problema: la presenza nativa del 'contesto'. Infatti gli LLM possiedono una comprensione semantica avanzata del linguaggio naturale. Tramite il paradigma Zero-Shot,

identificano PII deducibili puramente dal contesto verbale (es. dedurre "minorenne" dalla frase "mio figlio, nato 3 anni fa") senza alcun addestramento. Inoltre, offrono interpretazioni del loro operato, agevolando gli audit di conformità [20].

3.4.4 Limitazioni nella soluzione

Fra gli approcci esaminati, il rilevamento basato su NLP offre il compromesso più favorevole: raggiunge una quota di riconoscimento elevata senza richiedere le risorse computazionali dell'AI generativa.

Il vantaggio si annulla però di fronte al vincolo di **zero training** (cfr. 2.3.7): il sistema deve essere operativo non appena installato, e raggiungere un livello di rilevamento accettabile senza una fase di addestramento sui dati dell'organizzazione. Le metodologie NLP–NER, nella loro forma supervisionata, poggiano invece proprio sulla conoscenza dei dati che dovranno analizzare.

3.4.5 Soluzioni ibride

Per soddisfare il vincolo di zero training, bilanciando l'elevata richiesta di risorse dell'AI generativa e la rigidità dei sistemi rule-based, i sistemi oggi in produzione adottano prevalentemente un'**architettura ibrida**, che combina regole deterministiche e riconoscimento statistico; Presidio ne è l'esempio più diffuso [44].

3.4.6 Confronto empirico degli approcci

Le sezioni precedenti presentano ciascun approccio in termini qualitativi. Per motivare la scelta architetturale in modo rigoroso, è necessario un confronto quantitativo basato su evidenze empiriche dalla letteratura recente. Le metriche considerate in questa sede sono quelle della *capacità di rilevamento*: precisione (P), recall (R), F1-score e necessità di dati annotati. Il *costo* di ciascun approccio — energetico ed economico — pesa in modo altrettanto determinante sulla scelta, ma costituisce un tema a sé ed è raccolto per intero nello studio dedicato del Capitolo 5: le due tabelle vanno lette in corrispondenza, riga per riga.

Prestazioni dei sistemi rule-based

Le espressioni regolari raggiungono alta precisione su pattern strutturati come gli indirizzi email, ma la precisione dei metodi basati su regole varia sensibilmente a seconda del tipo di dato e del corpus [36]. Tuttavia, il limite strutturale è la *brittleness*: i detectors rule-based non considerano il contesto del testo circostante e falliscono sistematicamente su PII contestuali [45]. Una rassegna sistematica sul riconoscimento di entità nominate conferma il quadro:

nei metodi rule-based la precisione è tipicamente alta, mentre il recall resta basso perché le regole sono calibrate su un dominio specifico, e introdurne di nuove ha un costo elevato [27].

Prestazioni dei modelli NER supervisionati

Con corpus annotati adeguati, i modelli encoder fine-tuned raggiungono le prestazioni più elevate in assoluto tra tutti gli approcci considerati. In un benchmark comparativo su documenti clinici, BERT, ModernBERT e ClinicalBERT raggiungono tutti $P = 0.99$ e $R = 0.99$ ($CIRE \approx 0.99$), con throughput di 1528–2446 parole al secondo [46]. Un modello DeBERTa fine-tuned sul dataset *ai4privacy* raggiunge $F1 = 0.9757$ su un benchmark PII multi-dominio [47].

Su documenti finanziari non strutturati (audit report, fatture), una pipeline ibrida NLP rule-based + ML + NER custom raggiunge $P = 94.7\%$, $R = 89.4\%$, $F1 = 91.1\%$ su dataset sintetici, con il 93% di accuratezza su documenti reali [48].

Il principale ostacolo non è il costo di inferenza (basso dopo il training) ma il costo di annotazione del corpus: le soluzioni supervisionate richiedono tipicamente un corpus annotato di grandi dimensioni [28]. Si tratta però di un costo da sostenersi *una tantum*, e non per ogni documento analizzato: la distinzione fra oneri *una tantum* e oneri ricorrenti è dirimente per la scelta architetturale ed è ripresa nel Capitolo 5 (cfr. 5.2).

Prestazioni dei modelli NER zero-shot

GLiNER (cfr. 3.1.3) usa un encoder bidirezionale (DeBERTa) con un meccanismo di span-extraction (individuare inizio e fine di una label, invece di processare parola per parola) prompt-style (inserire le etichette da estrarre come testo, senza averne di predefinite nell'architettura del modello): le label vengono fornite come input testuale e il modello identifica span corrispondenti nel documento senza retraining [33]. Test su dataset PII multi-domain mostrano GLiNER-base a circa l'81% di F1, superiore ad approcci spaCy o regex standard, con recall particolarmente elevato su ID numerici e date [49]. Il modello *gliner-pii-base-v1.0* raggiunge $F1 = 80.99\%$ ed è ottimizzato per applicazioni privacy-focused in cui i servizi NER cloud pongono rischi di conformità [50].

Tuttavia, la specificità di dominio rimane determinante anche per approcci zero-shot: su testo clinico, GLiNER scende a circa $F1 = 0.41$, non rilevando oltre metà delle entità PHI individuate dai modelli specializzati sul dominio [51]. Questo impone una valutazione empirica obbligatoria sul corpus aziendale target prima di escludere il fine-tuning.

Prestazioni dell'AI generativa

Su post di forum di discussione scritti da studenti di corsi online (MOOC), GPT-4o, LLaMA 3.3 70B e LLaMA 3.1 8B raggiungono tutti recall medio superiore a 0.90, ma la precisione resta il limite irrisolto: il migliore dei tre, GPT-4o, si ferma a $P = 0.579$, seguito da LLaMA 3.3 a 0.506

e LLaMA 3.1 a 0.262. La causa è la tendenza a segnalare come PII nomi propri e località che non lo sono [40]. Su benchmark strutturati multi-task (PII-Bench), GPT-4o con metodi avanzati (Self-CoT, Self-Consistency) raggiunge $F1 \approx 0.76\text{--}0.77$, con Llama 3.1 e Qwen 2.5 in linea [52].

Il limite decisivo per il contesto di questa tesi non è però l'accuratezza, bensì il **costo ricorrente** dell'inferenza generativa: un onere che, a differenza dell'addestramento di un encoder, va sostenuto per ogni documento e a ogni nuova analisi del corpus. Poiché è questo il fattore che determina il ruolo assegnabile all'AI generativa nel sistema, il confronto quantitativo — energetico ed economico — è raccolto nello studio dedicato del Capitolo 5.

Convergenza verso la pipeline ibrida

La convergenza della letteratura recente indica la pipeline ibrida come soluzione ottimale per il rilevamento PII in contesti di compliance. Il sistema RECAP (workshop NeurIPS 2025) combina regex deterministiche con LLM context-aware: una prima fase di rilevamento affida gli identificatori strutturati alle espressioni regolari e quelli non strutturati a un LLM impiegato in *zero-shot*; seguono tre fasi di raffinamento — disambiguazione multi-etichetta, consolidamento degli *span* e filtraggio contestuale — anch'esse affidate al modello generativo. Nei suoi test, RECAP riporta un *weighted F1-score* superiore dell'82% ai modelli NER fine-tuned e del 17% agli LLM impiegati in zero-shot. Entrambi i valori sono miglioramenti **relativi** misurati in quel regime, nel quale le *baseline* sono assai più deboli di quelle dei benchmark ad alta disponibilità di risorse riportati sopra: non sono quindi confrontabili con i valori assoluti della Tabella 3.1 [17].

Un sistema ibrido indipendente (CHPDA) documenta lo stesso principio: le regex eccellono su pattern chiaramente definiti ma mancano di flessibilità per gestire complessità contestuali; i modelli NER eccellono nell'identificare entità ambigue come nomi e indirizzi, difficili da catturare con regex, ma producono falsi positivi senza pattern guida. La combinazione risolve i limiti di entrambi [37].

La letteratura non esclude l'AI generativa in assoluto, ma ne circoscrive il ruolo: l'impiego su casi ambigui, come supervisore di una pipeline ibrida, è la configurazione che i sistemi recenti adottano [17]. Che sia anche la più conveniente sul piano del costo non è misurato da questi lavori, ma discende dal divario di costo unitario fra approcci discriminativi e generativi raccolto nel Capitolo 5. L'esclusione riguarda l'inferenza sistematica su tutto il corpus, non l'impiego selettivo su un sottoinsieme di casi irrisolti.

Matrice di confronto sintetica

La Tabella 3.1 riassume le evidenze empiriche discusse nelle sezioni precedenti. Il confronto sui costi energetici ed economici dei medesimi approcci è riportato separatamente nella Tabella 5.1.

Tabella 3.1: Prestazioni di rilevamento degli approcci di PII detection secondo le evidenze di letteratura. Precisione e recall sono espressi sulla scala ordinale riportata nella legenda; i valori misurati da cui ciascun livello discende sono richiamati nelle note.

Approccio	P	R	F1 tipico	Corpus annotato
Regex / rule-based	Alta	Bassa	0.60–0.75 ^a	No
NER supervisionato	Alta	Alta	0.94–0.99 ^b	Sì, esteso
NER zero-shot (GLiNER)	Media	Media	~0.81 ^c	No
LLM generativo (GPT-4o)	Bassa	Alta	0.70–0.77 ^d	No
Pipeline ibrida	Alta	Alta	~0.91 ^e	Opzionale

Legenda dei livelli: Alta ≥ 0.85 ; Media 0.70–0.85; Bassa < 0.70 .

^a Le fonti caratterizzano l'approccio in termini qualitativi — precisione elevata sui pattern strutturati, recall limitato dalla dipendenza da regole specifiche di dominio — senza riportare valori aggregati [36, 27]. ^b P = 0.99 e R = 0.99 su documenti clinici [46]; F1 = 0.9757 su benchmark PII multi-dominio [47]. L'annotazione di un corpus di dimensioni adeguate è il principale ostacolo all'adozione [28]. ^c F1 ≈ 0.81 su benchmark PII multi-dominio [49, 50]; nessuna delle due misure proviene da una sede sottoposta a revisione tra pari, e i livelli di P e R sono ricavati dal solo F1. ^d R medio > 0.90 a fronte di P = 0.579 su post di forum di corsi online, per la tendenza a segnalare come PII nomi e località che non lo sono [40]; F1 ≈ 0.76 –0.77 su PII-Bench [52]. ^e P = 94.7%, R = 89.4%, F1 = 91.1% su documenti finanziari non strutturati [48]. L'annotazione è necessaria solo se la componente NER viene specializzata: la pipeline resta impiegabile anche interamente zero-shot [17].

3.5 Panoramica dei sistemi esistenti

L'analisi dello stato dell'arte evidenzia una netta separazione tra gli strumenti open-source, spesso circoscritti a un singolo compito tecnico — l'anonimizzazione, oppure la sola estrazione delle entità — e le suite commerciali, ricche di funzionalità ma proprietarie e con costi di licenza fuori scala rispetto al contesto qui considerato.

3.5.1 Sistemi open-source

- **Presidio** è un framework open-source originariamente sviluppato da Microsoft (ora passato vendor-free), diventato uno standard industriale per l'identificazione e l'anonimizzazione. Offre un'architettura modulare basata su classi estensibili come *EntityRecognizer* (per i modelli di Machine Learning) e *PatternRecognizer* (per il pattern matching classico) [53]. Il motore di rilevamento (*Analyzer*) è disaccoppiato dal componente di anonimizzazione (*Anonymizer*), include riconoscitori pre-costruiti anche per il contesto italiano (ad esempio il codice fiscale, con validazione a checksum) e consente di collegare motori NER esterni.

Il suo **limite rispetto allo scopo del progetto** non è tecnologico ma di *finalità*: nasce per la rilevazione e l'anonimizzazione del dato, non per la governance e la verifica di conformità. Non implementa alcuna logica di registro dei trattamenti (ROPA), di gap analysis o di reportistica per il DPO, e non copre nativamente identificatori del contesto svizzero (ad esempio il numero AVS). Il suo motore di detection resta però riutilizzabile

come componente di un sistema più ampio, che ne governi gli esiti anziché adottarne la finalità: è la forma di riutilizzo che questo lavoro considera.

- **GLiNER**: architettura Zero-Shot basata su Transformers, in grado di identificare PII passandole dinamicamente come query, senza addestramento. Le sue iterazioni recenti (GLiNER2-PII) mostrano prestazioni eccellenti in contesti multilingua [54].
Limiti: è esclusivamente un "motore algoritmico". Prende stringhe in input ed estrae etichette. Non implementa il parsing di documenti (PDF/Word), non genera reportistiche strutturate e non possiede alcuna logica di compliance (ROPA). [33]
- **Scrubadub / Datafog**: strutture open-source modulari. Scrubadub utilizza un'architettura a *Detector* multipli (Regex, detector predefiniti, NLP esterni) [55].
Limiti: sono fortemente legati al test-masking e all'oscuramento. Spesso si affidano pesantemente al pattern matching deterministico (Regex) (cfr:3.4.1), una tecnologia ormai insufficiente da sola per gestire documenti destrutturati e ambigui.

3.5.2 Sistemi commerciali

L'esame delle soluzioni commerciali non ha finalità comparative rispetto al sistema qui progettato, con il quale non condividono né scala né perimetro: serve a documentare quali funzionalità il mercato consideri costitutive di uno strumento di conformità, e a quali condizioni di adozione siano associate. Le piattaforme considerate integrano in un'unica architettura rilevamento, classificazione e gestione del dato, ma a costi e vincoli strutturali che ne rendono l'adozione difficilmente sostenibile per le piccole e medie imprese, o incompatibile con l'esecuzione interamente locale. Le prestazioni riportate nel seguito sono dichiarazioni dei rispettivi produttori e non misurazioni indipendenti: sono richiamate per documentare il posizionamento del prodotto, non come termine di paragone.

- **Varonis**: piattaforma di *Data Security Posture Management* (DSPM). Impiega un motore ibrido per il quale il produttore dichiara un'accuratezza del 98% su dati strutturati e non strutturati (SharePoint, OneDrive, NAS), senza addestramento sui dati del cliente [3]. Due elementi sono pertinenti in questa sede: la *User and Entity Behavior Analytics* (UEBA), che rileva accessi anomali ai file classificati come sensibili [56], e il ricorso a scansioni *incrementali* per contenere il costo delle rianalisi periodiche — la medesima esigenza che il presente lavoro registra fra i propri requisiti (cfr. 2.3.6).
Limiti: il prodotto è incentrato sul *controllo degli accessi* e sulla sicurezza delle reti aziendali, non sulla verifica del contenuto documentale rispetto a un registro dei trattamenti. Il costo di licenza, riportato attorno agli 80.000 CHF/anno [57], e l'architettura chiusa lo collocano su una scala di adozione diversa da quella qui considerata.
- **BigID**: piattaforma di *data privacy* e *data governance*; è, fra i sistemi esaminati, quello più vicino al problema affrontato in questa tesi. Il rilevamento è *identity-aware*: i dati

individuati non sono soltanto classificati per categoria, ma correlati a specifiche persone fisiche (*data subjects*) incrociando più sorgenti [58]; la priorità di scansione è determinata da un componente proprietario (*Hyperscan*). Rilevante in questa sede è soprattutto la gestione automatizzata del registro dei trattamenti: i dati rilevati vengono posti in corrispondenza con le attività di trattamento e con le politiche di conservazione dichiarate nel registro, fino alla cancellazione di quelli scaduti o ridondanti [2]. Il confronto fra dato rilevato e dato dichiarato risulta quindi una funzionalità già attesa dal mercato, e non un'ipotesi ancora da validare.

Limiti: l'offerta è di fascia *enterprise* e il listino non è pubblico: il prezzo viene definito caso per caso su richiesta. Diverse operazioni di classificazione poggiano su architetture cloud o ibride, che sollevano problemi di sovranità del dato incompatibili con il vincolo di mantenere l'intera inferenza all'interno dell'infrastruttura dell'organizzazione.

- **PII Tools:** soluzione verticale sul solo rilevamento delle PII. Adotta un approccio basato su apprendimento automatico che, secondo il produttore, riduce i falsi positivi delle espressioni regolari grazie a una maggiore considerazione del contesto [59]. Due caratteristiche la distinguono dalle precedenti: è distribuibile interamente *on-premise* in un contenitore, senza dipendenze da servizi remoti, e riporta la posizione del dato all'interno del documento anziché il solo esito della classificazione [4].

Limiti: resta un prodotto proprietario e non ispezionabile. Il modello impiegato non è sostituibile, e l'assenza di accesso al funzionamento interno impedisce sia di verificare come un esito sia stato prodotto, sia di estendere il rilevamento a categorie o ordinamenti non previsti dal fornitore.

3.6 Sintesi del gap e posizionamento del progetto

Alla luce di questa panoramica, il **gap nel mercato** risulta evidente. Le organizzazioni, in particolare le PMI svizzere, si confrontano con un panorama fortemente polarizzato:

1. **Soluzioni open-source:** Rilevano i dati in locale ma lo fanno con l'ottica prioritaria dell'*anonimizzazione* (Presidio), oppure offrono solo modelli potenti linguistici disconnessi da qualsiasi logica di governance e reportistica aziendale (GLiNER).
2. **Soluzioni Commerciali:** Dimostrano quali siano le reali necessità delle aziende, introducendo l'Identity-aware discovery, l'automazione del registro dei trattamenti, lo scanning incrementale e le dashboard per DPO. Tuttavia, lo fanno imponendo ecosistemi Black-box, costi proibitivi e massicce dipendenze da Cloud/API di terze parti, con conseguenti problematiche di *Data Sovereignty*.

3.6.1 Quadro comparativo

Le due famiglie non si distinguono per la qualità del rilevamento — su quel piano gli strumenti liberamente disponibili reggono il confronto con i prodotti a listino — bensì per l'**estensione della catena** che ciascuna copre. Il problema qui affrontato si articola infatti in cinque anelli: acquisire i documenti, rilevarvi le PII, ricondurre il rilevato a quanto il registro dei trattamenti dichiara, valutare la conservazione dei documenti che le contengono e restituire un esito al DPO. I motori open-source coprono il secondo anello e lo lasciano isolato; le suite commerciali li coprono tutti, ma a condizioni di adozione — prezzo, opacità del funzionamento, dipendenza da infrastrutture remote — che il contesto considerato non può sostenere. La Tabella 3.2 mette i due gruppi sullo stesso piano, insieme al profilo che il sistema oggetto di questa tesi deve presentare.

Tabella 3.2: Copertura funzionale e condizioni di adozione dei sistemi esaminati. Le righe sono formulate come feature richieste nel contesto d'impiego considerato (Capitolo 2); le caselle riprendono quanto documentato nel §3.5. L'ultima colonna non riporta misure, ma le proprietà *richieste* al sistema da realizzare.

Proprietà	Open-source			Commerciali			Questo lavoro
	Presidio	GLiNER	Scrubadub	Varonis	BigID	PII Tools	
Rilevamento di PII in testo non strutturato	✓	✓	✓	✓	✓	✓	✓
Acquisizione ed estrazione da formati documentali (PDF, Office)	✗	✗	✗	✓	✓	✓	✓
Riconduzione del rilevato al registro dei trattamenti	✗	✗	✗	✗	✓	✗	✓
Verifica dei termini di conservazione dichiarati	✗	✗	✗	✗	✓	✗	✓
Reportistica di conformità destinata al DPO	✗	✗	✗	✓	✓	✓	✓
Rianalisi incrementale del corpus	✗	✗	✗	✓	(✓)	n.d.	✓
Esecuzione interamente locale, senza dipendenze remote	✓	✓	✓	n.d.	✗	✓	✓
Ispezionabilità del funzionamento e degli esiti	✓	✓	✓	✗	✗	✗	✓
Estensione a nuove categorie senza riaddestramento né intervento del fornitore	✓	✓	✓	n.d.	n.d.	✗	✓
Costo di licenza (ordine di grandezza)	Nulla	Nulla	Nulla	~80k CHF/a.	Su richiesta	Su richiesta	Nulla

Legenda: ✓ proprietà presente; (✓) presente in forma parziale o indiretta; ✗ assente; n.d. non documentata pubblicamente.

I motori open-source operano su testo già estratto: l'acquisizione documentale ricade sul sistema che li integra [53, 33, 55]. La rianalisi incrementale è dichiarata da Varonis [3], mentre in BigID il componente proprietario *Hyperscan* governa la priorità di scansione, non l'omissione dei documenti invariati [60]. La corrispondenza fra dati rilevati, attività di trattamento e politiche di conservazione è documentata per il solo BigID [2], che però esegue parte della classificazione su architetture cloud o ibride [58]. PII Tools è distribuibile on-premise in un contenitore, ma resta non ispezionabile e non estendibile a categorie non previste dal fornitore [4, 59]. Il valore di licenza di Varonis è una stima di mercato [57]; per BigID e PII Tools il listino non è pubblico [61]. Il costo di licenza nullo non esaurisce il costo di adozione, che per gli strumenti eseguiti in proprio comprende l'esercizio dell'infrastruttura: il confronto energetico ed economico fra gli approcci di rilevamento è raccolto nel Capitolo 5 (Tabella 5.1).

Letta per colonne, la tabella dice che nessuno strumento privo di costi di licenza oltrepassa il secondo anello della catena, e che l'unico prodotto a coprirlo per intero è anche quello che ne colloca una parte fuori dall'infrastruttura dell'organizzazione, a un prezzo non pubblico. Letta per righe, mostra che ciò che manca agli strumenti liberi non è capacità di *rilevamento* — che possiedono — ma capacità di *governo* del rilevato: il confronto con il dichiarato, il controllo della conservazione, l'esito destinato a chi risponde della conformità. È una lacuna di finalità, non di tecnologia, e come tale colmabile costruendo sopra i motori esistenti anziché in loro sostituzione.

Posizionamento del progetto Il software proposto si inserisce al centro di questo divario. L'obiettivo è fornire uno strumento open-source ed eseguibile **esclusivamente in locale** (on-premise), rispondendo all'esigenza di confidenzialità totale. Abbandonando il paradigma dell'anonimizzazione, il sistema non si propone di rimuovere il dato personale, ma di stabilire se esso possa legittimamente trovarsi dove si trova: il rilevamento cessa di essere il fine e diventa il presupposto della verifica. A questo scopo il riconoscimento poggia su motori NLP Zero-Shot (cfr. 3.1.3), che eliminano i costi di addestramento discussi in precedenza. Il sistema fungerà così da ponte tra il dato reale e le policy aziendali (ROPA), fornendo un quadro clinico dell'aderenza normativa aziendale.

Sintesi Ad oggi, c'è la mancanza strumento che tenga insieme le tre proprietà che i due gruppi si dividono: la gratuità e l'ispezionabilità del primo, la copertura dell'intera catena di conformità del secondo, e l'esecuzione interamente locale che il trattamento di dati personali impone. È la colonna che la Tabella 3.2 riserva a questo lavoro, e che il Capitolo 2 traduce in requisiti verificabili.

Capitolo 4

Analisi dei rischi

4.1 Rischi di progetto

4.1.1 Disponibilità e qualità dei dataset di valutazione

La letteratura sul trattamento delle PII è orientata in prevalenza all'anonimizzazione [8], e i corpus pubblici disponibili sono costruiti per quel fine [9]: scarseggiano insiemi di valutazione pensati per la sola *detection*, annotati in modo esaustivo e liberamente utilizzabili. Il problema è noto e non specifico di questo lavoro: il principale ostacolo alle soluzioni supervisionate è riconosciuto proprio nel costo di costruzione di un corpus annotato di dimensioni adeguate [28]. A ciò si aggiunge un vincolo proprio di questo progetto: un corpus di dati reali sarebbe esso stesso un archivio di dati personali, in contraddizione diretta con il principio di minimizzazione che il sistema è chiamato a rispettare (cfr. 2.3.5).

Se il rischio si materializza, il sistema resta privo di una base di misura: le prestazioni del rilevamento non sarebbero verificabili in modo riproducibile e ogni scelta architeturale — quale rilevatore adottare, se l'AI aggiunga valore — si ridurrebbe a una preferenza soggettiva, indebolendo l'intero impianto argomentativo della tesi.

La contromisura prevista è la generazione di corpus **sintetici** e riproducibili a partire da un seme — pratica consolidata quando i dati reali non sono utilizzabili, e oggetto essa stessa di studio nella costruzione di insiemi annotati per il riconoscimento di entità [34]: i valori devono essere realistici e, dove il formato lo prevede, aritmeticamente validi, mentre il *gold* (cfr. 3.1.4) va derivato dalla stessa sorgente che produce il testo, così che annotazione e documento non possano divergere. La valutazione va inoltre articolata su più livelli di granularità, dal documento singolo all'albero documentale, con le soglie e i protocolli definiti in sede sperimentale (cfr. Capitolo 8). Resta un rischio residuo di cui tenere conto nella lettura dei risultati: un corpus sintetico è per costruzione più ordinato del reale, e i valori misurati su di esso vanno intesi come limite superiore delle prestazioni attese sul campo.

4.1.2 Complessità dell'integrazione multi-approccio

Il rilevamento non può essere affidato a una tecnica sola: il requisito impone la composizione di meccanismi complementari — espressioni regolari, riconoscimento di entità nominate e, in modo selettivo, modelli generativi (cfr. 2.1.3, 2.1.11) — ed è la direzione che la letteratura indica come standard (cfr. 3.4.5). Ciascuna sorgente produce però risultati in modo autonomo, con confini di testo e categorie che possono sovrapporsi, coincidere o contraddirsi.

Far convivere risultati eterogenei senza una disciplina esplicita produce duplicazione delle segnalazioni, esiti incoerenti sullo stesso frammento di testo e, soprattutto, l'impossibilità di attribuire un errore a uno stadio preciso: se il sistema sbaglia, non si saprebbe se la responsabilità sia del rilevamento, della fusione dei risultati o dell'estrazione a monte.

Il rischio va contenuto imponendo a tutte le sorgenti una **interfaccia comune**: qualunque sia la tecnica con cui opera, ciascuna deve restituire i propri risultati nella medesima forma — il tipo di dato personale, scelto da un catalogo condiviso; la posizione nel testo; il grado di confidenza; l'indicazione della sorgente che l'ha prodotto. Risultati espressi nella stessa forma sono confrontabili fra loro, e la riconciliazione può quindi concentrarsi in un solo punto, che li unifichi secondo un vocabolario chiuso di esiti e che, in caso di disaccordo sul tipo, conservi entrambe le ipotesi anziché sceglierne una.

Da questa disciplina discendono le tre risposte ai problemi enunciati sopra: le segnalazioni che descrivono lo stesso frammento sono riconoscibili come tali e vengono fuse anziché duplicate; l'incoerenza sul medesimo testo non resta implicita ma diventa un esito dichiarato; e poiché ogni rilevamento conserva la traccia delle sorgenti che l'hanno prodotto, come già impone il requisito di tracciabilità (cfr. 2.4.3), l'attribuzione di un errore a uno stadio preciso diventa un'operazione di lettura e non di ricostruzione.

4.1.3 Costo computazionale e tempi di elaborazione

L'analisi deve percorrere l'insieme di documenti dell'organizzazione che ricadono nel perimetro definito in sede di requisiti — i file non strutturati residenti su file system, non l'insieme delle sorgenti in cui l'organizzazione conserva dati personali (cfr. 2.1.1) — e il costo unitario dell'inferenza si moltiplica quindi per il numero di documenti. Il rischio è particolarmente acuto per i modelli generativi, il cui costo per interrogazione — in energia e in denaro — è superiore di ordini di grandezza a quello dei modelli discriminativi e va sostenuto *a ogni* nuova analisi, non una sola volta: il confronto quantitativo fra le due famiglie, con i dati di letteratura che lo sostengono, è riportato nel Capitolo 5.

Un sistema che richiedesse giorni o settimane per completare una passata non sarebbe semplicemente lento: sarebbe inutilizzabile nel modo in cui è pensato, ossia come verifica periodica e non presidiata (cfr. 2.4.1). Il vincolo, inoltre, non è solo di tempo ma di

dotazione hardware: richiedere acceleratori dedicati escluderebbe dall'adozione proprio le organizzazioni di piccole e medie dimensioni a cui il progetto si rivolge.

La risposta si articola su due fronti, entrambi già fissati in sede di requisiti. Sul piano della scelta tecnologica, l'analisi sistematica dell'intero corpus in perimetro va affidata ad approcci computazionalmente leggeri ed eseguibili su hardware ordinario, mentre l'impiego di modelli generativi resta circoscritto a sottoinsiemi mirati (cfr. 2.3.2, 2.1.11). Sul piano dell'esecuzione, le analisi successive alla prima devono rielaborare i soli documenti effettivamente modificati (cfr. 2.3.6), così che il costo a regime risulti proporzionale al cambiamento e non al volume complessivo.

4.1.4 Dipendenza da modelli e librerie di terze parti

Il sistema poggia su una catena di componenti esterni: il motore di rilevamento e i suoi riconoscitori, i modelli di riconoscimento di entità e l'infrastruttura che li distribuisce, i lettori dei diversi formati documentali e il runtime che esegue localmente il modello linguistico. Ciascuno di essi può introdurre modifiche incompatibili, essere abbandonato o adottare condizioni di licenza inadatte all'impiego previsto.

La forma più insidiosa di questo rischio non è però l'obsolescenza, ma l'**incompatibilità di piattaforma**: le librerie di apprendimento automatico vincolano in modo stringente versione dell'interprete, architettura del processore e sistema operativo, al punto che una dipendenza può risultare indisponibile non perché abbandonata, ma perché l'ambiente su cui si opera non ne soddisfa i vincoli. Un sistema destinato a infrastrutture aziendali eterogenee (cfr. 2.3.3) è esposto a questa eventualità per definizione.

La mitigazione è duplice. Le dipendenze più onerose vanno dichiarate come componenti opzionali e caricate solo nel momento in cui servono davvero, così che l'indisponibilità di una di esse non impedisca il funzionamento del resto del sistema; e l'esecuzione di riferimento va collocata in un ambiente isolato e riproducibile, che renda irrilevante la piattaforma dell'host (cfr. 2.3.3). Sul piano della progettazione, l'interfaccia comune imposta alle sorgenti di rilevamento (cfr. 4.1.2) e la definizione dichiarativa dei detector richiesta dal requisito di estensibilità (cfr. 2.3.4) fanno sì che la sostituzione di un componente resti circoscritta alla sua sede, senza propagarsi al resto dell'elaborazione.

4.1.5 Dilatazione del perimetro (*scope creep*)

Il dominio affrontato confina con almeno tre ambiti attigui e tutti pertinenti: l'anonimizzazione dei documenti, la gestione degli incidenti di sicurezza e la valutazione d'impatto sulla protezione dei dati. Ciascuno di essi appare, preso singolarmente, come un'estensione naturale — il sistema individua già i dati personali, sembrerebbe ragionevole che li mascherasse anche — e la loro somma eccede di gran lunga ciò che un lavoro di tesi può portare a compimento.

Il rischio non è di realizzare la funzionalità sbagliata, ma di realizzarne troppe in modo superficiale: un prototipo che tocca molti ambiti senza approfondirne alcuno non consente né una valutazione significativa né una conclusione difendibile, e la dispersione si paga sull'unica parte che dovrebbe essere solida, ossia la verifica di conformità.

La contromisura è il governo esplicito del perimetro. Le esclusioni sono dichiarate nei requisiti in forma altrettanto vincolante delle inclusioni (cfr. 2.2.4, 2.2.5), e ogni funzionalità che ecceda la verifica documentale va registrata come sviluppo futuro anziché essere assorbita in corso d'opera. Il criterio di ammissione adottato è la riconducibilità al confronto fra dato rilevato e dato dichiarato: ciò che non vi contribuisce resta fuori, per quanto affine.

4.1.6 Competenze richieste e curva di apprendimento

Il progetto richiede la padronanza simultanea di due domini distanti fra loro: da un lato l'elaborazione del linguaggio naturale, con le famiglie di modelli, i paradigmi di addestramento e le metriche che le governano; dall'altro il quadro normativo sulla protezione dei dati, in due ordinamenti che non coincidono. Nessuno dei due può essere trattato superficialmente senza compromettere il risultato: un errore di interpretazione normativa produrrebbe un sistema che verifica la cosa sbagliata, un errore di valutazione dei modelli produrrebbe una scelta tecnica indifendibile.

La manifestazione tipica del rischio è la decisione presa per inerzia: adottare la prima tecnologia incontrata perché documentata meglio, o assumere un'interpretazione normativa perché più semplice da implementare, rendendo di fatto arbitraria la parte più qualificante del lavoro.

La contromisura agisce su due versanti. Sul versante tecnico, ogni scelta rilevante va fondata su prove — misure proprie o dati di letteratura — anziché su preferenze, e il confronto fra le alternative va documentato insieme al suo esito (cfr. Capitolo 3, Capitolo 5). Sul versante normativo, il sistema è deliberatamente costruito per *non* pronunciarsi sulle questioni che richiedono una valutazione giuridica (cfr. 2.2.5): le assunzioni normative sono ridotte al minimo indispensabile e il giudizio resta all'organizzazione, il che circoscrive il danno di un'eventuale interpretazione imprecisa.

4.2 Rischi di prodotto

4.2.1 Accuratezza insufficiente nella detection delle PII

Il sistema non modifica i documenti: il suo esito è una segnalazione destinata a un giudizio umano. L'intero valore del prodotto dipende perciò dall'attendibilità di ciò che segnala, e un rilevamento impreciso non abbassa semplicemente la qualità del risultato — ne annulla la funzione, perché un report di cui non ci si può fidare non viene usato.

Il rischio è aggravato dal fatto che l'accuratezza non è una grandezza uniforme: le sorgenti di rilevamento hanno profili complementari e molto diversi fra loro. Gli identificatori a formato rigido sono riconoscibili con precisione prossima al massimo, mentre le categorie che solo il contesto rivela — nomi di persona, indirizzi — dipendono interamente dalla qualità del modello di riconoscimento impiegato, le cui prestazioni variano sensibilmente fra le famiglie disponibili (cfr. 3.4.6). Una misura aggregata, quindi, nasconderebbe più di quanto riveli.

La contromisura è misurare in modo **disaggregato per categoria** e su un protocollo riproducibile (cfr. Capitolo 8), assumendo la copertura delle singole categorie — e non la media — come criterio di accettabilità: una categoria non coperta è un difetto anche quando l'indicatore complessivo è buono, perché corrisponde a una classe di dati personali che il sistema non vede affatto.

4.2.2 Falsi positivi e impatto sull'usabilità

Una segnalazione errata ha un costo che non ricade sul sistema ma sull'operatore: ogni falso positivo si traduce in una verifica manuale infruttuosa. Poiché il tempo del DPO è la risorsa più scarsa dell'intero processo, un tasso elevato di segnalazioni errate non rende il report semplicemente meno preciso, ma progressivamente inutilizzato — l'esito peggiore per uno strumento di conformità, perché indistinguibile dall'assenza dello strumento stesso.

Il rischio si concentra sui documenti privi di dati personali ma ricchi di sequenze che vi somigliano — numeri di protocollo, importi, codici di articolo — e sulle categorie prive di un formato che le vincoli, che sono anche le più esposte a un riconoscimento fondato sul solo contesto. Non è la fragilità di un singolo approccio: i rilevatori basati su espressioni regolari producono falsi positivi sistematici, con una precisione che per alcuni tipi di dato risulta molto bassa [36], e i modelli generativi, pur raggiungendo un *recall* elevato, mostrano precisioni sensibilmente inferiori [40]. È inoltre un rischio che nessun corpus composto di soli documenti «popolati» potrebbe rilevare: senza documenti negativi, i falsi positivi sono per costruzione invisibili alla misura.

Per questa ragione il corpus di valutazione deve comprendere una quota deliberata di documenti **privi di qualsiasi PII**, corredati di elementi distrattori, così che la precisione sia una grandezza effettivamente misurata e non presunta (cfr. Capitolo 8). Sul piano della presentazione dei risultati, ogni rilevamento deve esporre il proprio grado di conferma e le sorgenti che lo hanno prodotto (cfr. 2.4.3), consentendo al DPO di affrontare per prime le segnalazioni più solide anziché scorrere un elenco indifferenziato.

4.2.3 Falsi negativi e mancata rilevazione di PII critiche

Un dato personale presente e non rilevato non produce alcun segnale: è un rischio che resta *invisibile* tanto al sistema quanto all'organizzazione, e che si manifesta solo quando è troppo

tardi per rimediare. L'asimmetria rispetto al rischio precedente è netta: un falso positivo costa una verifica, un falso negativo costa una non-conformità non rilevata, con conseguenze tanto più gravi quando riguarda le categorie particolari di dati previste dall'art. 9 GDPR [6] e dall'art. 5 lett. c nLPD [7] (cfr. 2.2.3).

La conseguenza è che i due errori non possono essere trattati come equivalenti in un bilanciamento simmetrico: il sistema deve dichiarare quale dei due preferisce commettere.

L'impostazione assunta è **recall-oriented**, già fissata fra i requisiti di qualità (cfr. 2.4.2) e coerente con l'orientamento consolidato nella letteratura sull'anonimizzazione testuale, che individua nel richiamo la metrica decisiva e ne raccomanda la valutazione con l'F2 in luogo dell'F1 [9]. In concreto: nessuna sorgente deve scartare autonomamente un candidato debole, i casi di disaccordo vanno conservati anziché risolti eliminando un'ipotesi, e la decisione finale è rimessa all'operatore. È una scelta che aumenta deliberatamente i falsi positivi per ridurre i falsi negativi, e va letta congiuntamente al rischio precedente (cfr. 4.2.2): la tensione fra i due non è risolta a livello di algoritmo ma arbitrata dal requisito di precisione (cfr. 2.4.2), che assegna la priorità al *recall* entro un limite di segnalazioni errate compatibile con l'usabilità.

4.2.4 Generalizzazione cross-domain (*domain-silo problem*)

I sistemi di rilevamento addestrati o messi a punto su un dominio specifico perdono sensibilmente in accuratezza quando vengono applicati a testi di natura diversa, fenomeno documentato in letteratura come problema del *domain-silo* [35] (cfr. 3.2). Il contesto d'impiego previsto è però l'opposto di un dominio omogeneo: una condivisione documentale aziendale accosta contratti, corrispondenza, verbali, documentazione tecnica e fogli di calcolo, spesso nella stessa cartella.

Se il rilevamento fosse tarato implicitamente su una sola di queste famiglie, la copertura risulterebbe buona in fase di verifica e insoddisfacente sul campo, con un divario tanto più pericoloso quanto meno visibile: il sistema continuerebbe a produrre report apparentemente normali, semplicemente omettendo intere porzioni del patrimonio documentale.

La contromisura opera su due piani. Sul piano della scelta tecnologica, la preferenza per il riconoscimento *zero-shot* — in cui le categorie sono fornite come descrizioni testuali anziché apprese da un corpus di dominio — riduce alla radice la specializzazione implicita ed è del resto imposta dal requisito di operatività senza addestramento (cfr. 3.1.3, 2.3.7). Sul piano della verifica, il corpus di valutazione va costruito **deliberatamente eterogeneo** per genere documentale, lunghezza e presenza di dati personali. Il rischio residuo resta comunque significativo e va dichiarato. Su domini fortemente specialistici i modelli *zero-shot* subiscono cali marcati: applicati a testo clinico scendono a un F1 di circa 0.41, perdendo oltre metà delle entità rispetto ai modelli specializzati sul dominio [51]. Che la specializzazione resti necessaria lo conferma, indirettamente, l'esistenza di suite di modelli sviluppate appositamente per

singoli ambiti [62]. Ciò rende la valutazione empirica sul corpus effettivo dell'organizzazione un passaggio non eludibile prima dell'adozione.

4.2.5 Supporto multilingue e multilocalizzazione

Il contesto normativo di riferimento — svizzero ed europeo — implica la coesistenza di più lingue nello stesso patrimonio documentale, e con esse di identificatori nazionali differenti: il numero AVS svizzero, il codice fiscale italiano e i rispettivi omologhi seguono formati fra loro incompatibili. La dipendenza dalla lingua, inoltre, non riguarda solo il rilevamento: attraversa i modelli di riconoscimento, le espressioni regolari ancorate a parole chiave e persino l'interpretazione dei registri dei trattamenti.

La manifestazione tipica di questo rischio non è un errore visibile, ma un **silenzio**. Un componente che interpreta testo libero riconosce le formulazioni delle lingue per cui è stato predisposto e ignora le altre, senza poter distinguere l'espressione che non comprende da quella che non è presente: una durata di conservazione dichiarata nel registro in una lingua non prevista non verrebbe interpretata, il controllo corrispondente si disattiverebbe senza alcuna segnalazione, e un documento fuori termine risulterebbe conforme. È il tipo di errore più pericoloso proprio perché non produce alcun sintomo.

Va inoltre osservato che il trasferimento di un modello di de-identificazione da una lingua all'altra non è un'operazione neutra e costituisce un problema di ricerca a sé, affrontato in letteratura con tecniche di adattamento a pochi esempi [63]: assumere che un modello multilingue si comporti in modo uniforme su tutte le lingue che dichiara di coprire sarebbe imprudente.

Il rischio va mitigato mantenendo in **configurazione**, e non nel codice, sia il catalogo delle categorie sia le regole che le riconoscono, così che l'estensione a una lingua o a un ordinamento ulteriore sia un intervento dichiarativo (cfr. 2.3.4); e privilegiando, per il riconoscimento delle entità, modelli addestrati su più lingue. Resta il fatto che ogni componente che interpreta testo libero va verificato lingua per lingua: è una superficie di rischio che si riduce, ma non si elimina.

4.2.6 Scalabilità su filesystem di grandi dimensioni

L'analisi opera per attraversamento ricorsivo di interi alberi di cartelle, il cui contenuto non è noto in anticipo né per volume né per composizione. Il sistema deve quindi reggere corpus dell'ordine delle migliaia di documenti (cfr. 2.4.1), con dimensioni dei singoli file estremamente variabili e formati eterogenei, alcuni dei quali onerosi da interpretare.

I modi in cui questo rischio si manifesta sono tre, di gravità crescente: tempi di esecuzione incompatibili con una verifica periodica; consumo di memoria proporzionale alla dimensione del documento più grande incontrato; e, soprattutto, l'interruzione dell'intera analisi a causa

di un singolo file illeggibile o malformato — l'esito peggiore, perché fa dipendere il risultato complessivo dall'elemento meno affidabile del corpus.

Le contromisure riguardano il modo in cui l'elaborazione va organizzata. Ogni documento deve essere elaborato in **isolamento**: un file che non può essere letto va registrato come errore lasciando proseguire l'analisi, cosicché nessun singolo elemento possa comprometterla. I componenti costosi da inizializzare vanno predisposti una sola volta e riutilizzati per l'intero lotto, e il testo estratto va mantenuto in memoria per la sola durata dell'elaborazione del documento, come già impone il vincolo di minimizzazione (cfr. 2.3.5). Le esecuzioni successive alla prima devono infine rielaborare i soli documenti modificati (cfr. 2.3.6), il che rende il costo a regime proporzionale al cambiamento anziché al volume. Resta un rischio residuo sul fronte della parallelizzazione richiesta dal requisito di prestazioni (cfr. 2.4.1): su corpus molto ampi l'elaborazione incrementale riduce il lavoro ricorrente ma non quello della prima passata, per la quale la riduzione dei tempi passa dalla distribuzione del carico su più processi.

4.2.7 Qualità e completezza del ROPA in input

Il registro delle attività di trattamento è il termine di paragone rispetto al quale ogni verdetto viene formulato, ma è un documento redatto dall'organizzazione, spesso in prosa sbrigativa: categorie indicate con formule generiche («dati anagrafici»), durate di conservazione espresse come criterio anziché come termine, attività dichiarate in modo incompleto. Il sistema, in altre parole, dipende da un input che non controlla e che è tipicamente meno preciso di quanto il confronto richiederebbe.

Un registro povero non produce un errore evidente, ma un verdetto *silenziosamente* inaffidabile: le categorie non riconducibili ad alcun tipo rilevabile non parteciperebbero al confronto e le durate non interpretabili disattiverebbero il controllo di conservazione, con il risultato paradossale che un registro più vago genererebbe un esito apparentemente migliore — l'incentivo esattamente opposto a quello desiderabile.

Il principio adottato è che il sistema **non deve mascherare la povertà del registro, ma esporla**. Le categorie dichiarate che non si risolvono in alcun tipo rilevabile devono restare esplicite e venire segnalate al DPO anziché essere scambiate per conformi (cfr. 2.2.2); allo stesso modo, una conservazione espressa come criterio deve produrre una segnalazione di *non verificabilità* distinta dall'esito conforme (cfr. 2.2.1). La risoluzione delle categorie espresse in linguaggio naturale, infine, va sottoposta a conferma umana (cfr. 2.1.11), così che l'arricchimento del registro resti una decisione dell'organizzazione e non un'inferenza del sistema.

4.2.8 Conformità normativa e aggiornamento dei framework legali

Il sistema incorpora assunzioni normative: quali dati siano da considerare personali, quali godano di protezione rafforzata, quali obblighi il registro debba documentare. Tali assunzioni derivano da fonti in evoluzione — il Regolamento (UE) 2016/679 [6] e la legge federale svizzera sulla protezione dei dati [7] — e possono differire in ordinamenti ulteriori a cui un'organizzazione sia soggetta.

Il rischio è che il sistema invecchi senza darne segno: un catalogo di categorie fissato al momento della realizzazione diventerebbe progressivamente incompleto, continuando a produrre verdetti formalmente corretti ma fondati su una nozione di dato personale superata.

La contromisura è di progettazione prima che normativa. Il catalogo delle categorie di PII e le regole che le riconoscono devono essere **configurazione e non codice**, il che rende l'adeguamento un'operazione di manutenzione dichiarativa (cfr. 2.3.4). Soprattutto, il sistema non giudica rispetto a una lista di obblighi cablata al proprio interno, ma rispetto al *registro dell'organizzazione*: l'onere dell'aggiornamento normativo resta dove la norma lo colloca, sul titolare del trattamento, mentre al sistema compete verificarne la coerenza con i dati effettivamente presenti (cfr. 2.2.5). Resta un rischio residuo dichiarato: l'allineamento puntuale fra le due normative di riferimento, che divergono nella struttura oltre che nel dettaglio, non è oggetto di una mappatura formale.

4.2.9 Sicurezza e protezione dei dati durante l'analisi

Il sistema è esposto a un paradosso costitutivo: per verificare il trattamento dei dati personali deve accedervi, e nel farlo diventa a sua volta un trattamento. Ogni sottoprodotto dell'analisi — il testo estratto, il registro interno delle rilevazioni, il report per il DPO — può contenere dati personali, e uno strumento di conformità che diventasse la nuova concentrazione di dati sensibili dell'organizzazione avrebbe peggiorato la situazione che intendeva migliorare.

Il rischio riguarda due aspetti distinti: la *trasmissione* dei dati verso l'esterno, qualora l'elaborazione si appoggiasse a servizi remoti, e la loro *persistenza* all'interno del sistema, che moltiplicherebbe le copie da proteggere.

Sul primo fronte la contromisura è il vincolo di elaborazione interamente locale, che si estende ai modelli impiegati (cfr. 2.3.1, 2.3.2). Sul secondo, la minimizzazione va realizzata come proprietà **strutturale** e non come cautela procedurale: il registro deve conservare unicamente riferimenti — categoria, posizione, provenienza del rilevamento — e nessuna delle sue strutture dati deve prevedere una sede in cui il valore di una PII possa essere memorizzato, cosicché la violazione del principio non sia semplicemente sconsigliata ma impossibile per costruzione (cfr. 2.3.5, 2.1.8); il testo estratto deve vivere per la sola durata dell'elaborazione. Un rischio residuo permane e va riconosciuto: pur non contenendo valori,

il registro costituisce una *mappa* di dove i dati personali si trovino, informazione di per sé sensibile che va protetta con le stesse cautele riservate all'infrastruttura che sorveglia.

4.3 Valutazione dei rischi

4.3.1 Criteri di classificazione

I rischi individuati nelle sezioni precedenti sono classificati secondo due dimensioni — la **probabilità** che si manifestino nel corso del progetto e l'**impatto** che avrebbero sul risultato — combinate in un livello di **priorità** che ordina gli interventi di mitigazione.

Le scale adottate sono *qualitative* a tre livelli. È una prassi riconosciuta nella valutazione del rischio quando manchi una base statistica su cui fondare stime di frequenza: la guida del NIST alla conduzione delle valutazioni di rischio prevede scale qualitative per la probabilità e per l'impatto, e la loro combinazione in un livello di rischio complessivo [64]. È il caso di questo lavoro: non esistono serie storiche di progetti comparabili da cui derivare stime di frequenza, e una scala numerica trasmetterebbe una precisione che i dati sottostanti non giustificano. La probabilità è quindi espressa come *bassa*, *media* o *alta*, e l'impatto come *basso*, *moderato* o *critico*, dove per critico si intende un impatto che comprometterebbe la validità del risultato e non soltanto la sua qualità.

La combinazione delle due dimensioni non è simmetrica, ma riflette l'asimmetria già discussa a proposito dei falsi negativi (cfr. 4.2.3): l'impatto pesa più della probabilità. La priorità è pertanto *alta* quando a un impatto critico si accompagna una probabilità almeno media; *bassa* quando una probabilità bassa si accompagna a un impatto non critico; *media* in tutti i casi intermedi — inclusi quelli, come i rischi di sicurezza, in cui un impatto critico è associato a una probabilità contenuta.

Le valutazioni riportate nel paragrafo seguente sono **residue**: esprimono cioè il rischio così come risulta *dopo* l'applicazione delle contromisure descritte, non la sua gravità potenziale in loro assenza.

4.3.2 Matrice probabilità–impatto

La Tabella 4.1 posiziona i quindici rischi identificati secondo i criteri appena definiti.

Tabella 4.1: Valutazione dei rischi identificati: probabilità, impatto e priorità residua.

§	Rischio	Probabilità	Impatto	Priorità
<i>Rischi di progetto</i>				
4.1.1	Dataset di valutazione	Alta	Critico	Alta
4.1.2	Integrazione multi-approccio	Media	Moderato	Media
4.1.3	Costo computazionale	Alta	Critico	Alta
4.1.4	Dipendenze di terze parti	Alta	Moderato	Media
4.1.5	Dilatazione del perimetro	Alta	Moderato	Media
4.1.6	Curva di apprendimento	Media	Moderato	Media
<i>Rischi di prodotto</i>				
4.2.1	Accuratezza del rilevamento	Media	Critico	Alta
4.2.2	Falsi positivi	Alta	Moderato	Media
4.2.3	Falsi negativi	Media	Critico	Alta
4.2.4	Generalizzazione cross-domain	Alta	Critico	Alta
4.2.5	Supporto multilingue	Media	Moderato	Media
4.2.6	Scalabilità	Media	Moderato	Media
4.2.7	Qualità del ROPA in input	Alta	Critico	Alta
4.2.8	Aggiornamento normativo	Bassa	Moderato	Bassa
4.2.9	Sicurezza dei dati in analisi	Bassa	Critico	Media

Il quadro che ne emerge presenta due caratteristiche degne di nota. La prima è che i rischi a priorità alta si concentrano sugli **ingressi** e sui **criteri di misura** del sistema — la disponibilità di un corpus di valutazione, la qualità del registro fornito, la capacità di generalizzare su documenti eterogenei — più che sulla sua realizzazione interna: è un sistema la cui affidabilità dipende in misura sostanziale da ciò che riceve dall'esterno. La seconda è che la priorità alta coincide esattamente con l'impatto critico: tutti e sei i rischi classificati come prioritari sono quelli che, manifestandosi, comprometterebbero la validità del risultato e non soltanto la sua qualità, mentre la probabilità stimata si limita a distinguerli fra loro senza mai promuovere alla fascia alta un rischio di impatto moderato. L'unica eccezione in senso opposto è la sicurezza dei dati durante l'analisi, che pur avendo impatto critico resta a priorità media in ragione della probabilità contenuta: la contromisura ne riduce la frequenza attesa, non la gravità potenziale.

4.4 Strategie di mitigazione

Le contromisure sono descritte insieme a ciascun rischio, poiché è in quella sede che se ne comprende la ragione. Questa sezione le rilegge trasversalmente, per mostrare come un

numero ristretto di scelte di progettazione risponda a più rischi contemporaneamente: è la proprietà che rende la mitigazione sostenibile, perché una contromisura dedicata a ciascun rischio moltiplicherebbe la complessità del sistema fino a generarne di nuovi.

Disaccoppiamento tramite interfacce comuni e configurazione Imporre un'interfaccia unica ai rilevatori e mantenere fuori dal codice il catalogo delle categorie e le regole di riconoscimento risponde a quattro rischi distinti: consente di comporre approcci eterogenei senza incoerenze (cfr. 4.1.2), di sostituire un componente divenuto indisponibile senza propagare la modifica (cfr. 4.1.4), di estendere il sistema a nuove lingue e ordinamenti (cfr. 4.2.5) e di adeguarlo all'evoluzione normativa (cfr. 4.2.8).

Asimmetria dichiarata fra i due tipi di errore L'impostazione *recall-oriented*, unita all'esposizione del grado di conferma di ogni rilevamento, governa congiuntamente i due rischi speculari dei falsi negativi e dei falsi positivi (cfr. 4.2.3, 4.2.2). La scelta non elimina né l'uno né l'altro: stabilisce quale dei due il sistema preferisce commettere e fornisce all'operatore lo strumento per assorbire il costo del secondo.

Minimizzazione come proprietà strutturale Non conservare i valori delle PII in alcuna struttura dati, e non trasmettere nulla all'esterno, riduce l'esposizione del sistema alla violazione che esso stesso intende prevenire (cfr. 4.2.9). Il punto qualificante è che la minimizzazione non va affidata a una cautela procedurale, ma resa impossibile da violare per costruzione: se nel modello dei dati non esiste una sede in cui un valore potrebbe essere scritto, non c'è modo di violare il principio per distrazione.

Misura riproducibile su corpus costruiti a questo scopo La generazione di corpus sintetici a partire da un seme, articolati su due livelli e comprendenti una quota deliberata di documenti privi di dati personali, è la contromisura di quattro rischi: rende possibile la valutazione in assenza di benchmark pubblici (cfr. 4.1.1), consente di misurare l'accuratezza per singola categoria (cfr. 4.2.1), rende osservabili i falsi positivi (cfr. 4.2.2) e mette alla prova la generalizzazione su documenti eterogenei (cfr. 4.2.4).

Contenimento del costo di esecuzione L'impiego di approcci leggeri per l'analisi sistematica, l'isolamento dell'elaborazione di ciascun documento e la rielaborazione dei soli documenti modificati agiscono insieme sui rischi di sostenibilità computazionale e di scalabilità (cfr. 4.1.3, 4.2.6), mentre l'esecuzione in un ambiente isolato e riproducibile neutralizza la dipendenza dalla piattaforma su cui il sistema viene installato (cfr. 4.1.4).

Esposizione delle lacune, non occultamento Dove il sistema non è in grado di pronunciarsi — una categoria dichiarata che non si risolve in alcun tipo rilevabile, una durata di conservazione espressa come criterio — l'esito non è il silenzio ma una segnalazione

dedicata, distinta dall'esito conforme (cfr. 4.2.7). È la contromisura all'errore più insidioso di tutti, quello in cui l'assenza di verifica è indistinguibile dalla verifica superata.

Governo esplicito del perimetro La formulazione dei requisiti, che enuncia le esclusioni con la stessa forza delle inclusioni, e l'obbligo di fondare ogni decisione tecnica su prove anziché su preferenze costituiscono nel loro insieme la contromisura ai rischi di natura organizzativa (cfr. 4.1.5, 4.1.6): tengono traccia di ciò che è deliberatamente fuori ambito e impediscono che le scelte più qualificanti del lavoro siano compiute per inerzia.

Rischi accettati Alcuni rischi residui non sono mitigati e vengono assunti consapevolmente, poiché la loro riduzione richiederebbe uno sforzo sproporzionato rispetto agli obiettivi del lavoro: la rinuncia, in questa fase, alla parallelizzazione dell'analisi, il carattere ottimistico di una valutazione condotta su corpus sintetici, la natura intrinsecamente sensibile del registro delle rilevazioni in quanto mappa della collocazione dei dati personali, e la necessità di una validazione empirica sul corpus specifico dell'organizzazione prima di ogni adozione operativa. Sono limiti dichiarati, non trascurati, e come tali sono ripresi fra gli sviluppi futuri.

Capitolo 5

Studi e valutazioni

Le scelte tecnologiche che reggono il sistema sono state fatte in maniera ponderata: ciascuna è stata preceduta da uno studio, condotto su evidenze di letteratura o su misurazioni dirette, il cui esito è riportato qui insieme al ragionamento che vi conduce. Il capitolo precede la descrizione dell'architettura, in quanto vi sono presenti le decisioni che hanno plasmato la struttura del sistema.

Gli studi sono quattro, e procedono dal generale al particolare. Il primo stabilisce dove l'AI generativa sia efficace in relazione al suo costo e dove no, questione che da sola determina l'impianto del rilevamento; il secondo ne isola un corollario spesso frainteso, la differenza fra costo di addestramento e costo di inferenza. I due successivi scelgono i componenti: se costruire il rilevamento sopra un motore esistente, e quale modello generativo adottare per gli impieghi selettivi che il primo studio lascia aperti. La verifica sperimentale sul sistema realizzato è invece nel Capitolo 8.

5.1 Uso dell'AI in rapporto al suo impatto ambientale ed economico

Nella scelta della metodologia per il rilevamento delle PII all'interno dei documenti si è posto un problema che le sole metriche di accuratezza non permettono di risolvere. I candidati (cfr. 3.4) sono tre — rule-based, NER e AI generativa — e la letteratura è concorde nell'attribuire ai modelli generativi il *recall* più elevato, in particolare sulle PII che dipendono dal contesto, dai riferimenti impliciti e dai collegamenti fra più documenti: proprio le categorie che sfuggono agli altri approcci. Se il criterio fosse la sola capacità di rilevamento, la scelta sarebbe già compiuta.

Tuttavia, il sistema qui progettato non analizza un documento: analizza un insieme di documenti, in maniera ripetuta. È questa condizione d'impiego — inferenza sistematica e **ricorrente** su un intero patrimonio documentale — a rendere il costo per interrogazione un parametro architetturale e non un dettaglio di esercizio. Questa sezione raccoglie le evidenze

quantitative su cui la scelta si fonda; i capitoli che vi rimandano (cfr. 3.4.6, 4.1.3, 2.3.2) ne assumono la conclusione senza ripeterne i dati.

5.1.1 Il divario energetico fra modelli discriminativi e generativi

La misurazione di riferimento è quella di Luccioni, Jernite e Strubell, che confrontano su hardware omogeneo (NVIDIA A100) il consumo di famiglie di modelli diverse a parità di compito [39]. Il risultato rilevante per questo lavoro è la separazione netta fra due regimi: i modelli *discriminativi* specializzati su un compito — la famiglia a cui appartengono gli encoder NER — consumano circa **0.002 kWh ogni 1000 inferenze**, mentre i modelli *generativi* decoder-only richiedono circa **0.047 kWh per 1000 interrogazioni** nella generazione di testo. Il rapporto è di oltre un ordine di grandezza, e non dipende dalla qualità dell'implementazione ma dal modo in cui le due architetture producono il risultato: l'una emette una classificazione in un solo passaggio, l'altra genera token in modo autoregressivo.

Il divario si riproduce sul piano economico. A parità di F1 su compiti di classificazione testuale, l'inferenza di un encoder come RoBERTa costa circa \$0.003 per 1000 esempi contro i \$13.50 di GPT-4: un rapporto di circa **4900:1** [41]. La profilazione sistematica del costo dell'inferenza generativa — in energia, latenza e costo monetario — è del resto divenuta essa stessa oggetto di studio, segno che il problema è riconosciuto come strutturale e non contingente [42].

Un terzo dato completa il quadro e va richiamato per correttezza, perché lavora *contro* l'argomento appena esposto: l'addestramento di un modello generativo consuma ordini di grandezza più energia di una singola inferenza, tanto che il costo cumulato delle inferenze eguaglia quello del pre-addestramento solo dopo un numero di interrogazioni compreso fra 200 e 590 milioni [39]. Su scala globale, dunque, l'inferenza pesa meno dell'addestramento. Ciò non attenua però il problema qui in esame, per la ragione che segue: l'organizzazione che adotta il sistema non sostiene il costo di addestramento — che è già stato sostenuto da altri, una volta sola — ma sostiene per intero e ripetutamente quello di inferenza.

5.1.2 Il quadro comparativo

La Tabella 5.1 raccoglie i dati discussi, in corrispondenza degli approcci confrontati nella Tabella 3.1 sul piano dell'accuratezza.

Tabella 5.1: Costo energetico ed economico degli approcci di PII detection, secondo le evidenze di letteratura.

Approccio	Energia di addestramento	Energia / 1k inferenze	Costo / 1k inferenze
Regex / rule-based	Nessuna	Trascurabile	Trascurabile
NER supervised	Una tantum ^a	~0.002 kWh ^b	~\$0.003 ^c
NER zero-shot (GLiNER)	Nessuna ^d	~0.002 kWh ^b	~\$0.003 ^c
LLM generativo (GPT-4o)	Altissima ^e	~0.047 kWh ^b	~\$13.50 ^c
Pipeline ibrida	Bassa (sola componente NER)	Basso-medio ^f	Basso-medio ^f

^a L'intera campagna di fine-tuning di nove modelli su quattro dimensioni di campione richiede ~44 h GPU complessive su A100; per un singolo encoder si tratta di poche ore: costo sostenuto una volta sola e non ricorrente [46]. ^b Misure su A100: classificazione testuale ~0.002 kWh/1k inferenze, generazione di testo ~0.047 kWh/1k inferenze [39]. ^c Ordine di grandezza per compiti di classificazione a parità di F1: RoBERTa contro GPT-4 [41]. ^d L'approccio zero-shot impiega un encoder pre-addestrato senza fine-tuning specifico per il compito [54]. ^e Il pre-addestramento supera l'inferenza cumulata fino a 200–590 M interrogazioni [39]. ^f Regex e NER su tutto il corpus, modello generativo sui soli casi selezionati: configurazione adottata dai sistemi ibridi recenti [17]. La collocazione in fascia «basso-medio» è una *stima derivata* dalle righe precedenti, non un dato di letteratura: finché la quota di documenti sottoposti al modello generativo resta piccola, il costo complessivo è dominato dalla componente NER.

5.1.3 Conseguenza per l'architettura del sistema

I dati riportati non conducono a un rifiuto dell'AI generativa, ma a una sua collocazione precisa, che si può riassumere in un criterio: **alle condizioni d'impiego assunte in questo lavoro — inferenza locale su hardware ordinario e analisi ricorrente dell'intero corpus — l'AI generativa non risulta sostenibile là dove il costo si moltiplica per il numero di documenti**. Ne discendono due impieghi opposti dello stesso strumento.

Il rilevamento sistematico attraversa l'intero patrimonio documentale a ogni esecuzione: su un corpus di alcune migliaia di documenti, ciascuno dei quali richiede più interrogazioni per essere analizzato per intero, il numero di inferenze per singola passata si colloca nell'ordine delle decine di migliaia, e va moltiplicato per la frequenza di riesecuzione. In questo regime la differenza di un ordine di grandezza nel costo unitario non è un'inefficienza marginale: incide direttamente sulla possibilità di eseguire il sistema periodicamente su hardware aziendale. Il rilevamento sistematico è pertanto affidato ad approcci discriminativi.

Gli impieghi **selettivi** (cfr. 2.1.11) si trovano nella condizione opposta: la normalizzazione delle categorie dichiarate in un registro dei trattamenti si esegue una volta per registro, l'analisi approfondita si applica a un campione dell'ordine dell'uno per cento, l'interpretazione della struttura di cartelle riguarda l'albero e non i documenti. Qui il numero di inferenze non scala con il corpus, il costo complessivo resta di ordini di grandezza inferiore, e la comprensione semantica del modello generativo apporta un valore che gli altri approcci non possono fornire. La conclusione, quindi, non è che l'AI generativa costi troppo: è che *costa troppo per interrogazione*, e va perciò riservata ai compiti in cui le interrogazioni sono poche.

5.2 Costo di addestramento e costo di inferenza

L'argomento dell'impatto ambientale viene talvolta impiegato per escludere in blocco l'uso di modelli di apprendimento automatico. Applicato a questo lavoro, sarebbe un uso improprio: l'argomento della sezione precedente esclude l'**inferenza generativa ricorrente**, non l'impiego di modelli in quanto tale, e in particolare non esclude l'eventuale *fine-tuning* di un encoder locale. La distinzione merita di essere resa esplicita, perché le due voci di costo hanno natura e ordine di grandezza incomparabili.

Il **costo di inferenza** di un modello generativo è *ricorrente* e proporzionale al lavoro svolto: si sostiene per ogni documento, e nuovamente a ogni riesecuzione dell'analisi. Un sistema pensato per la verifica periodica lo paga indefinitamente, e il totale cresce con il tempo di esercizio e con la dimensione del patrimonio documentale. È un costo che l'organizzazione adottante sostiene per intero.

Il **costo di fine-tuning** di un piccolo encoder è invece *una tantum*: nell'ordine di poche ore GPU [46], sostenute una volta sola, dopo le quali il modello risultante ha lo stesso profilo di consumo in inferenza di quello da cui deriva — circa 0.002 kWh ogni 1000 inferenze [39], ossia un ventesimo di un modello generativo. Ammortizzato sull'intera vita operativa del sistema, tale costo diventa marginale.

Ne segue che le due decisioni sono **indipendenti** e vanno motivate separatamente. L'esclusione dell'AI generativa dal rilevamento sistematico si fonda sul costo ricorrente, ed è definitiva. La rinuncia al fine-tuning di un encoder, invece, non si fonda sull'impatto ambientale — che sarebbe un argomento debole — bensì sul requisito di operatività immediata senza addestramento su dati dell'organizzazione (cfr. 2.3.7) e sul costo, questo sì proibitivo, dell'annotazione di un corpus di dati personali reali (cfr. 4.1.1). Sono due vincoli di natura diversa, e confonderli indebolirebbe entrambi.

5.3 Riutilizzo di Presidio

Come emerso dallo stato dell'arte (§3.6), Presidio non risolve il problema di governance affrontato dal progetto; il suo *motore di rilevamento*, però, copre gran parte del livello regex e NER che il sistema deve comunque realizzare. Occorre quindi decidere se realizzare tale livello riutilizzando il detection engine di Presidio oppure implementandolo ex novo. La decisione è affidata a una **valutazione empirica**: questa sezione ne fissa i termini, mentre l'esito — l'adozione di Presidio, con GLiNER per il livello NER — è riportato nel §8.4.

Due modelli di integrazione sono possibili, con implicazioni opposte sull'impianto del sistema:

- usare l'*AnalyzerEngine* di Presidio come **orchestratore** dell'intera detection: massimizza il riutilizzo, ma affiderebbe a un componente esterno la *riconciliazione* fra le sorgenti

di rilevamento — cioè il punto su cui poggia la contromisura al rischio di integrazione (cfr. 4.1.2) e in cui risiede il contributo proprio del lavoro;

- incapsulare Presidio **dietro il contratto unico** imposto a ogni sorgente, come un detector fra gli altri che ne traduce i risultati nel formato interno: la riconciliazione e l'eventuale sorgente generativa restano di competenza del progetto, e Presidio diventa il fornitore di espressioni regolari, checksum e NER.

Il secondo modello è preferito, perché preserva il contributo proprio del lavoro e rende la scelta **reversibile**: un contratto unico consente di sostituire il fornitore senza toccare il resto del sistema. L'architettura che ne discende è descritta nel Capitolo 6.

I criteri di valutazione sono:

- **copertura** delle categorie del catalogo: Presidio fornisce detector pre-confezionati per gran parte di esse (email, IBAN, carta di credito, indirizzo IP, codice fiscale italiano), diversi con **validazione a checksum** già inclusa — proprio ciò che il progetto aveva rimandato;
- **estensibilità** sugli identificatori mancanti: il numero AVS svizzero richiederebbe comunque un detector su misura, come nell'implementazione attuale;
- **integrazione del livello NER**: Presidio supporta ufficialmente modelli esterni, incluso GLiNER, coerente con la scelta zero-shot (§3.1.3);
- **vincoli di progetto**: esecuzione interamente on-premise e configurabilità da file, coerente con l'estensibilità config-driven richiesta.

Profilo hardware Un ulteriore elemento a favore riguarda i requisiti di calcolo. L'engine di Presidio e il livello di rilevamento basato su spaCy possono operare su CPU senza richiedere una GPU dedicata [65],[66]; l'accelerazione GPU è infatti opzionale e viene utilizzata quando disponibile, mentre, in sua assenza, Presidio effettua automaticamente il fallback sulla CPU. Questo profilo è coerente con il vincolo di esecuzione on-premise su infrastruttura di PMI, dove hardware accelerato è raramente disponibile, e costituisce quindi un rafforzamento del posizionamento del progetto più che una limitazione.

Restano in ogni caso a carico del progetto, indipendentemente dall'esito: il riconoscitore per il numero AVS, il motore di riconciliazione delle sorgenti con i relativi livelli di conferma, la sorgente generativa selettiva e l'intero blocco di ingestione ROPA e verifica di conformità.

Il giudizio è affidato a una **misurazione diretta**: Presidio viene incapsulato come detector e valutato sullo stesso corpus annotato e con la stessa procedura di scoring (precision, recall, F1) impiegati per la baseline regex, così da confrontare i due approcci sullo stesso metro; l'esito del confronto è nel §8.4.

5.4 Scelta dello Small Language Model per gli impieghi selettivi dell'AI

Il progetto fa dell'AI generativa un uso **selettivo**: l'uso di LLM è escluso dal rilevamento sistematico e ricorrente per le ragioni di costo esposte in 5.1, e riservato a compiti mirati e non ricorrenti. Sono tre i compiti candidati: (1) la **normalizzazione del ROPA** (§6.2.2) — ricondurre descrizioni inaccurate («dati anagrafici») a un vocabolario chiuso di categorie, cioè estrazione di informazione strutturata a output vincolato; (2) l'**analisi profonda di documenti campione** (tipicamente 1 su 100/1000, dove la latenza non è vincolante); (3) l'**analisi della struttura del file system** per individuare collocazioni anomale (ad esempio un curriculum in una cartella *turni/*). I vincoli non sono scelti qui, ma derivano dai requisiti già fissati (cfr. 2.3.2, 2.3.3): il modello deve essere **piccolo, eseguibile su CPU, containerizzabile e scalabile** su hardware ordinario. Si adotta la definizione operativa di *Small Language Model* (SLM) del survey di riferimento — modelli *decoder-only* da 100M a 5B parametri [12]. Come per Presidio (§5.3), la scelta è **empirica**: seguono i criteri, le misurazioni raccolte in letteratura e la conclusione su quale modello sia il migliore.

5.4.1 Criteri di selezione

I criteri, in ordine di priorità per il caso d'uso, e le fonti metodologiche che li sostengono sono riassunti nella Tabella 5.2.

Tabella 5.2: Criteri di selezione dello SLM e relative fonti.

Criterio	Motivazione nel progetto	Fonte
Eseguibilità su CPU (throughput, memoria)	Ambiente target senza GPU	[67]
Quantizzabilità (GGUF / llama.cpp)	Stare in RAM commodity e in Docker	[13, 68]
<i>Instruction following</i> / output strutturato	Task 1 e 3 (aderenza a schema/istruzioni)	[69, 70]
Licenza permissiva + on-premise	Sovranità del dato (dati sensibili, nLPD)	[71]
Contesto lungo	Task 2 e 3 (documenti/alberi ampi)	[72, 73]

5.4.2 Fattibilità dell'inferenza su CPU

La fattibilità dell'inferenza su sola CPU deriva da considerazioni empiriche. La **quantizzazione a 4 bit** riduce l'occupazione di memoria di circa $4\times$ rispetto a `float16` con perdita di accuratezza mediamente inferiore all'1% [13]; nel formato GGUF (K-quant Q4_K_M), uno studio su un'istanza cloud a 4 vCPU riporta il passaggio da 4.5 a 19.5 token/s (circa $4.3\times$) con la RAM che scende da 3.30 GB a 0.93–1.14 GB (fra il 65 e il 72% in meno) [74], e un secondo studio misura su CPU uno *speedup* di $1.5\text{--}2.5\times$ da F16 a Q4 [75]. Il motore di inferenza di riferimento è `llama.cpp` (grafo compilato in C++, formato GGUF a file singolo), che il progetto serve concretamente tramite **Ollama** — un server che incapsula `llama.cpp`, espone i modelli GGUF su porta ed è configurato da variabile d'ambiente (§7.10.2); le

alternative orientate alla GPU non sono adatte allo scopo: vLLM, il riferimento del settore per il *serving* ad alto *throughput*, è progettato principalmente per acceleratori grafici [13]. Il costo va dichiarato: su Llama-3.2-3B l'MMLU passa da 64.2 a 61.8 con GGUF Q4_K_M [43].

Lo studio più direttamente trasferibile al progetto misura sette modelli *instruction-tuned* quantizzati a 4 bit su tre ambienti CPU-only, in un runtime containerizzato in Docker [13]. I dati di throughput e di memoria sono riportati nelle Tabelle 5.3 e 5.4 (E5-2695 v2 = Xeon 12-core del 2013; 8480+ = Xeon Sapphire Rapids 56-core; i7-13700H = laptop 14-core).

Tabella 5.3: Throughput (token/s), 4-bit GGUF su CPU [13].

Modello	Param.	E5-2695 v2	Xeon 8480+	Core i7-13700H
Gemma-3-1B-it	1B	30	100	70
Llama-3.2-1B-Instruct	1.2B	25	120	80
Phi-4-mini-instruct	3.8B	15	80	40
Qwen2.5-7B-Instruct	7.6B	8	45	15
DeepSeek-R1-Distill-Llama-8B	8B	7	40	10
Gemma-3-12B-it	12B	5	30	9
Mistral-Small-3.1-24B	24B	2	15	4

Tabella 5.4: Occupazione di memoria: FP16 vs 4-bit (Q4) e RAM minima [13].

Modello	Param.	FP16 (GB)	Q4 (GB)	RAM min.
Llama-3.2-1B-Instruct	1.2B	~2.4	0.5	4 GB+
Gemma-3-1B-it	1B	~2.0	0.6	4 GB+
Phi-4-mini-instruct	3.8B	~7.6	1.9	8 GB+
Qwen2.5-7B-Instruct	7.6B	~15	3.7	8 GB+
Gemma-3-12B-it	12B	~24	6.7	12 GB+

I modelli 1–4B offrono interazione quasi in tempo reale su CPU; i 6–8B sono lo *sweet spot* qualità/latenza su hardware datato. Per la scalabilità, oltre 8–16 thread i ritorni sono marginali: conviene eseguire **più istanze in parallelo** su gruppi di core disgiunti (un thread per core fisico, con *pinning* NUMA) piuttosto che una singola istanza su tutti i core [13]. Le versioni *instruction-tuned* sono essenziali. Il caso di studio dell'autore fa co-risiedere due modelli piccoli in Docker su una VM da 16 vCPU / 32 GB con latenza mediana inferiore a 2 s su Xeon moderno — l'analogo diretto del deployment previsto qui [13].

5.4.3 Modelli candidati

Il confronto fra i candidati poggia sui test convenzionali con cui la letteratura misura i modelli linguistici, che conviene richiamare brevemente — anche perché nessuno di essi misura

il compito qui in esame. *MMLU* (*Massive Multitask Language Understanding*) è un test a scelta multipla su cinquantasette materie, dalla matematica elementare al diritto, e misura l'ampiezza della conoscenza acquisita in fase di addestramento [76]; *MMLU-Pro* ne è la variante più difficile, con dieci alternative per domanda anziché quattro e maggior peso sul ragionamento [77]. *GSM8K* raccoglie problemi aritmetici di livello scolastico e misura la capacità di ragionamento a più passaggi [78]. *HumanEval* valuta la generazione di codice a partire dalla descrizione di una funzione [79]. *IFEval* misura l'aderenza a istruzioni verificabili in modo automatico, del tipo «rispondi in non più di cinquanta parole» [80]. *BFCL* (*Berkeley Function Calling Leaderboard*) misura la capacità di invocare una funzione con gli argomenti corretti a partire dalla sua descrizione [81]. *MT-Bench*, infine, valuta la qualità delle risposte in conversazioni a più turni, giudicate da un altro modello [82].

Dei sette, i due pertinenti agli impieghi previsti sono *IFEval* e *BFCL*: l'estrazione a output vincolato richiede anzitutto di rispettare un formato imposto, non di possedere conoscenza enciclopedica. *MMLU* e *GSM8K* vanno letti come indicatori generali di capacità, utili a collocare un modello nella propria fascia di taglia ma non a prevederne la resa sui compiti di questo lavoro.

Qwen2.5 (Alibaba) Taglie 0.5B–72B con versioni quantizzate ufficiali; pre-addestramento su 18T token e supporto multilingue (29+ lingue) [83]. La granularità di taglie (0.5B/1.5B/3B) lo rende tra i più flessibili per CPU.

Llama 3.2 1B/3B (Meta) Costruiti con *pruning* e distillazione per l'*edge*, contesto fino a 128K token; sul 3B, valori auto-riportati: IFEval 77.4, MMLU 63.4, *tool use* BFCL V2 67.0 (contro 25.7 del 1B) [73]. È il più efficiente in energia/latenza della sua fascia (§5.4.6).

Phi-3-mini / Phi-4-mini (Microsoft) Phi-3-mini (3.8B) raggiunge MMLU 69% e MT-bench 8.38 pur girando su telefono [84]; Phi-4-mini (3.8B) aggiunge vocabolario a 200K token, *grouped-query attention* e **instruction following e function calling nettamente migliorati**, con matematica/codice pari a modelli di taglia doppia [85, 84]. È il candidato migliore nella fascia ~4B per output vincolato.

Gemma 3 (Google) Famiglia 1B–27B con distillazione, contesto $\geq 128K$ e KV-cache alleggerita; Gemma3-4B-IT è dichiarato competitivo con Gemma2-27B-IT dopo il nuovo post-training [72].

SmolLM2 (Hugging Face) Famiglia 135M/360M/1.7B completamente aperta; SmolLM2-1.7B supera Qwen2.5-1.5B di ~6 punti su MMLU-Pro [86]. Opzione minimale/riproducibile, meno adatta a ragionamento complesso.

IBM Granite 3.x Famiglia dense 2B/8B e MoE 1B/3B sotto licenza Apache 2.0, progettata per l'impresa e l'on-premise (RAG, classificazione, *entity extraction*, *tool use*); i modelli MoE sono indicati per CPU/edge [71]. Rilevante soprattutto sul piano della sovranità del dato: la licenza e l'orientamento dichiarato all'installazione presso il titolare del trattamento non pongono vincoli all'esecuzione interamente locale.

5.4.4 Analisi per task

Task 1 — ROPA verso categorie È estrazione a output vincolato. Il benchmark *LLMStruct-Bench* valuta 22 modelli aperti (da 0.6B a 70B) su 995 campioni verificati manualmente e conclude che **la strategia di *prompting* conta più della taglia**, e che fornire lo **schema JSON nel prompt** è l'opzione più sicura per output parsabili, specialmente sui modelli piccoli [69]. Ciò richiama la distinzione tra output *corretto* e *utilizzabile*: una risposta che viola lo schema è inutilizzabile quanto una sbagliata [87]. Ne discende la raccomandazione: non il modello più grande, ma *constrained decoding*/schema nel prompt più un modello con forte *instruction following* — nella fascia CPU i migliori sono Phi-4-mini e Llama-3.2-3B; per formati verificabili è pertinente IFEval-FC [70].

Task 2 — Analisi di documenti campione Cadendo il vincolo di throughput, dominano ragionamento e contesto lungo; lavorando sul contenuto fornito (setting *RAG*) la conoscenza parametrica pesa meno e anche modelli piccoli rendono bene [13]. È quindi lecito salire di taglia: un 7B (Qwen2.5-7B) o Phi-4-mini restano eseguibili su CPU (~45 e ~80 token/s su Xeon moderno).

Task 3 — Struttura del file system **Lacuna dichiarata:** non è stato individuato un benchmark che misuri il rilevamento di anomalie su gerarchie di file system. In assenza di metrica diretta, l'ancoraggio è alle capacità più prossime — *instruction following* (Llama-3.2-3B, Phi-4-mini) e contesto lungo (modelli a 128K) — ma la validazione su un dataset costruito ad hoc resta necessaria.

5.4.5 Confronto e conclusione

Nessun singolo modello *domina* su tutti e tre i task; tuttavia, per contenere la complessità del deployment — una sola immagine, un solo peso da distribuire e mantenere, nessuna gestione di co-residenza di modelli diversi — il progetto adotta **un unico modello per l'intera pipeline**. La Tabella 5.5 mostra i candidati migliori per ciascun task; il criterio di scelta è quindi l'**intersezione**: quale modello sia adeguato a tutti e tre insieme.

Tabella 5.5: Raccomandazione per task (fascia CPU).

Task	Requisito dominante	Candidati
1 — ROPA→categorie	Output strutturato + instruction following	Phi-4-mini 3.8B , Llama-3.2-3B, Qwen2.5-3B
2 — Analisi documenti	Ragionamento + contesto lungo (latenza libera)	Qwen2.5-7B , Phi-4-mini 3.8B
3 — Struttura FS	Instruction following + contesto lungo	Llama-3.2-3B , Phi-4-mini, Gemma-3-4B

Il modello scelto è **Phi-4-mini (3.8B)**. È l'**unico candidato presente in tutte e tre le righe** della Tabella 5.5: offre il miglior equilibrio *instruction-following/function-calling* nella fascia ~4B [85], ciò che il Task 1 e il Task 3 richiedono come requisito dominante, e resta al contempo adeguato al Task 2, dove la caduta del vincolo di throughput rende sufficiente un modello da ~4B lavorando in *RAG* sul contenuto fornito [13]. Sul piano operativo è la scelta più leggera compatibile con tutti i compiti: **1.9 GB in Q4** e ~80 token/s su Xeon moderno, un solo peso GGUF servito da un solo `llama.cpp` in Docker [13]. Rinunciare a un secondo modello dedicato (ad esempio `Qwen2.5-7B` per il ragionamento del Task 2) è un compromesso consapevole: si accetta un margine di qualità potenzialmente inferiore sul Task 2 in cambio di un deployment radicalmente più semplice e portabile, coerente con il vincolo di containerizzazione del progetto. Come **alternativa orientata a sovranità e governance**, a parità di modello unico, resta **IBM Granite 3.x** (dense 2B/8B, Apache 2.0), la cui licenza e il cui orientamento all'esecuzione presso il titolare rispondono al medesimo vincolo di località dell'inferenza [71].

La scelta resta soggetta a **validazione empirica** sul dominio, come per Presidio: un *gold set* etichettato (per il Task 1, frasi di «categorie di dati» con l'insieme atteso di `pii_type`) su cui misurare precision/recall/F1 a prompt fissato, riusando l'impianto di scoring del progetto. Il componente è isolato dietro il contratto `CategoryMapper`, per cui cambiare modello — o passare a un dizionario deterministico — non tocca il resto della pipeline.

5.4.6 Energia, efficienza e limiti

Sul versante *edge*, uno studio sull'impronta energetica riporta che su Raspberry Pi 5 Llama 3.2 è il più efficiente in potenza (fino a 2.05 Wh) e a più bassa latenza (4.06 s su MMLU), mentre Phi-3 Mini raggiunge l'accuratezza più alta (MMLU 62.3%) ma con la latenza peggiore (34.36 s) [88]. Esiste dunque un compromesso esplicito accuratezza–energia–latenza: poiché i tre task sono a **bassa frequenza** (non ricorrenti sull'intero corpus), è accettabile un modello più accurato e più lento — l'inverso del ragionamento applicato al rilevamento sistematico, dove è la ricorrenza dell'inferenza a escludere l'AI generativa (cfr. 5.1).

Le evidenze su cui questa scelta si fonda hanno però limiti che è bene dichiarare, perché ne circoscrivono la portata. Il primo riguarda la loro provenienza: gran parte dei valori proviene dai *technical report* dei produttori, dunque da misure auto-risportate, che vanno trattate come dichiarazioni di parte e scontate del rischio di contaminazione del testing [84]. Il secondo riguarda la quantizzazione, che non è gratuita: la riduzione a 4 bit comporta un decadimento misurabile della qualità — su MMLU si passa da 64.2 a 61.8 — per cui sul Task 1, il più

esigente in accuratezza, converrà valutare la quantizzazione a 8 bit come compromesso [43].

Gli altri tre limiti riguardano la trasferibilità delle misure al caso in esame. Per il Task 3 non esiste alcun benchmark diretto, e l'ancoraggio a *instruction following* e contesto lungo resta quindi un indicatore indiretto, non una misura del compito. Sull'output strutturato la variabile dominante non è il modello ma la strategia di interrogazione, il che rende i confronti fra modelli meno decisivi di quanto appaiano [69]. I valori di throughput, infine, dipendono in misura rilevante dal processore su cui vengono ottenuti, e vanno riprodotti sull'hardware di destinazione prima di essere assunti come attendibili [67].

Capitolo 6

Architettura del sistema

Questo capitolo descrive l'architettura del sistema: da quali componenti è costituito, di che cosa risponde ciascuno di essi e in quale ordine si scambiano i dati. Il piano dell'esposizione è quello del *progetto* — la struttura adottata, le decisioni che l'hanno determinata e le alternative scartate con le rispettive ragioni — mentre le tecniche e gli strumenti con cui ogni componente è stato poi realizzato appartengono al Capitolo 7.

La struttura scompone il sistema in otto blocchi funzionali, indicati con le sigle B1–B8, ciascuno con una responsabilità unica e un confine esplicito verso i blocchi adiacenti. Due di essi operano *una tantum* sui dati che descrivono l'organizzazione — il registro dei trattamenti e la struttura dell'archivio documentale — mentre i restanti formano la pipeline ricorrente, rieseguita a ogni scansione: estrazione del testo, rilevamento delle PII, registrazione, associazione al trattamento, verifica di conformità e restituzione al DPO. La Figura 6.1 riassume i blocchi e i flussi che li collegano; la Figura 6.2 espande il blocco di rilevamento, l'unico la cui articolazione interna richieda uno schema dedicato.

L'esposizione segue lo stesso ordine. La prima sezione inquadra i due input del sistema, il registro dei trattamenti e l'insieme dei documenti da analizzare; le sezioni successive trattano un blocco ciascuna, nella sequenza in cui i dati li attraversano. La medesima scomposizione governa anche il capitolo di implementazione, così che a ogni sezione di architettura corrisponda quella omologa sulla realizzazione.

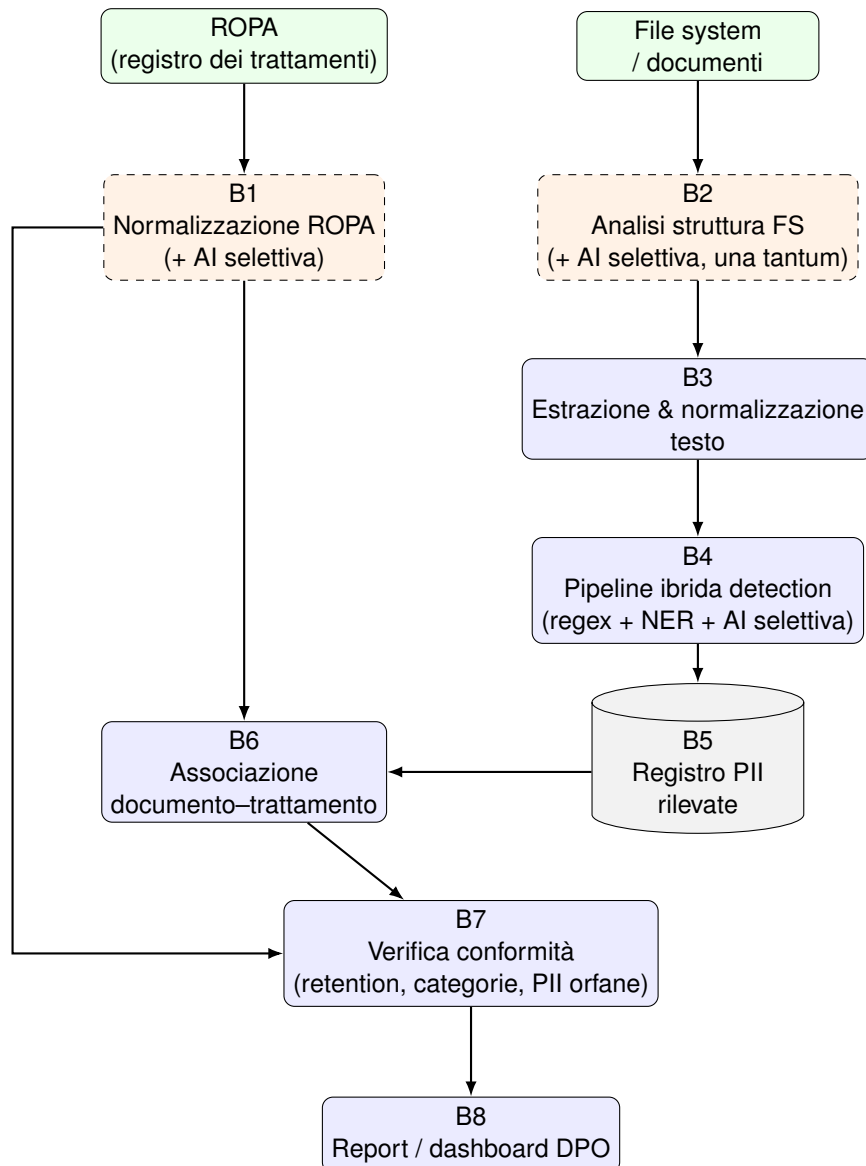


Figura 6.1: Architettura a macro-blocchi del sistema

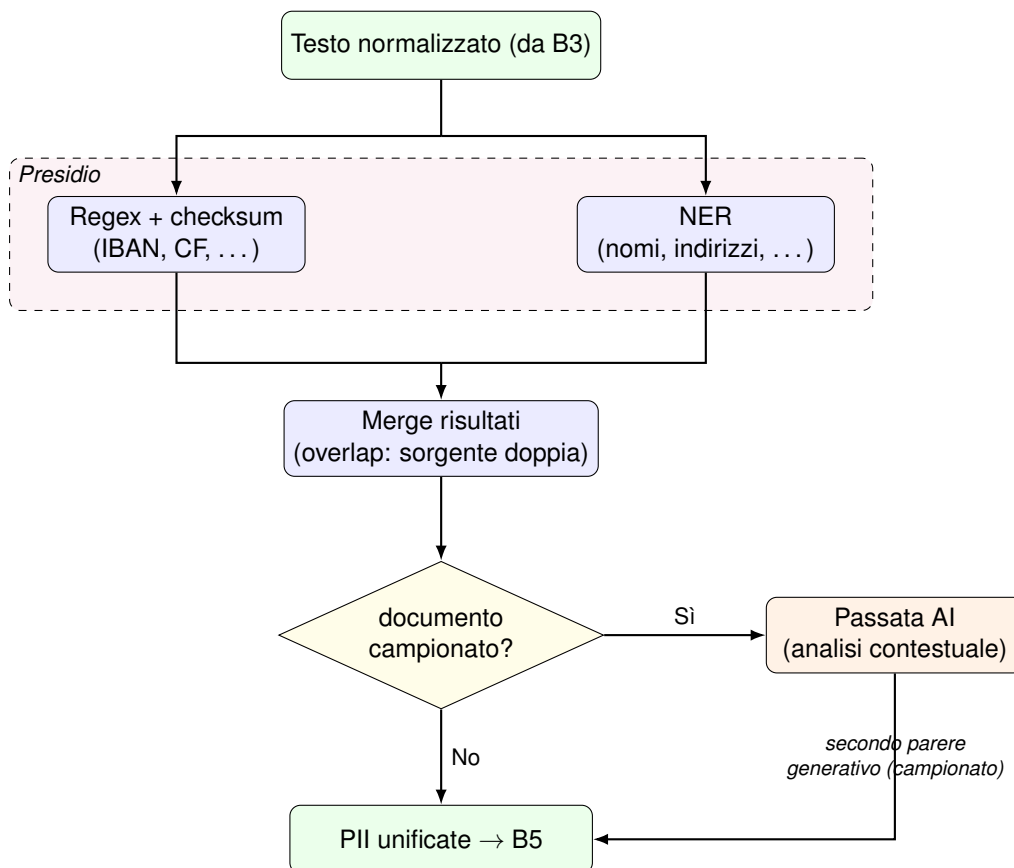


Figura 6.2: Dettaglio del blocco B4: pipeline ibrida di rilevamento PII. I detector regex e NER sono forniti da Presidio (AnalyzerEngine) dietro il contratto PIIDetector; il merge resta proprietario. Il ramo in basso a destra — il campionamento e la passata AI contestuale — è *realizzato*: su un campione di documenti un rilevatore generativo produce candidati che entrano nel merge come terza sorgente, confermando o scoprendo PII (§6.5.6). Lo schema è una semplificazione di flusso: i candidati generativi confluiscono nel merge accanto a quelli di regex e NER, non a valle di esso.

6.1 Input del sistema

Prima di discutere dei componenti attivamente presenti nel sistema, è importante inquadrare quali sono gli input usati dal sistema all'inizio della pipeline di esecuzione.

6.1.1 ROPA - Registro dei Trattamenti

Il primo input è il *Registro dei Trattamenti (ROPA)* di un'azienda. In generale, non esiste un template riconosciuto universalmente. Questa disomogeneità porta la sua complessità, per una serie di motivi:

- Tipo di file: essendo un tipo di dato libero, la sua interpretazione e decisione su come salvarlo ricade sull'azienda in questione. Esistono centinaia di modi diversi per salvare un registro, ognuno dei quali richiede particolare attenzione.
- Sintassi di scrittura: seppur inquadrando un certo tipo di file come più adatto, è comunque difficile predire esattamente dove il dato a noi importante (le categorie salvate, per esempio), si troverà all'interno del file. Questa caratteristica implica l'introduzione di un tipo di complessità basata sul contesto, ambito in cui gli algoritmi tradizionali peccano in efficienza. Una strada è sicuramente l'uso di AI generativa per la comprensione completa del documento in questione e il suo adattamento. Tuttavia, questo porta numerosi problemi, che inoltre, trascendono dallo scope di questo lavoro (AI generativa per parsing dei parametri, giusto per leggere leggermente meglio un documento)

6.1.2 File system

Il secondo input è l'insieme dei documenti da sottoporre ad analisi. L'unità selezionata può essere un singolo file, una cartella, oppure — il caso più aderente all'ambito del progetto — un intero albero di sottocartelle: la cartella indicata è la radice da cui la scansione procede in modo ricorsivo. Dei formati esistenti se ne supporta un sottoinsieme: quelli di uso più diffuso tanto in ambito *consumer* quanto *enterprise*. La ragione è di perimetro — estendere il supporto a un numero arbitrario di formati aggiungerebbe copertura, ma nulla alla domanda a cui questo lavoro risponde: il supporto è garantito perciò per i formati più comuni quando si parla di *file non strutturati*.

6.2 Normalizzazione ROPA - B1

Questo blocco è responsabile per la lettura e trasformazione del registro dei trattamenti in una forma standardizzata del sistema. Questo passaggio è obbligatorio, in quanto il termine di paragone per la verifica di conformità è tra il Registro dei Trattamenti e le PII identificate nei

documenti: senza una struttura fissa e standardizzata, un confronto sarebbe estremamente dispendioso e complesso.

È stata effettuata una scelta ponderata per quanto riguarda lo schema della struttura dati che avrebbe contenuto il ROPA. Si sono presentate due possibili strade:

- Struttura dati definita a monte: viene imposto uno schema noto e il database viene popolato in maniera deterministica partendo dal ROPA in input.
- Struttura scelta da un agente di intelligenza artificiale: il ROPA viene fornito al modello, delegando a lui la creazione della struttura del database, in base al contesto e a ciò che reputa migliore per il salvataggio in maniera efficiente dei dati.

È stata adottata la prima opzione per il seguente motivo: un Registro dei trattamenti è un documento di conformità **legale**. Bisogna prediligere affidabilità e riproducibilità nella sua creazione. Un modello di AI, seppur potenzialmente in grado di fornire informazioni aggiuntive, può essere soggetto ad allucinazioni o a risultati non riproducibili, casistiche non tollerabili per questi contesti.

Avere uno schema esplicito garantisce che il database sia interrogabile in maniera prevedibile dal blocco di confronto B7 (6.8) e mantenibile nel tempo.

L'AI non viene perciò esclusa dal blocco, ma confinata all'unico punto in cui un'interpretazione è indispensabile: la riconduzione delle categorie dichiarate in testo libero al vocabolario condiviso dei tipi di PII, descritta in §6.2.2.

6.2.1 Template registro CNIL

Per fronteggiare al problema dell'inesistenza di un template universalmente condiviso a livello mondiale o europeo per i Registri dei Trattamenti, è stato deciso di adottare come template supportato quello fornito dalla *Commission Nationale de l'Informatique et des Libertés (CNIL)*, autorità di controllo francese che mette a disposizione online una versione basilare ma completa, per aiutare le aziende ad orientarsi nella creazione del proprio registro [89, 19]. Questo modello è organizzato per attività di trattamento: un foglio Excel per ogni trattamento, con i seguenti campi.

- identificazione del trattamento (nome, date di creazione e aggiornamento, base giuridica);
- attori: titolare del trattamento, eventuali co-titolari, responsabile della protezione dei dati (DPO), responsabili del trattamento esterni (*processors*);
- finalità del trattamento;
- categorie di persone interessate;
- categorie di dati personali, con evidenziazione delle categorie particolari (dati sensibili);

- categorie di destinatari (interni, esterni, responsabili esterni);
- trasferimenti verso Paesi terzi (destinazione e garanzie);
- durate di conservazione;
- misure di sicurezza tecniche e organizzative.

Per lo scopo del lavoro, **non verranno salvati tutti i dati** presenti in un trattamento. Questa scelta è stata fatta per ridurre la complessità ed evitare di salvare dati potenzialmente sensibili all'interno del database del sistema, senza un vero e proprio scopo. Di conseguenza, gli unici dati che sistematicamente vengono salvati sono quelli presenti nelle sezioni:

- identità del trattamento;
- categorie di dati personali.

La struttura scelta per rappresentare il registro segue la sua gerarchia naturale su **tre livelli**. Al primo, l'**attività di trattamento**, con la sua identità (un nome e una finalità). Al secondo, le **macro-categorie di dati** che vi appartengono — le voci della sezione «categorie di dati personali» del template —, ciascuna con la propria **durata di conservazione**. Al terzo, le **categorie dichiarate**, ossia le singole voci in cui una macro-categoria si scompone, che ne ereditano la durata e sono ognuna ricondotta al vocabolario condiviso dei **tipi di PII** — lo stesso usato dal rilevamento (B4). È quest'ultimo aggancio a rendere il confronto fra ciò che il registro *dichiara* e ciò che il motore *rileva* (B7) una semplice operazione insiemistica: e una categoria dichiarata che non si risolve in alcun tipo rilevabile resta **esplicita** nel modello — dichiarata ma non verificabile dal motore — e come tale va segnalata al DPO, non scambiata per conforme. La durata di conservazione, infine, è tenuta accanto al testo originale in forma *confrontabile dalla macchina*, così che il superamento del termine (B7) sia *calcolato* al momento del controllo e non registrato come stato. La resa concreta di questo modello — tabelle, campi e tipi — è nella sezione di implementazione (§7.1.4).

6.2.2 Ruolo AI in ROPA

Il popolamento dei primi due livelli del modello — l'attività di trattamento e le macro-categorie di dati — è deterministico: la struttura CNIL è nota e i campi si trasferiscono direttamente. Il terzo livello, la singola categoria dichiarata, è invece spesso testo libero e non sufficientemente dettagliato («dati anagrafici, coordinate bancarie, contatti»): ricondurlo al **vocabolario condiviso dei tipi di PII** — lo stesso che il rilevamento (B4) usa — richiede un'interpretazione contestuale, per la quale un algoritmo puramente sintattico è inadeguato. È l'unico punto dell'ingestione in cui interviene un modello generativo, ed entro due vincoli architetturali stretti. Primo, è un intervento **circoscritto**: una volta, a livello di singolo campo, per scindere ed espandere un testo libero in categorie normalizzate — non una delega della struttura

del documento, coerente con la scelta *non agentica* del blocco. Secondo, opera dentro un **vocabolario chiuso**: ogni categoria proposta è validata contro quel catalogo condiviso, e una proposta che non vi rientra è scartata, sicché il modello non può introdurre categorie che il sistema non sa rilevare.

Soprattutto, l'esito del modello è una *proposta*, non una decisione: la mappatura resta tale finché il DPO non la conferma. È l'istanza, in questo blocco, di un principio che attraversa l'intero sistema: qualunque sia l'origine di un'inferenza — un dizionario deterministico, un modello generativo, una regola che associa una cartella a un trattamento, la segnalazione di una PII orfana — l'atto che la rende parte della *verità dichiarativa* è sempre umano. Il sistema propone, il DPO dispone. È il gemello, sul versante della decisione, dell'impostazione *recall-oriented* e del principio di *minimizzazione*: come quelli evitano rispettivamente di scartare e di trattenere, questo evita di **decidere al posto dell'umano**. La realizzazione — i due mapper intercambiabili, il prompt, la validazione, la conferma — è descritta in §7.1.4.

6.3 Analisi struttura File System - B2

Feature presente nell'architettura originale, ma non implementata per vincoli tempistici e di scarsa sicurezza e utilità del possibile risultato. Se ne discute più diffusamente fra gli sviluppi futuri (§10.1).

6.4 Estrazione e normalizzazione del testo - B3

Il blocco B3 sta a monte del rilevamento: trasforma un file, in uno dei formati supportati (§6.1.2), nel **testo normalizzato** su cui opererà il motore di detection. Il suo prodotto è deliberatamente semplice — un flusso di testo lineare, spogliato della formattazione — ed è *questo*, non il documento originale, ciò che entra nel blocco B4: la medesima forma di testo è consumata allo stesso modo dai rilevatori a regole e dal modello generativo, sicché a valle nessuno deve più sapere da quale formato il documento provenisse. La normalizzazione è leggera (spazi e a-capo) e non tenta di ricostruire il *layout*: il sistema tratta documenti *born-digital*, non scansioni, e l'OCR resta fuori ambito (cfr. §10).

Una scelta di progetto merita di essere resa esplicita, perché ribalta un'assunzione iniziale. L'estrazione era stata concepita attorno a uno strumento dedicato — Docling [90] — pensato per convertire formati complessi in testo pulito, fruibile tanto da un modello generativo quanto dai rilevatori a regole. All'atto dell'implementazione, però, i formati selezionati per il supporto si sono rivelati sufficientemente semplici da non richiedere quell'apparato: bastano conversioni leggere che riducono ogni file a un flusso di testo direttamente analizzabile. Docling resta possibile risorsa qualora l'insieme dei formati si allargasse a documenti realmente complessi, nella forma di un'attivazione selettiva sui soli documenti che la richiedono (§10.3); allo stato

attuale sarebbe complessità non ripagata. Coerentemente con la separazione fra architettura e realizzazione, le scelte implementative appartengono al capitolo di implementazione (§7.3).

6.5 Pipeline ibrida di detection di PII - B4

Il blocco B4 della Figura 6.1 è il motore di rilevamento vero e proprio: dato il testo già estratto e normalizzato dal blocco a monte, individua le PII presenti nel documento e le consegna, unificate, al registro (B5). È un livello *ibrido*, che combina due tecniche indipendenti e complementari — un’impostazione che la letteratura recente sul rilevamento di PII indica come la più robusta rispetto alle singole tecniche prese isolatamente [48, 17]; la sua architettura interna è dettagliata nella Figura 6.2.

6.5.1 Due sorgenti complementari dietro un contratto unico

Le due tecniche coprono classi di PII diverse. Le **espressioni regolari**, affiancate dalla verifica di *checksum* dove il formato la prevede, riconoscono gli *identificatori strutturati* — IBAN, codice fiscale, numero di carta, indirizzo IP — dove la forma è rigida e la precisione può essere massima. Il **riconoscimento di entità nominate** (NER) copre invece i dati meno strutturati e dipendenti dal contesto — nomi di persona, indirizzi — dove nessuna regola fissa è sufficiente. La validazione a *checksum* non è un affinamento previsto ma una proprietà già operante: un IBAN o un codice fiscale formalmente ben scritto ma incoerente nella cifra di controllo viene scartato, ed è la ragione per cui il corpus di valutazione (Cap. 8) deve generare valori *aritmeticamente validi* — altrimenti sarebbe il *gold* stesso a essere respinto, gonfiando artificiosamente i falsi negativi.

Entrambe le tecniche sono esposte al resto del sistema attraverso un unico contratto, `PIIDetector` (nella forma `detect(testo) → lista di candidati`): l’orchestrazione dipende solo da questa forma, non dalla tecnologia sottostante, così una sorgente può essere sostituita senza toccare il resto del livello. Nella realizzazione le due sorgenti poggiano sul motore *open source* Presidio [53], adottato come base per regex e NER a valle di una valutazione sperimentale (§5.3).

6.5.2 Estendere il rilevamento senza scrivere codice

Lo stesso contratto rende possibile *comporre* più rilevatori in un unico posto della pipeline. Su questa possibilità poggia il punto di estensione richiesto dai requisiti (cfr. §2.3.4): accanto ai detector predefiniti opera un rilevatore **guidato da configurazione**, che esegue le regole dichiarate in un file esterno. Aggiungere un identificatore proprio di un settore o di un ordinamento — e la categoria corrispondente nel catalogo — è quindi una modifica *dichiarativa*, non un intervento sul codice sorgente. La coerenza di questa scelta è stata verificata applicandola a sé stessa: l’unico identificatore che era stato cablato nel motore

come caso speciale (il numero AVS svizzero) è stato ricondotto al medesimo meccanismo, così che esista una sola strada per tutti i pattern anziché due.

6.5.3 Il merge: unificazione e grado di certezza

Le due sorgenti producono candidati in modo autonomo; il *merge* li unifica in un unico insieme di PII, decidendone il grado di certezza in base a *quante* e *quali* sorgenti concordano su una stessa porzione di testo. Confrontando gli span per sovrapposizione, ogni PII risultante ricade in uno di quattro casi — un vocabolario chiuso:

- **Fonte singola** — rilevata da una sola tecnica, senza sovrapposizioni. Non viene mai scartata (cfr. l'impostazione *recall-oriented* più sotto), ma etichettata come meno certa.
- **Doppia conferma** — due o più sorgenti concordano sullo stesso tipo di PII su span sovrapposti: pattern e NER, oppure l'una o l'altra insieme alla passata generativa. È il caso più solido: la confidenza viene rinforzata.
- **Conflitto** — gli span si sovrappongono ma le sorgenti assegnano tipi diversi. Il merge **non arbitra**: conserva entrambe le ipotesi e demanda la risoluzione al blocco a valle, che dispone di più contesto.
- **Scoperta dall'AI** — individuata dalla passata generativa selettiva (campionata), assente dalle altre sorgenti. È il caso in cui il modello generativo aggiunge qualcosa che gli algoritmi tradizionali non hanno visto (§6.5.6).

6.5.4 Impostazione *recall-oriented*

Nessuna sorgente scarta di propria iniziativa un candidato debole: al più lo segnala con confidenza bassa. La ragione è la finalità di conformità del sistema: un falso positivo costa al DPO una verifica in più, mentre una PII *mancata* è un rischio che resta invisibile. La filosofia scelta è di prediligere il recall rispetto ad altri possibili parametri. Questo approccio è descritto in letteratura per quanto riguarda l'anonimizzazione testuale, dove il recall misura la **capacità di rilevare tutti i dati** [9]. Il merge eredita questo principio — non elimina, ma unifica e classifica — lasciando ogni eventuale scarto a una decisione umana o a un blocco a valle.

6.5.5 Il ruolo della passata generativa

La terza sorgente, quando presente, entra nel merge per ultima e vi gioca due ruoli. Se ricade su una PII già trovata e ne concorda il tipo è *confermatrice*: la sua provenienza si aggiunge alle altre e la certezza sale (una fonte singola diventa a doppia conferma, una già doppia acquista una terza voce). Se cade dove nessun'altra sorgente ha rilevato nulla è *scopritrice*, e produce il quarto caso. Il disaccordo segue la stessa impostazione *recall-oriented*, ma

pesata: un parere generativo singolo che contraddice un'altra fonte mette le due sullo stesso piano (entrambe in conflitto, arbitraggio a valle), ma **non basta a incrinare** un accordo già raggiunto fra pattern e NER, che resta tale. La certezza, in altre parole, non si perde per una contestazione più debole di ciò che contesta.

6.5.6 AI generativa come secondo parere

Le due sorgenti descritte finora appartengono al paradigma degli algoritmi *tradizionali*: veloci, deterministici, economici, ma ciechi a tutto ciò che non è previsto in una regola o in un'etichetta. Il ramo generativo affianca loro un modello linguistico come *secondo parere*, per rispondere a una domanda che è di merito e non di ingegneria — se e quando un modello che legge il documento come farebbe un lettore umano [32] trovi dato personale che quegli algoritmi non trovano, e se il guadagno giustifichi il suo costo, superiore di ordini di grandezza in tempo ed energia. La scelta del modello e la sua valutazione sono l'oggetto degli studi che precedono la progettazione (§5.4, distinzione Task 1 / Task 2 in §5.4.4); l'esito comparativo è nel Capitolo 9.

Ciò che qui rileva è la forma architeturale, ed è dettata proprio da quella domanda: perché l'AI sia *confrontabile* con le sorgenti tradizionali, va costruita come una sorgente fra le altre, non come un meccanismo a parte. Il detector generativo è perciò incapsulato dietro lo stesso contratto `PIIDetector` di regex e NER, così che il merge lo accolga come terza voce (§6.5.3) e lo stesso apparato di misura che assegna precisione e richiamo a un rilevatore possa assegnarli al modello generativo senza codice speciale.

6.5.7 Quando usare l'AI

Una passata generativa costa da secondi a minuti per documento, quindi non la si esegue su ogni file a ogni scansione (cfr. l'impiego *selettivo* previsto dai requisiti, §2.1.11, e il costo in §5.1). La decisione su *quali* documenti sottoporre all'AI è però tenuta **fuori** dal rilevatore — che si limita ad analizzare il testo che riceve — e affidata a una *decisione* dell'utilizzatore: nessuna analisi o una serie di ricorrenze di utilizzo dell'AI (1 ogni 10, 100, 1000 o scansione). Separare la politica dal rilevatore è ciò che consente di innescare l'AI in modi diversi (a campione durante una scansione, o su richiesta su un singolo documento) senza duplicare la logica di rilevamento. La realizzazione del detector e della politica — prompt a catalogo chiuso, difese anti-allucinazione, campionamento — è descritta in §7.5.

6.6 Registro PII rilevate - B5

Il blocco B5 della Figura 6.1 è il **contenitore dei dati personali rilevati** nei documenti. Raccoglie ciò che il livello di rilevamento (B4) individua e lo tiene a disposizione dei blocchi

sottostanti — l'associazione ai trattamenti (B6) e la verifica di conformità (B7) — come stato su cui interrogarsi.

Nella concezione iniziale il registro doveva anche seguire l'**evoluzione** delle PII nel tempo, riconoscendo fra scansioni ricorrenti che cosa fosse cambiato da una all'altra. Quella capacità è stata ricondotta a una forma più semplice: il registro conserva uno **snapshot** della situazione corrente del file system analizzato, che ogni nuova scansione *sostituisce*. Il disegno del tracciamento temporale — progettato e poi non realizzato in questa iterazione — è documentato fra gli sviluppi futuri (§10.4).

Un vincolo di progetto governa *che cosa* può entrare nel registro: la **minimizzazione del dato** (§2.3.5). Il registro non conserva mai il valore della PII, ma solo **riferimenti** a essa — il tipo, la posizione nel documento, la provenienza del rilevamento. Salvare i valori sarebbe una contraddizione prima ancora che un rischio: uno strumento che esiste *per mappare* dove stanno i dati personali, se li duplicasse, diventerebbe una seconda copia di ciò che deve proteggere — una ridondanza che aggiunge esposizione senza aggiungere informazione. La resa concreta di questo principio, a livello di tabelle e campi — inclusa l'assenza deliberata di una colonna che contenga il valore — appartiene al capitolo di implementazione (§7.6).

6.7 Associazione documento-trattamento - B6

Questo blocco si occupa di associare un certo documento ad uno o più trattamenti del ROPA. Questa associazione è informazione cardine del sistema, in quanto stabilisce **quali categorie di dati è lecito che siano presenti** in un certo documento.

Perché l'associazione esplicita sia **praticabile su cartelle reali** — dove attribuire un documento alla volta è improponibile — il DPO dispone di due modalità, entrambe *a sua discrezione*: l'associazione **diretta** di un singolo documento e l'associazione **di una intera cartella**, con cui mappa una volta il *prefisso del percorso in questione* a uno o più trattamenti; ogni documento che ricade sotto quel prefisso — i file nuovi inclusi, alla scansione successiva — *eredita* l'associazione.

Essendoci due modi per associare i documenti, e che uno non esclude l'altro, deve essere stabilita una gerarchia tra le regole.

- In caso di una sola regola applicata (di cartella o di documento), vale solamente la regola applicata.
- In caso di più regole *omogenee* (più di cartella o più di documento), le regole si fondono.
- In caso di più regole *disomogenee* (più regole di più tipologie), vince l'associazione atomica *per documento*.

La relazione è multi-a-multi: un documento può appartenere a più trattamenti (un modulo usato tanto nella selezione del personale quanto nelle paghe) e un trattamento raccoglie

molte documenti. Ne discende il criterio con cui la verifica (B7) legge il dichiarato: le categorie lecite sono l'unione di quelle di tutti i trattamenti associati.

L'associazione è resa esplicita e responsabilità del DPO. Una modalità di associazione non comporta nessuna funzionalità diversa: porta solo semplicità e rapidità nell'associare numerosi documenti, specialmente se possono essere divisi logicamente in più aree.

Oggi l'attribuzione è sempre del DPO; l'architettura lascia però il posto a un proponente futuro (per contenuto o assistito dall'AI) che suggerisca l'associazione lasciando all'umano la conferma — lo stesso schema human-in-the-loop del mapping del ROPA (§6.2.2).

6.8 Verifica conformità PII presenti - B7

Una volta fissata l'associazione tra documento e trattamento, il lavoro del blocco B7 è semplicemente di confronto tra i due. La verifica avviene usando *tutti* i trattamenti a cui il documento è associato. Quali mappature concorrano a formare quell'insieme è un parametro esplicito del confronto: il verdetto *autorevole* considera le sole mappature confermate dal DPO (§6.2.2), mentre la consultazione interattiva conta anche quelle ancora proposte, così da restare informativa prima che la revisione del registro sia stata completata. La distinzione va tenuta presente nel leggere un verdetto, perché la stessa categoria può risultare orfana nell'una lettura e coperta nell'altra; ciò che invece non è ammissibile è che i due criteri convivano *dentro* un medesimo verdetto, e la regola su quali categorie continuo è perciò definita in un unico punto e usata da tutti i controlli, conservazione inclusa. Dall'altro l'insieme dei *pii_type rilevati*: i tipi delle PII effettivamente presenti nel documento secondo il registro. Confrontando i due insiemi, ogni tipo ricade in una voce di un **vocabolario chiuso**, che è il linguaggio con cui il sistema riporta il verdetto al DPO:

Coperta un tipo *rilevato* che è anche *dichiarato* da almeno uno dei trattamenti associati. È il caso conforme: il dato presente è previsto dal registro.

Orfana un tipo *rilevato* che **nessuno** dei trattamenti associati dichiara. È il segnale di conformità cardine: un dato personale presente nel documento che il ROPA non prevede — un trattamento di fatto non dichiarato (art. 30 GDPR, art. 12 nLPD). Si dice «orfana» perché priva di un trattamento-genitore che la giustifichi.

Mancante un tipo *dichiarato* da un trattamento associato ma **non** riscontrato nel documento. Non è di per sé una violazione: può indicare una sovra-dichiarazione, oppure semplicemente che quel documento non contiene quel dato.

Non risolvibile una categoria *dichiarata* nel ROPA che non si risolve in alcun *pii_type* (insieme vuoto, §6.2): è dichiarata ma non rilevabile dal motore. Resta esplicita e va segnalata al DPO, invece di essere scambiata per conforme.

Sotto la cardinalità molti-a-molti la definizione di orfana è deliberatamente **permissiva**: un tipo è orfano solo se *nessuno* dei trattamenti associati lo dichiara (unione, non intersezione). Un dato coperto anche da un solo trattamento pertinente è giustificato. Specularmente, a ogni singola PII rilevata la verifica assegna il primo trattamento associato che ne dichiara il tipo come «trattamento giustificante» (*processing_activity_id*), oppure nessuno se la PII è orfana: è il momento in cui il riferimento opzionale (0..1) lasciato vuoto dal registro (§7.6) viene finalmente popolato.

Due principi ereditati dal resto del sistema governano la lettura del verdetto. Il primo è l'impostazione **recall-oriented** (§6.5.3): il sistema preferisce segnalare un'orfana di troppo che mancarne una, perché un falso positivo costa al DPO una verifica, mentre una PII non dichiarata e non vista è un rischio invisibile. Il secondo è che il verdetto è un **segnale, non una sentenza**: un'orfana afferma «non risulta dichiarata», non «è certamente una violazione». Può infatti nascere da una violazione reale oppure da un buco di mappatura a monte — una categoria del ROPA non ancora risolta sul tipo corrispondente. È il DPO, *human-in-the-loop*, a chiudere il giudizio; il sistema gli mette davanti l'evidenza strutturata.

6.8.1 Verifica della conservazione

Alla verifica delle categorie si affianca un controllo di **conservazione** (*retention*): un dato può essere legittimamente dichiarato eppure trattenuto *oltre* il periodo consentito. Il registro dei trattamenti fissa, per ciascuna macro-categoria, una durata massima di conservazione; confrontandola con l'**età del documento** si segnala una categoria trattenuta oltre il limite quando un suo dato è ancora presente.

L'età del documento è però una **stima, e come tale dichiarata**: il sistema le associa una **provenienza** e la espone nel verdetto, così che una segnalazione fondata su un indizio forte non venga confusa con una fondata su uno debole. *Come* quella data venga ricavata, e con quali garanzie di plausibilità, è dettaglio realizzativo (§7.8.1).

Il verdetto distingue infine due esiti da non confondere: la **violazione**, quando il documento eccede un termine dichiarato, e la **conservazione non verificabile**, quando il registro esprime un *criterio* anziché una durata — caso in cui il **silenzio non è conformità**, ma un controllo che nessuno ha potuto svolgere. Come il resto della restituzione, questo esito è leggibile sull'intero **corpus**, non soltanto documento per documento (§6.9).

6.9 Dashboard per DPO - B8

Il blocco B8 chiude la catena: rende consultabile ciò che i blocchi precedenti hanno prodotto. La scelta architetturale che lo caratterizza è che il sistema si presenta come una **applicazione web unica**, servita su una sola porta, e non come una raccolta di comandi da terminale. La ragione è di destinatario: il DPO non è necessariamente una figura tecnica, e uno strumento di conformità utilizzabile solo da chi sa invocare un comando resta, di fatto, inutilizzato. Le

interfacce a riga di comando continuano a esistere e sono pienamente funzionali, ma il loro ruolo è quello di strumenti di servizio — automazione, esecuzioni schedate, diagnostica — non di via d'accesso ordinaria.

6.9.1 Una sola applicazione per più scopi

La revisione del registro dei trattamenti (B1) e la dashboard di conformità nascono come applicazioni distinte, con cicli di vita e destinatari in parte diversi; sono però *montate* nella stessa applicazione, che le espone sotto percorsi differenti. Il risultato è un unico servizio da avviare e configurare — coerente con il requisito di semplicità di installazione (§2.3.3) — senza che le due parti debbano fondersi in un solo componente: ciascuna conserva il proprio insieme di pagine e i propri test. La dashboard calcola il verdetto di conformità nel momento in cui una pagina viene aperta, e questo pone un vincolo preciso: la sola consultazione non deve produrre effetti sullo stato del sistema. Il verdetto mostrato in lettura è quindi calcolato in modalità *non persistente*, mentre la scrittura degli esiti per singola PII avviene soltanto in risposta a un'azione esplicita del DPO — tipicamente l'associazione di un documento a un trattamento. È una distinzione che vale la pena rendere esplicita, perché il medesimo calcolo serve due scopi diversi: informare e registrare.

6.9.2 Visualizzazione e minimizzazione

La minimizzazione del dato (§2.3.5) non è un vincolo della sola persistenza ma attraversa anche la restituzione: le PII sono presentate al DPO come *riferimenti* — categoria, posizione nel documento, provenienza del rilevamento, stato di conformità — e in nessuna schermata compare il valore del dato personale rilevato. Una dashboard che mostrasse i valori trasformerebbe lo strumento di verifica in una nuova copia dei dati da proteggere, vanificando la scelta compiuta a monte nel registro.

La restituzione è organizzata su due livelli complementari. La vista *per documento* permette di verificare che il documento in questione sia in regola, riunendo tutte le PII rilevate, l'associazione ai trattamenti e il verdetto in un unico luogo. Le viste *per corpus* rispondono invece alle domande che un responsabile pone su un intero archivio — che cosa è oltre il termine di conservazione, quali documenti nessuna regola ha ancora associato — e sono ciò che rende lo strumento praticabile su volumi reali, dove l'esame documento per documento non è una procedura eseguibile.

Capitolo 7

Implementazione

7.1 Ingestione del ROPA - B1

Il blocco B1 realizza il percorso che porta un registro dei trattamenti, così come lo compila un'organizzazione, alla rappresentazione interna descritta nell'approfondimento sul modello (§7.1.4). La realizzazione segue il principio *sottrattivo* già motivato in architettura — del documento sorgente si conserva soltanto ciò che alimenta la verifica di conformità — e lo traduce in una catena di tre step indipendenti (lettura, interpretazione, persistenza), seguiti da una passata separata di mappatura assistita. Ogni passo è un modulo a sé, così che un cambiamento di formato del file o di strategia di mappatura non tocchi gli altri. La Figura 7.1 riassume la catena e le due interazioni che la circondano: la passata di mappatura e le operazioni del DPO.

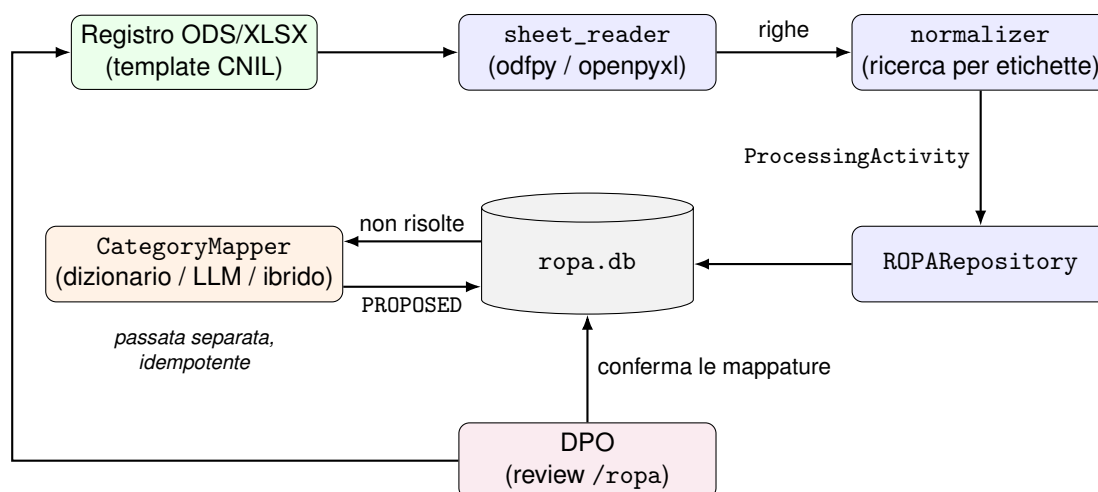


Figura 7.1: Blocco B1: la catena di ingestione (lettura, interpretazione, persistenza) e ciò che la circonda. La mappatura verso i `pii_type` è una passata separata dal parsing deterministico e scrive solo proposte (PROPOSED); il DPO interagisce dalla review web con le due sole operazioni ammesse — importare un registro e confermare le mappature (§2.2.2).

7.1.1 Uso del template CNIL

La fonte è il **modello di registro ufficiale della CNIL**, l'autorità francese per la protezione dei dati [19]: un foglio di calcolo (ODS/XLSX) con una scheda operativa (*fiche*) per ciascun trattamento, oltre a schede di servizio (tutorial, elenco dei trattamenti, un modello vuoto da replicare, liste di supporto). Basarsi su un template condiviso e testato, rende la funzionalità dimostrabile su un input che un'organizzazione userebbe davvero. Nella *fiche*, l'ingestione modella **una sola sezione** — «Categories of personal data», con le sue colonne *categoria*, *descrizione* e *periodo di conservazione* — e scarta deliberatamente tutte le altre (attori, finalità secondarie, categorie di interessati, destinatari, trasferimenti, misure di sicurezza), coerentemente con la scelta sottrattiva. È questa sezione, e solo questa, a fornire il dichiarato che B7 confronterà con il rilevato.

7.1.2 Lettura: un'unica forma per ODS ed Excel

Il primo passo (*sheet_reader*) legge una *fiche* formattata in tabella, dietro un'interfaccia **indipendente dal formato** (*Adapter*): l'estensione del file decide il *back-end* — .ods tramite *odfpy*, .xlsx/.xlsm tramite *openpyxl* — ma entrambi restituiscono la stessa struttura, così che il passo interpretativo in seguito ignori del tutto la provenienza. Un dettaglio pratico del formato ODS ha richiesto attenzione: nel modello CNIL diverse celle portano un **commento** d'aiuto (il «triangolino rosso»), che nel documento risiede *dentro* la cella come annotazione. Letto ingenuamente, quel testo verrebbe concatenato davanti al valore reale (per esempio «Cf. Article 87...Social Security Number»); il lettore esclude perciò i paragrafi contenuti nelle annotazioni, restituendo il solo valore della cella. È una piccola scelta di robustezza, ma tocca proprio la sezione che si vuole conservare, e come tale è coperta da test sul file CNIL originale.

7.1.3 Interpretazione: guidata dalle etichette, non dalle posizioni

Il secondo passo (*normalizer*) trasforma la tabella in un'attività di trattamento. La regola cardine è che le informazioni si cercano per **etichetta**: il nome del trattamento è il valore accanto alla cella «Name of the processing operation», la finalità quello accanto a «Main purpose», e le categorie sono le righe che seguono l'intestazione «Categories of personal data» fino alla prima riga vuota. Così la lettura supporta spostamenti di righe e la presenza delle numerose sezioni ignorate. La stessa regola fa da filtro naturale sulle schede da non processare: una scheda priva del nome di trattamento — come nel template: il tutorial, le liste, il modello *vuoto* — non è un'attività e viene saltata sollevando un errore che il chiamante intercetta, sicché un intero *workbook* CNIL produce esattamente un'attività per ogni *fiche* compilata. Il periodo di conservazione, infine, viene reso **calcolabile**: la dicitura originale («5 years from the payment of the salary») è conservata per audit accanto a una

durata in mesi estratta da essa (60), quando il testo esprime una durata fissa; resta *None* quando esprime un criterio, coerentemente con il campo del modello (§7.1.4).

7.1.4 Persistenza e mappatura assistita

Tre tabelle rispecchiano la gerarchia a tre livelli del registro CNIL.

Livello 1 - *processing_activity*

id : identificatore dell'attività (chiave primaria), assegnato dal chiamante.

name : nome del trattamento, per leggibilità e report al DPO.

purpose : finalità dichiarata del trattamento, elemento cardine dell'art. 30 GDPR.

Livello 2 - *macro_category*

id : chiave surrogata (auto-generata).

activity_id : chiave esterna all'attività: colloca la macro-categoria nella gerarchia.

raw_text : dicitura originale della macro-categoria, conservata per audit.

retention_text : dicitura originale del periodo di conservazione, conservata per audit.

retention_months : la stessa conservazione resa **calcolabile dalla macchina** (durata in mesi), affinché il controllo di retention (B7) sia un confronto e non una lettura di prosa; vale *None* quando il registro esprime un criterio anziché una durata fissa.

Livello 3 - *declared_category*

id : chiave surrogata.

macro_category_id : chiave esterna alla macro-categoria di appartenenza.

raw_text : testo libero originale della categoria (es. «nome»), conservato per audit.

pii_types : l'insieme di *pii_type* del catalogo condiviso a cui la categoria è ricondotta: è l'**aggancio** che rende il confronto fra ciò che il ROPA *dichiara* e ciò che il motore *rileva* (B7) una semplice operazione insiemistica.

mapping_state : stato del mapping (PROPOSED dall'analisi iniziale / CONFIRMED dal DPO)

Il terzo passo affida l'attività così ottenuta al *ROPARepository*, che la scrive nella base dati ROPA riutilizzando lo stesso modello di dominio come schema (*Active Record*, §7.4). La risoluzione delle categorie sui *pii_type* del catalogo condiviso con la detection — l'aggancio verso il rilevamento — è invece una azione separata, non parte del parsing deterministico: per

ogni categoria ancora non risolta, un meccanismo allineato con il contratto `CategoryMapper` propone i `pII_type` corrispondenti al suo testo libero. La strategia è intercambiabile — un **dizionario** deterministico (parole chiave), un **modello generativo** (§5.4) o una combinazione dei due — e la passata è **idempotente**: ciò che il mapper non sa risolvere resta invariato, così una riesecuzione mappa solo ciò che è nuovo. Il risultato è comunque una *proposta*: lo stato del mapping resta `PROPOSED` finché il DPO non lo conferma, a ribadire che l'AI arricchisce ma non decide.

L'interazione del DPO avviene tramite un'interfaccia web dedicata (blocco B8 7.9, la review montata sotto `/ropa`), da cui può confermare le mappature e **importare** un registro caricandone il file dal browser — il file viaggia verso il servizio come *byte* via *multipart*, coerente con il deployment containerizzato in cui l'applicazione non accede al file system del client. L'import riusa senza modifiche la catena di ingestione appena descritta: la stessa lettura, la stessa interpretazione, lo stesso mapping.

Quelle due sono le sole operazioni disponibili, ed è una scelta dettata dal requisito che tratta il registro fornito come riferimento dichiarativo autoritativo (§2.2.2). La copia interna è una proiezione del file dell'organizzazione, non una sua versione modificabile: se dall'interfaccia si potessero aggiungere categorie, correggere durate o cancellare trattamenti, si otterrebbero due registri divergenti — quello che fa fede presso il titolare e quello su cui il sistema calcola — senza che nulla dica quale dei due valga. Ciò che il sistema può fare su un registro carente resta perciò quanto il requisito prescrive: *segnalarne* le lacune nel resoconto finale. L'unica scrittura ammessa riguarda le mappature verso il catalogo dei `pII_type`, che non sono un campo del ROPA ma l'aggancio interno fra dichiarato e rilevato, e che il DPO conferma esplicitamente.

7.2 Analisi struttura file system - B2

Non realizzato, motivazioni nella sezione al capitolo precedente (6.3) e in sviluppi futuri (10).

7.3 Estrazione del testo - B3

L'estrazione è realizzata da `extract_document(path)`, che dall'estensione del file sceglie il lettore appropriato e restituisce un `NormalizedDocument` (il testo più l'identità del documento). I reader coprono i formati più diffusi in ambito *consumer* ed *enterprise*:

- `.pdf` tramite PyMuPDF (testo *born-digital*);
- `.docx` tramite python-docx;
- `.txt` per lettura diretta;
- i fogli `.xlsx/.xlsm/.ods` riusando il `sheet_reader` del blocco B1 (`openpyxl/odfpy`), con le celle rese come testo;

- il Word *legacy* .doc tramite il binario di sistema *antiword*, presente nell'immagine container (assente in locale, il file finisce in errore e il lotto prosegue).

L'elenco dei formati ha un'unica fonte (`supported_suffixes()`), così che antepima, scansione e lettore lo ereditino. OCR, layout multi-colonna e formati come .xls/.rtf restano fuori ambito (§10).

L'estrazione stima la **data di riferimento** del documento (`extraction/dates.py`), che alimenta il controllo di conservazione (B7, §6.8.1). `reference_date(path)` interroga per primi i metadati interni del file — `modDate/creationDate` del PDF, le *core properties* di DOCX e XLSX — e ricade sul *mtime* del file system solo quando non trova nulla di utilizzabile, restituendo un `ReferenceDate` che porta con sé la **provenienza** (`DateSource: content_metadata` oppure `file_mtime`), sicché il verdetto di conformità può dichiarare quanto vale la stima che espone. È stato introdotto un controllo di plausibilità, scartando le date anteriori al 1990 o collocate nel futuro e passa al candidato successivo; metadati illeggibili non sollevano un'eccezione ma ricadono sul *mtime*.

7.4 Tecnologia di persistenza

Il modello concettuale della persistenza — il registro delle PII rilevate (§7.6) e la gerarchia del registro ROPA (§7.1.4) — è *technology-agnostic*: sono state valutate più opzioni come la realizzazione. I candidati erano due:

- una base dati **relazionale** (SQLite), che offre *join* e interrogazioni relazionali «gratuite» ed è coerente col deployment locale in container senza servizi esterni;
- una persistenza **su file semi-strutturati** (JSON/JSONL), più leggera ma priva di garanzie relazionali native.

La scelta è ricaduta su **SQLite**, per ragioni che discendono dalla natura del dato e dai vincoli di progetto:

- **Il dato è intrinsecamente relazionale.** Entrambi i registri sono grafi di entità legate da chiavi esterne (documento—scansione—istanza; attività—macro-categoria—categoria) e le interrogazioni che servono ai blocchi a valle — stato corrente di un documento, PII orfane, confronto dichiarato-vs-rilevato — sono *join* e operazioni insiemistiche che un motore relazionale risolve nativamente. Su file JSON andrebbero reimplementate a mano, riscrivendo ciò che un DBMS offre già.
- **SQLite è *embedded*, senza server.** È un singolo file, nessun processo o servizio esterno da gestire: coerente col deployment containerizzato e, soprattutto, con la **minimizzazione del dato** — i dati non lasciano l'infrastruttura on-premise e nessun servizio di terzi è coinvolto.

- **La porta per accedere è sempre disponibile.** SQLAlchemy poggia su SQLAlchemy: migrare a un RDBMS in rete (ad esempio PostgreSQL), qualora si superi il singolo nodo, cambia l'URL di connessione — letto da variabile d'ambiente (§7.10) — e non il modello dei dati.

La stessa tecnologia serve entrambi i registri, su due basi dati separate (variabili d'ambiente `ROPA_DB_URL` e `PII_DB_URL`); *che cosa* vi si conserva, campo per campo, è spiegato nelle relative sezioni di ROPA 7.1.4 e PII 7.6.

7.5 Livello di rilevamento - B4

Le due scelte tecnologiche che governano questo livello — costruire il rilevamento a pattern e NER sopra Presidio, e quale modello generativo adottare per gli impieghi selettivi — non sono state compiute qui: sono l'esito degli studi condotti prima della progettazione (§5.3, §5.4), la cui verifica sperimentale è nel Capitolo 8. Questa sezione ne descrive la realizzazione, e in particolare la proprietà che ne governa la manutenzione nel tempo. La Figura 7.2 riassume la struttura: tre rilevatori dietro un unico contratto, fusi da un unico componente.

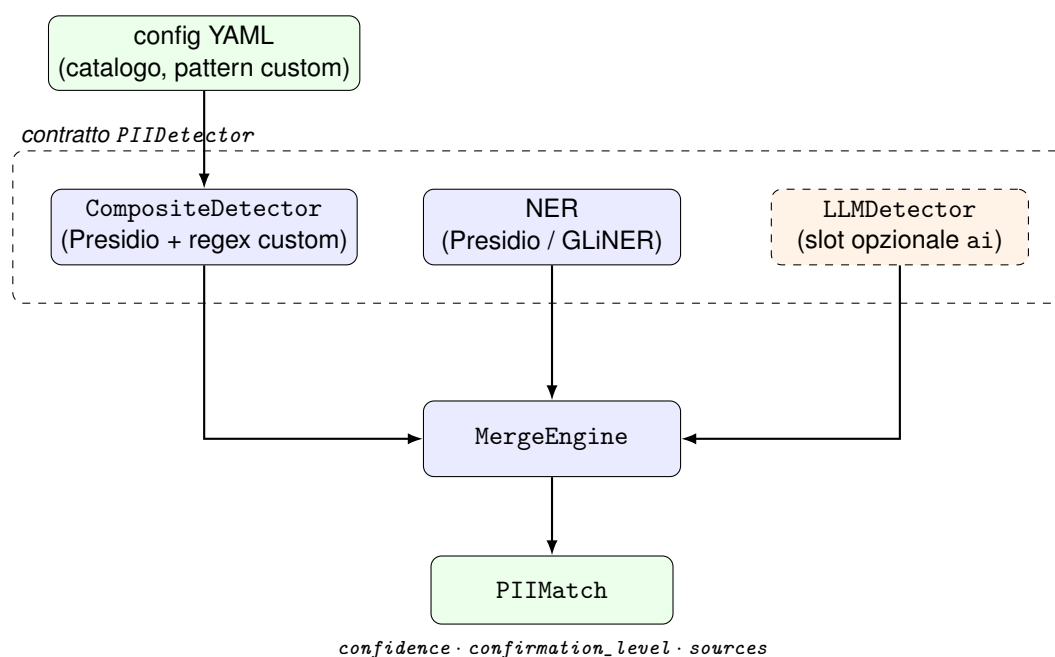


Figura 7.2: La realizzazione del livello di rilevamento: tre rilevatori intercambiabili dietro lo stesso contratto `PIIDetector` — il livello a pattern, esteso dalla configurazione dichiarativa (§7.5.1); il NER; il rilevatore generativo, slot opzionale della scansione — fusi dal `MergeEngine` in `PIIMatch`, dove la certezza del rilevamento è un insieme di campi. Il flusso concettuale della pipeline è in Figura 6.2.

7.5.1 Estensibilità config-driven: aggiungere categorie e pattern

Una proprietà di progetto del livello di rilevamento è che il suo vocabolario non è *hard-coded* nel codice ma **dichiarato in configurazione**. Il catalogo delle categorie di PII (`categories.yaml`) e i pattern a espressioni regolari personalizzati (`custom_patterns.yaml`) sono file validati al caricamento; aggiungere una nuova categoria o un nuovo identificatore strutturato — un codice aziendale, un riferimento interno, un numero di settore — è quindi una **modifica di configurazione, non di codice**. Le regole così dichiarate vengono eseguite a runtime dal detector (`RegexDetector`) e affiancate ai detector interni di Presidio tramite un `CompositeDetector`, così da presentarsi al merge come un'unica sorgente di pattern.

La scelta separa *ciò che* si rileva — competenza del DPO o del manutentore, in configurazione — da *come* lo si rileva, cioè l'engine di detection. Il file dei pattern personalizzati è riservato agli identificatori che Presidio non copre nativamente (il numero AVS svizzero ne è il primo esempio), per non duplicare ciò che l'engine già fornisce. La validazione al caricamento — il pattern deve compilare, ogni `pii_type` deve esistere nel catalogo — fa fallire subito una configurazione errata, invece di peggiorare il rilevamento senza warning o errori.

7.5.2 Presidio come engine

Il rilevamento a pattern e a NER poggia su un unico engine, l'`AnalyzerEngine` di Presidio [53], che è la dipendenza portante del livello: il sistema non riscrive i detector, li gestisce. Presidio è usato come **black box** — riceve il testo e restituisce i riscontri (tipo di entità, intervallo di caratteri, punteggio): non gli vengono fornite *quali* PII cercare; l'engine riporta tutte le entità che i suoi detector conoscono, ed è il sistema a **filtrarle e ricondurle** al proprio catalogo.

L'incrocio fra le entità di Presidio e il vocabolario dei `pii_type` del sistema è **dichiarato in configurazione** (`presidio_entities.yaml`): un'entità mappata diventa un candidato del tipo corrispondente, **un'entità non mappata viene scartata**. Quando la NER è affidata a GLiNER [33] (§5.4) si aggiunge un secondo passaggio dichiarativo — dalle etichette *zero-shot* di GLiNER alle entità di Presidio — così che il risultato ottenuto venga comunque convogliato verso l'insieme di PII supportate.

Una sottigliezza realizzativa merita di essere detta, perché è ciò che tiene *indipendenti* le due sorgenti che il merge poi confronta (§6.5.3). Presidio mescola in un solo engine i detector a pattern e quelli a NER; da quello stesso analyzer il sistema costruisce due detector distinti filtrandone l'output: ogni riscontro che Presidio restituisce porta con sé il nome del detector che l'ha prodotto, e i due detector si spartiscono i risultati in base a quel nome — uno trattiene i pattern, l'altro il NER. Presidio non viene modificato: la separazione avviene sui suoi risultati, non nel suo engine di detection. Senza questa separazione le due tecniche sarebbero un'unica sorgente e non potrebbero né essere distinte né *concordare* sui rilevamenti: non esisterebbe la doppia conferma. Gli identificatori che Presidio non copre nativamente — il

numero AVS svizzero — sono infine aggiunti dal `RegexDetector` guidato da configurazione e composti coi detector di Presidio in un'unica sorgente di pattern (`CompositeDetector`, §7.5.1).

7.5.3 Il detector generativo

Il terzo rilevatore — il ramo generativo (§6.5.6) — è realizzato dietro lo stesso contratto `PIIDetector` degli altri (`LLMDetector`), così da entrare nel merge senza codice speciale. Al modello locale si chiedono i *valori* delle PII, non i loro offset — che un modello linguistico non sa contare in modo affidabile — e ciascun valore è poi **rilocalizzato** cercandone le occorrenze nel testo: un valore inventato che nel documento non compare non viene trovato, ed è la prima difesa contro le allucinazioni; la seconda è il catalogo chiuso nel prompt, contro cui ogni tipo proposto è validato.

Il modello è scelto da una variabile d'ambiente dedicata, `PII_LLM_MODEL`, **distinta** da quella del mapper del ROPA (`ROPA_LLM_MODEL`): le due funzionalità AI possono così adottare pesi diversi, pur puntando per default allo *stesso* modello già presente nell'immagine container — `phi4-mini` [85] — scelto per leggerezza e per non introdurre dipendenze. Di questa scelta va detto anche il limite: un modello piccolo e a inferenza rapida non è l'ideale per *leggere* documenti di molte pagine, dove un modello più capace renderebbe di più. Lo scopo dell'impostazione è che sostituirlo costa una sola variabile d'ambiente — il *runtime* (Ollama) fornisce il nuovo modello senza altri interventi — ed è per questo che la scelta del modello è congrua con il peso del compito da svolgere (§9). *Quali* documenti ricevano il secondo parere è deciso, fuori dal detector, dalla policy di campionamento descritta in architettura (§6.5.6); i riscontri, comunque prodotti, portano con sé la provenienza generativa che il registro conserva (§7.6), ed è quella provenienza — *quale* detector ha trovato *che cosa* — ad abilitare i gradi di certezza del merge.

Tre accorgimenti ne fanno un rilevatore **robusto** in esercizio. Il testo è suddiviso in *chunk* prima di essere sottoposto al modello, perché un documento lungo eccede la finestra di contesto utile. L'output atteso è una breve lista JSON di valori, e un **tetto ai token generati** (`num_predict`, risolto da `resolve_num_predict` con un default e override via `PII_LLM_NUM_PREDICT`) impedisce che un modello divaghi all'infinito — patologia reale dei modelli *reasoning* su questo compito. Soprattutto, il detector non solleva mai esplicitamente errore: se il runtime è irraggiungibile o la risposta non è JSON valido, il *chunk* viene saltato e si restituisce un insieme vuoto, così un'AI indisponibile non fa fallire una scansione ma la priva soltanto del secondo parere.

7.5.4 Il merge: realizzazione

Il merge (§6.5.3) è realizzato da un unico componente (`MergeEngine`) che collassa i candidati delle sorgenti direttamente in `PIIMatch`, il tipo con cui le PII proseguono verso il

registro. Non esistono tipi intermedi: la certezza di una rilevazione è un insieme di campi del `PIIMatch` — `confidence`, `confirmation_level`, `sources`. Il merge incrocia i candidati per **sovrapposizione degli intervalli nel testo**: quelli che si sovrappongono e concordano sul tipo si fondono in una doppia (o tripla) conferma con confidenza maggiorata; quelli che si sovrappongono ma discordano restano **entrambi**, marcati come in conflitto, senza che il merge arbitri. Il candidato generativo entra per ultimo, nel doppio ruolo — conferma o scoperta — descritto in architettura. Il tutto secondo l'impostazione **recall-oriented**: nessun candidato viene scartato, al massimo segnalato come meno certo.

7.5.5 La politica di campionamento

La decisione su *quali* documenti ricevano il secondo parere è tenuta **fuori** dal rilevatore, in una politica a sé e senza stato (`AITriggerPolicy`). Il rilevatore generativo non è fisso nella pipeline ma occupa uno **slot opzionale** (`ai`): assente lo slot, la scansione è puramente tradizionale. La selezione del campione avviene per **hash del percorso** (`blake2b`, non l'hash di processo), così il sottoinsieme è *casuale ma riproducibile* e sparso nell'albero delle cartelle, non concentrato sui primi file di ogni blocco.

7.5.6 Il benchmark multi-modello

Poiché chiedersi se l'AI vale il suo costo è nello scope di questo lavoro, la realizzazione include l'apparato per rispondervi: un benchmark (`run_ai_benchmark`) che misura, sullo stesso corpus e con lo stesso metro delle sorgenti tradizionali, tre configurazioni — la sola baseline tradizionale, l'AI da sola (`ai: modello`) e la loro unione (`union: modello`) — su più modelli raggruppati in tre fasce di taglia. Per ciascuna riporta qualità (precision, recall, F1, anche **per categoria**) e costo (secondi per documento ed energia stimata per documento, quest'ultima come confronto via *codecarbon*). Alcune cautele nascono dall'esecuzione reale su CPU: il tetto `num_predict` che rende ogni modello *bounded*, e il vincolo di un solo modello residente in memoria (`OLLAMA_MAX_LOADED_MODELS`) per non esaurire la RAM caricandone cinque in fila. I numeri che ne risultano sono l'oggetto del Capitolo 8 e del Capitolo 9; qui rileva che la misura è **riproducibile**.

7.6 Registro PII rilevate - B5

Queste tabelle sono lo schema del registro-snapshot descritto in architettura (§6.6): conservano lo stato corrente delle PII per documento — non la loro storia nel tempo, semplificazione motivata fra gli sviluppi futuri (§10.4) — e mai il valore del dato (minimizzazione, §2.3.5), come si vede campo per campo qui sotto.

Tre tabelle realizzano il registro delle PII rilevate: il documento, la scansione e la singola istanza rilevata, che ne è lo **stato corrente**. Una quarta tabella, `registry_folder_rule`,

risiede nella stessa base dati ma appartiene al blocco di associazione ed è perciò descritta con esso (B6, §7.7). È nella tabella delle istanze che la minimizzazione si vede nel dettaglio.

Il documento analizzato - registry_document

`document_id` : identificatore del documento (chiave primaria), coerente con il `document_id` prodotto da B3; nella scansione ricorsiva di una cartella è il **percorso relativo** (POSIX) alla radice, ciò che consente alle regole cartella→trattamento di combaciare per prefisso (§7.7).

`path` : percorso originale del file, come riferimento; opzionale.

`first_seen` : istante di prima registrazione del documento.

`activity_ids` : gli identificatori dei trattamenti dichiarati a cui il documento è associato (B6): è una **lista** (cardinalità N:N, §7.7) e i suoi valori riferiscono `ProcessingActivity` nell'altra base dati, come collegamento *logico* senza chiave esterna fisica.

`association_source` : la **provenienza** dell'associazione — `MANUAL` (impostata dal DPO) oppure `RULE` (derivata da una regola cartella→trattamento) — oppure `None` se il documento non è ancora associato. È il campo che permette di dare priorità alla regole manuale.

`reference_date` : la data cui il documento si assume risalga, letta dai metadati interni del file quando li possiede e dal file system altrimenti, usata come età del contenuto per il controllo *approssimato* di conservazione (B7, §6.8.1); è un *riferimento*, non un valore di PII. Vale `None` se ignota.

`reference_date_source` : la provenienza della data nel campo precedente.

`source_mtime`, `source_size` : l'impronta *tecnica* del file osservata all'ultima analisi: serve unicamente a stabilire se il file sia cambiato e vada quindi rianalizzato.

`last_scanned_at` : il momento dell'ultima analisi effettiva del documento; una scansione in cui esso venga omesso perché invariato non lo aggiorna.

`detector_signature` : l'impronta della configurazione di rilevamento con cui le istanze correnti sono state prodotte: se cambia, il documento va rianalizzato anche a file immutato.

Il vocabolario di `association_source` è un enum (`AssociationSource`: `MANUAL/RULE`), come per gli altri campi a dominio fissato del modello.

Due campi temporali del documento rispondono a domande distinte e non vanno confusi. La `reference_date` è una grandezza **semantica**: dice quanto è vecchio il *contenuto*, può

provenire dall'interno del file ed è ciò su cui lavora il controllo di conservazione. La coppia `source_mtime/source_size` è invece un dato **tecnico**: descrive lo stato del file su disco all'ultima analisi e serve a stabilire se valga la pena rileggerlo. Impiegare la prima anche per la seconda domanda produrrebbe un errore difficilmente rilevabile, perché la data di modifica interna di un PDF resta spesso invariata quando il file viene riscritto: un documento effettivamente modificato apparirebbe intatto e verrebbe saltato indefinitamente. Per questa seconda domanda bastano `mtime` e dimensione: non si calcola un *hash* del contenuto, che richiederebbe di leggere ogni byte di ogni file — proprio il lavoro che la rilettura selettiva serve a evitare. Entrambi i campi, come la `reference_date`, si riscrivono solo quando la scansione ha effettivamente letto il file: la riconciliazione dei documenti spariti dal disco non ne legge alcuno, e non deve perciò sovrascrivere quanto registrato dall'ultima lettura reale.

Singola analisi - `registry_scan`

`id` : chiave surrogata.

`document_id` : chiave esterna al documento analizzato.

`created_at` : istante in cui la scansione è stata eseguita.

Singola PII in un certo momento - `registry_pii_instance` Nessuna colonna di questa tabella contiene il *valore* della PII: è il punto in cui la minimizzazione diventa concreta.

`id` : chiave surrogata.

`document_id` : chiave esterna al documento che la contiene.

`pii_type` : la categoria del catalogo condiviso (es. `iban`) — *non* il valore.

`start, end` : la **posizione** come intervallo di caratteri nel testo normalizzato: un riferimento a *dove* si trova la PII, non al suo contenuto.

`confidence` : confidenza aggregata dal merge, per consentire al DPO ordinamento e soglie.

`confirmation_level` : esito del merge (`single_source` / `double_confirmed` / `conflicting` / `ai_discovered`): traccia *quanto* è certo il rilevamento.

`sources` : gli identificatori dei detector che l'hanno trovata: traccia *chi* l'ha rilevata (tracciabilità, §2.7.3).

`processing_activity_id` : l'attività dichiarata a cui la PII corrisponde, oppure `None` se la PII è **orfana** (non riconducibile ad alcun trattamento); popolata dal blocco di associazione (B6), senza chiave esterna fisica finché i due database restano distinti.

`last_scan_id` : chiave esterna alla scansione che ha registrato l'istanza.

Ogni scansione **sostituisce** le istanze del documento con ciò che vi trova ora: la tabella descrive lo **stato corrente** delle PII del documento, non la loro storia nel tempo. L'**assenza** di un campo `value/text` è una scelta di progetto, non una dimenticanza: garantisce che un registro di conformità non si trasformi a sua volta in un archivio di dati personali.

In sintesi: il registro ROPA conserva il dichiarato (con il testo originale accanto alla forma strutturata, per audit), mentre il registro delle PII conserva soltanto *riferimenti* a ciò che è stato rilevato — mai i valori. I due Database restano separati; i campi `Document.activity_ids` (associazione N:N, B6) e `PIIInstance.processing_activity_id` (trattamento giustificante per singola PII, popolato da B7) sono i punti in cui le due macro categorie di dati del sistema si collegano *logicamente*, sull'identificatore-stringa del trattamento.

7.6.1 Scansione incrementale

Rileggere ogni documento a ogni esecuzione è il costo dominante di una scansione periodica, e su un albero aziendale è quasi tutto lavoro inutile: fra una passata e l'altra la maggior parte dei file non cambia (requisito §2.3.6). Prima di estrarre il testo, la scansione confronta perciò `mtime` e dimensione correnti con quelli registrati all'ultima analisi (`FileStamp` e `needs_rescan` in `registry/freshness.py`); se coincidono e la `detector_signature` non è cambiata, il documento non viene riletto.

Il confronto è per **differenza**. Un file ripristinato da un backup a una versione precedente ha un `mtime` più vecchio di quello registrato, ma il suo contenuto è cambiato quanto quello di un file modificato: un criterio del tipo «più recente di» lo escluderebbe da ogni analisi successiva. Una tolleranza di un secondo (`tolerance_seconds`) assorbe le diverse granularità dei timestamp fra filesystem, che altrimenti farebbero risultare modificato ogni documento.

Saltare un file ha però un effetto collaterale da evitare. A fine scansione il registro viene allineato al disco: i documenti che la scansione non ha trovato sono considerati spariti e le loro PII vengono cancellate. Un file saltato perché invariato è comunque presente sul disco, e deve quindi essere contato fra quelli trovati (`seen`); in caso contrario la seconda scansione cancellerebbe le PII di tutti i file che non sono cambiati. Lo stesso valeva per un file che non si riesce a leggere: veniva escluso da quel conteggio, e le sue PII sparivano dal registro benché il file fosse al suo posto. Un documento mai analizzato con successo, infine, non viene mai considerato invariato: si ritenta a ogni esecuzione.

Due casi restano fuori dall'ottimizzazione. Il primo è l'anteprima della scansione, che dichiara in anticipo quanti file verranno saltati: per non dire il falso applica lo stesso criterio della scansione vera (`count_unchanged`). Il secondo sono i file caricati dal browser, sempre riletti per intero: vengono scritti sul server al momento dell'upload, quindi il loro `mtime` è quello del caricamento e non dice nulla sul contenuto. In ogni caso l'opzione `--full` forza la rilettura completa.

7.7 Associazione documento-trattamento - B6

Il concetto dell'associazione documento-trattamento — e la gerarchia fra le sue modalità — è in architettura (§6.7); qui se ne descrive la realizzazione. L'associazione vive **nel registro delle PII**, come attributo del documento: il campo `Document.activity_ids` (§7.6) è la lista dei trattamenti a cui il documento appartiene (cardinalità N:N). Il blocco offre due strade per popolarlo, entrambe a discrezione del DPO, e un meccanismo che le tiene coerenti nel tempo (Figura 7.3).

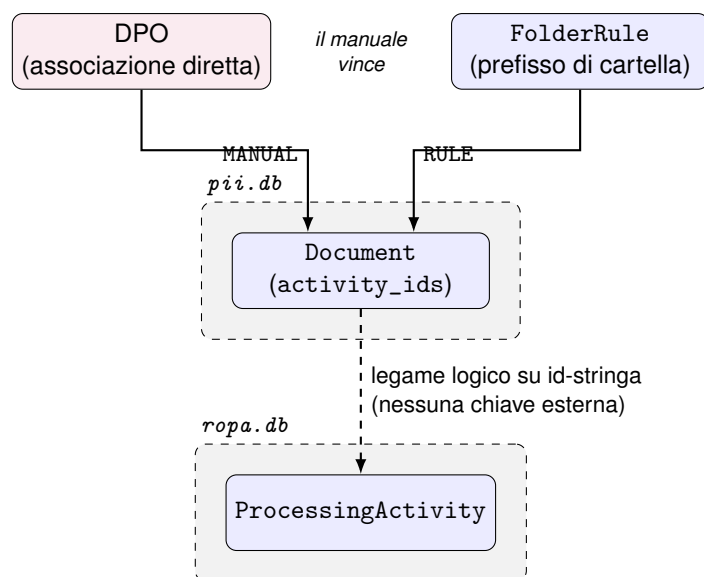


Figura 7.3: L'associazione documento-trattamento: le due strade che scrivono `Document.activity_ids` — diretta (MANUAL) e per regola di cartella (RULE, riconciliata a fine scansione) — con la precedenza del manuale, e il legame fra i due registri: gli identificatori riferiscono `ProcessingActivity` nell'altra base dati, senza chiave esterna fisica (§7.7.4).

7.7.1 Associazione diretta e regole cartella

L'associazione **diretta** scrive gli identificatori dei trattamenti sul singolo documento. L'associazione **per cartella** li deriva invece da una regola: una `FolderRule` mappa un *prefisso di percorso* a uno o più trattamenti, e ogni documento il cui `document_id` — il percorso relativo POSIX prodotto dalla scansione ricorsiva (§7.6) — cade sotto quel prefisso eredita l'associazione, file nuovi inclusi. La logica di match è una funzione singola (`match_activities` in `registry/folder_rules.py`): dato un percorso, restituisce l'**unione ordinata** dei trattamenti di tutte le regole che lo coprono — lo stesso criterio insiemistico con cui la verifica (B7) legge il dichiarato. Il prefisso è normalizzato (POSIX, senza slash finale; la stringa vuota è la radice e combacia con ogni documento), sicché il match resta un confronto testuale, senza glob né espressioni regolari.

7.7.2 Riconciliazione e precedenza del manuale

Ogni documento salva la **provenienza** della propria associazione (`association_source`: `MANUAL` o `RULE`, §7.6), ed è questo campo a fissare la gerarchia fra le due strade: **il manuale vince**. Sono previsti tre casi possibili di riconciliazione delle regole:

- i documenti che una regola copre vengono associati con provenienza `RULE`;
- i documenti a provenienza `MANUAL` sono **saltati** e restano intatti;
- quelli mai associati non si toccano.

Essendo una riconciliazione completa, l'operazione è **idempotente**. Ne discende il comportamento della cancellazione: `delete_rule` rimuove la riga-regola e *poi riconcilia* (restituendo un `ApplyRulesResult` con il conteggio delle associazioni azzerate), così che l'effetto della regola sparisca subito e non alla scansione successiva.

7.7.3 Applicazione automatica a fine scansione

Le regole si applicano come **passo separato dopo** la scansione, non durante: la scansione resta pura (rileva e registra), poi un auto-apply riconcilia il registro. Questo vale sia per il job della dashboard sia per la CLI `scan_folder` (con `--no-apply-rules` per disattivarlo). È ciò che rende l'associazione **autonoma**: una scansione associa da sola i file nuovi che ricadono sotto una regola già definita, senza intervento del DPO. L'utilizzo di `ActivityAssigner` — con l'unica implementazione odierna, `ExplicitAssigner` — lascia lo spazio a un proponente futuro (per contenuto o assistito dall'AI) che suggerisca l'associazione lasciando al DPO la conferma, con lo stesso schema *human-in-the-loop* del mapping del ROPA (§7.1.4).

7.7.4 Il collegamento fra i due database

L'associazione è il punto in cui i due registri — ROPA e PII — si confrontano, pur restando **basi dati distinte** (§7.4). Sia `Document.activity_ids` sia `FolderRule.activity_ids` contengono identificatori di `ProcessingActivity`, che vivono però *nell'altra* base dati: il legame è perciò **logico**, sull'identificatore-stringa del trattamento, e non una chiave esterna fisica. La separazione è ponderata e ha un costo accettabile: il join fra dichiarato e rilevato è risolto **a livello applicativo** (B7), non dal motore relazionale.

`registry_folder_rule` — **la regola cartella** → **trattamento** Fisicamente nella stessa base dati del registro delle PII, ma logicamente parte di questo blocco: mappa un prefisso di percorso ai trattamenti che i documenti sotto di esso ereditano.

`prefix` : il prefisso di percorso (POSIX, normalizzato senza slash finale; "" è la radice e combacia con ogni documento). Chiave primaria.

`activity_ids` : i trattamenti con cui associare i documenti che cadono sotto `prefix`; riferiscono `ProcessingActivity` nell'altra base dati (nessuna chiave esterna fisica), come `Document.activity_ids`.

`created_at` : istante di creazione della regola.

7.8 Verifica di conformità - B7

Il cuore della verifica è un confronto insiemistico, realizzato come funzione (`build_report`) affiancata da un sottile strato di I/O (`check_document`) che le procura gli insiemi e ne persiste l'esito. Da un lato l'insieme **dichiarato**: l'unione dei `pii_type` di tutte le `DeclaredCategory` dei trattamenti associati al documento (§7.7). Dall'altro l'insieme **rilevato**: i `pii_type` che il registro riporta per quel documento. Il confronto assegna a ogni tipo una delle quattro voci del vocabolario chiuso descritto in architettura — coperta, orfana, mancante, non risolvibile (§6.8); la Figura 7.4 ne riassume ingressi ed esito.

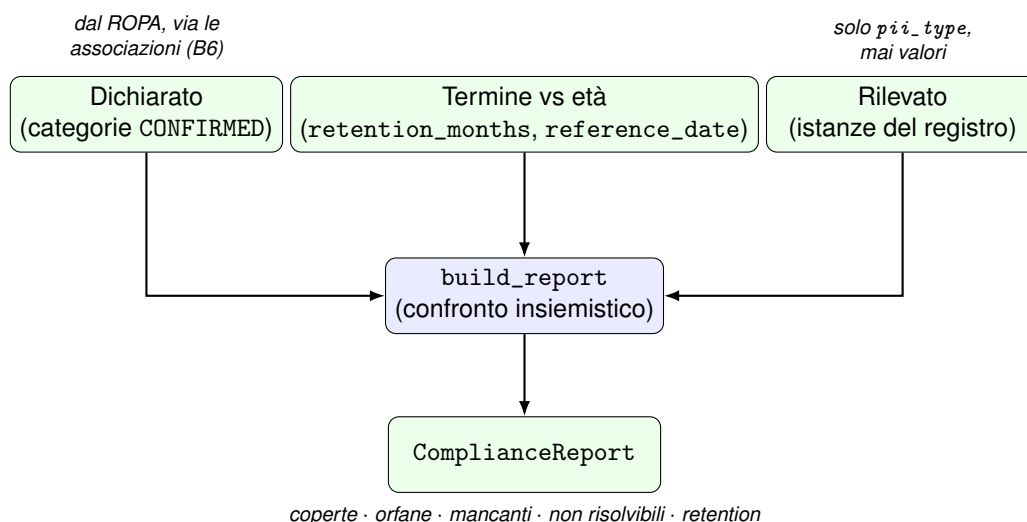


Figura 7.4: La verifica di conformità: `build_report` confronta l'insieme dichiarato dal ROPA con quello rilevato nel registro — sullo stesso terreno entra il controllo di conservazione (§7.8.1) — e produce un `ComplianceReport` di sola lettura, che trasporta tipi e conteggi, mai i valori delle PII. Lo strato di I/O (`check_document`) procura gli insiemi e, solo su azione esplicita, ne persiste la copertura.

Quali mappature entrino nell'insieme dichiarato è un **parametro esplicito** del confronto: si considerano le sole `DeclaredCategory` in stato CONFIRMED, mentre la consultazione interattiva conta anche quelle PROPOSED. Perché uno stesso documento non risulti orfano per una metà del verdetto e coperto per l'altra, la regola su «quali categorie contano» è definita in un **unico punto** (`_macro_types`) e riusata identica dal confronto delle categorie e dal controllo di conservazione (§7.8.1).

Il confronto non produce solo il verdetto aggregato: a ogni singola PII rilevata assegna il **primo trattamento associato che ne dichiara il tipo** come trattamento giustificante, scrivendolo in `PIIInstance.processing_activity_id` (§7.6), oppure `None` se la PII è orfana. È il momento in cui il riferimento cross-database lasciato vuoto dal registro viene popolato, sull'identificatore-stringa del trattamento (§7.7.4). Coerentemente con la minimizzazione, il report trasporta soltanto **tipi e conteggi**, mai i valori delle PII: è un valore di sola lettura (`ComplianceReport`) consegnato al blocco di restituzione (B8), che decide se limitarsi a mostrarlo oppure persistere la copertura (§7.9).

7.8.1 Il controllo di conservazione

Il concetto del controllo di conservazione è in architettura (§6.8.1); qui se ne descrive la realizzazione. Il verdetto confronta la durata dichiarata dal ROPA — `retention_months` della macro-categoria (§7.1.4) — con l'**età del documento**, cioè la sua `reference_date` con la relativa provenienza (`DateSource`), stimata dall'estrazione (§7.3): una categoria è in *violazione* quando un suo dato è ancora presente e la data del documento supera il termine dichiarato.

Il caso più insidioso non è la violazione ma la **mancata verifica**. Quando la macro-categoria non esprime una durata fissa ma un criterio (`retention_months` vale `None`, per esempio «per la durata del rapporto»), il primo prototipo saltava il controllo e il documento risultava *conforme* pur non essendo mai stato controllato. Ora quel caso produce l'esito esplicito `retention_unresolved`, così da distinguerlo dalla conformità effettiva. Il controllo applica inoltre la stessa regola su quali mappature continuo già descritta per il confronto delle categorie.

Il parsing del periodo di conservazione riconosce anche le **unità francesi** (`an/ans/mois`): il template è quello CNIL e una dicitura come «5 ans» non riconosciuta disattiverebbe di fatto il controllo. Il resoconto per singolo documento permette di sapere se un determinato file sia in regola, ma non è sufficiente per il DPO, che sarebbe costretto a scorrere centinaia di file singolarmente. La stessa lettura è perciò disponibile anche come **vista di corpus** (`compliance retention` da CLI, pagina `/retention` nella dashboard): riusa lo stesso calcolo su tutto il registro — così le due schermate non possono divergere — legge il ROPA **una sola volta**, non scrive nulla e ordina i documenti per mesi di sfioramento, mettendo per primi i casi più gravi. I documenti non associati ad alcun trattamento sono esclusi, perché senza un termine dichiarato non c'è nulla da confrontare; l'esclusione è dichiarata nella pagina.

7.9 Dashboard DPO - B8

Il blocco B8 chiude la catena e ne è l'unica **superficie d'accesso ordinaria**: rende consultabile ciò che i blocchi precedenti hanno prodotto (§6.9). È realizzato come un'unica

applicazione web (`pii_detection/web/`, FastAPI e template Jinja2), servita su una sola porta; le interfacce a riga di comando restano strumenti di servizio, non la via d'accesso del DPO.

7.9.1 Un'unica applicazione su una porta

La revisione del ROPA (B1) e la dashboard di conformità nascono come applicazioni distinte, ma la prima è **montata** nella seconda sotto il prefisso `/ropa`: non è stata trasformata in un semplice *router* — così i suoi test restano validi senza modifiche — e per funzionare sotto prefisso i suoi collegamenti interni sono generati con `url_for`, che incorpora il `root_path` del punto di montaggio. Il risultato è un unico servizio da avviare e configurare (comando `container python -m pii_detection.web`), coerente con il requisito di semplicità d'installazione (§2.3.3).

Un vincolo governa la sola consultazione: aprire una pagina **non deve** produrre effetti sullo stato. Il verdetto mostrato in lettura è perciò calcolato in modalità **non persistente** (`check_document` con `persist_coverage` disattivato), mentre la scrittura degli esiti per singola PII avviene solo in risposta a un'azione esplicita — tipicamente l'associazione di un documento a un trattamento — gestita con il pattern **Post/Redirect/Get**, che evita re-inviî accidentali e separa nettamente ciò che informa da ciò che registra.

7.9.2 Dati mostrati sempre per riferimento

La minimizzazione (§2.3.5) non governa solo la persistenza ma anche la restituzione: in nessuna schermata compare il *valore* di una PII: sono mostrati soltanto **riferimenti** — categoria, posizione, provenienza del rilevamento, stato di conformità. La restituzione è su due livelli: la vista **per documento** (la scheda, che riunisce PII rilevate, associazione e verdetto in un luogo) e le viste **per corpus** — la dashboard dei documenti, la pagina `/retention` (conservazione oltre il termine, per gravità, §7.8.1), la pagina `/rules` (regole cartella, §7.7.3) — che rispondono alle domande poste su un intero archivio e rendono lo strumento praticabile a scala.

7.9.3 La scansione asincrona e l'analisi AI

L'avvio della scansione dalla UI pone un problema di durata: analizzare una cartella su CPU dura da minuti a molto più, e non deve bloccare la pagina. Le scansioni sono perciò **job in background** (`ScanJob`: un registro in memoria protetto da *lock*, eseguito su un `Thread`), con la pagina che ne segue l'avanzamento per *polling*; l'assunzione è un **singolo worker** applicativo, adeguata a un uso occasionale e interno.

L'intera esperienza è costruita attorno alla lentezza dell'AI. La scansione procede in **due fasi**: prima i soli rilevatori tradizionali su tutti i documenti (`ingest_folder` senza AI) — veloci, così la dashboard si popola subito — poi una **fase 2** in background (`_run_ai_phase`) che ripassa

con l'AI il solo sottoinsieme campionato, *arricchendo* il registro invece di riscriverlo (grazie al *replace-on-scan* del blocco B5). La quota di AI è scelta dall'operatore da un menu (`ai_rate`: nessuna analisi, tutti i documenti, oppure uno ogni N), e la stessa analisi è innescabile **su un singolo documento** con un trigger asincrono che riusa lo stesso `ScanJob`. Perché l'operatore non resti mai al buio davanti a un job lungo, un **indicatore di scansione attiva** (funzione `active_jobs` esposta come globale ai template in `base.html`) è presente in ogni pagina.

Come la review del ROPA, la web app resta **fuori** dalla reference tecnica generata (Sphinx documenta la libreria, non l'interfaccia); la sua correttezza è coperta dai test funzionali (`TestClient`).

7.10 Deployment containerizzato e portabilità

La containerizzazione non è un **requisito di progetto**: i componenti sono concepiti come **servizi raggiungibili su porta e configurati da variabili d'ambiente**, con l'applicazione nel ruolo di interfaccia leggera. È inoltre funzionale alla **minimizzazione del dato**: tutti i servizi risiedono nella rete privata dell'orchestrazione e i dati trattati non lasciano l'infrastruttura on-premise. Questa sezione fissa l'approccio di deployment adottato e ne descrive la realizzazione.

7.10.1 Portabilità e isolamento delle dipendenze

L'obiettivo operativo è una build **riproducibile su qualsiasi piattaforma**: la stessa immagine deve girare identica in sviluppo e in produzione, indipendentemente dall'architettura dell'host. L'argomento più forte a favore emerge da un problema concreto incontrato nello sviluppo. Il livello di rilevamento NER poggia su uno *stack* di apprendimento automatico pesante e fragile (PyTorch e il modello NER GLiNER); sulla macchina di sviluppo — un interprete Python `x86_64` in emulazione su hardware ARM — la versione di PyTorch disponibile risultava troppo datata per la restante toolchain (NumPy, transformers), rendendo il livello NER **non costruibile in locale**. Un container Linux, con dipendenze moderne, risolve il problema alla radice e — punto generale — **disaccoppia la build dall'architettura dell'host**: ciò che in locale è un vicolo cieco diventa, nel container, l'ambiente di riferimento in cui la stessa immagine è riproducibile ovunque.

7.10.2 Architettura di deployment

Il sistema è orchestrato come un insieme di servizi. Il **servizio applicativo** racchiude il nucleo — ingestione ROPA, motore di detection con i livelli regex/checksum e NER, verifica di conformità, interfaccia per il DPO — insieme alle relative dipendenze, comprese quelle di apprendimento automatico più onerose (Presidio, spaCy, GLiNER). Il **runtime del**

modello generativo — un servizio **Ollama** (basato su `llama.cpp`) che serve lo SLM scelto (§5.4) su porta, con host e nome del modello letti da variabili d'ambiente (`OLLAMA_HOST`, `ROPA_LLM_MODEL`) — è un container separato, coerente con l'uso selettivo e non ricorrente dell'AI. La **base dati** è, allo stato attuale, un file SQLite su volume persistente (§7.4). Tutta la configurazione — URL della base dati, host e nome del modello — è letta da **variabili d'ambiente**, così la stessa immagine gira in locale e in container cambiando solo la configurazione; i pesi dei modelli e lo stato della base dati vivono su **volumi** dedicati, per non essere riscaricati o persi tra un'esecuzione e l'altra. Coerentemente col profilo hardware del progetto (§5.3), tutti i servizi operano su **CPU**. La Figura 7.5 riassume la suddivisione.

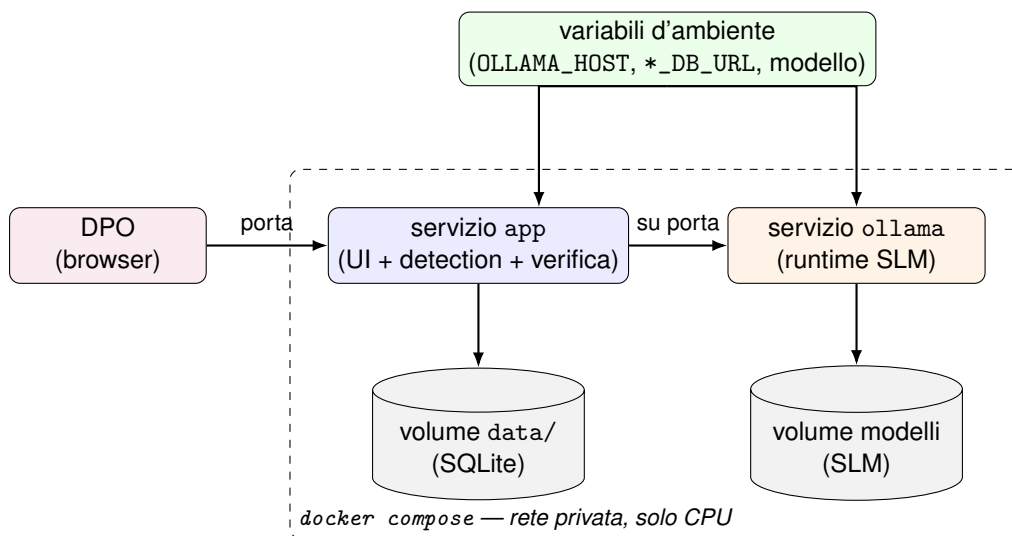


Figura 7.5: L'architettura di deployment: due servizi su porta nella rete privata dell'orchestrazione — l'applicazione (nucleo e interfaccia) e il runtime del modello generativo — con lo stato su volumi dedicati (base dati e pesi dei modelli) e tutta la configurazione iniettata da variabili d'ambiente: la stessa immagine gira ovunque cambiando solo la configurazione. L'applicazione è l'unica superficie esposta al DPO; i dati trattati non lasciano la rete privata (minimizzazione, §2.3.5).

7.10.3 Contratti e direzione

Il disaccoppiamento è garantito dai contratti già introdotti nell'architettura: `PIIDetector` per i rilevatori, `CategoryMapper` per la normalizzazione del ROPA, un client astratto per il modello generativo. Poiché ogni funzionalità dipende solo dalla *forma* del contratto e non dalla sua collocazione, **spostare una funzionalità in un sotto-container dedicato ha un costo quasi nullo**. La direzione è quindi l'articolazione progressiva in **sotto-container per funzionalità**: il runtime generativo è già un servizio a sé; il livello NER, oggi in-process, è scorporabile in un servizio dedicato qualora se ne debba scalare il carico; la persistenza migrerà da SQLite a una base dati relazionale in rete (ad esempio PostgreSQL) quando si superi il singolo nodo. Rimane come sviluppo la pubblicazione di immagini **multi-architettura**

(amd64/arm64). Il container costituisce la via canonica per eseguire il sistema completo, dipendenze di apprendimento automatico incluse, mentre il nucleo leggero resta eseguibile nativamente per l'iterazione rapida di sviluppo.

Capitolo 8

Test e validazione

Questo capitolo è, insieme allo studio sui modelli generativi, uno dei due pilastri sperimentali della tesi, ed è organizzato per gradi di specializzazione crescente. Prima si dà conto della **verifica funzionale** del sistema (i test di unità che accompagnano il codice); poi si fissa il **metodo di valutazione** (corpus, regola di *matching*, metriche); quindi si misura il **sistema di rilevamento nel suo insieme**, scomponendone il contributo per singolo rilevatore e confrontando l'approccio tradizionale con quello generativo; infine si conduce uno **studio mirato sui modelli generativi**, dove la qualità del rilevamento è pesata contro il suo costo in tempo di inferenza ed energia.

8.1 Verifica funzionale: i test di unità

I test di unità accompagnano il codice del motore di rilevamento. Ogni area funzionale è verificata da una propria suite, scritta contestualmente allo sviluppo ed eseguita per intero a ogni modifica come **verifica di non regressione**, accanto al controllo statico dei tipi e all'analisi statica del codice; una modifica è considerata conclusa solo quando tutti e tre i controlli sono superati. Questi test verificano la *correttezza* del rilevamento, mentre la sua *qualità* è oggetto delle sezioni successive. Al momento della stesura la suite conta **388 casi**, tutti superati (~16 secondi in *container*); la Tabella 8.1 ne riepiloga la distribuzione per area. Il dettaglio dei singoli casi è documentato nella reference tecnica generata dai docstring del codice.

Tabella 8.1: Distribuzione dei test di unità per area del sistema (388 casi, tutti superati).

Area	N. casi
Rilevamento (detection)	118
Valutazione (evaluation)	76
Ingestione ROPA (ropa)	49
Registro PII (registry)	48
Interfaccia web (web)	36
Estrazione (extraction)	29
Conformità (compliance)	26
Client LLM (llm)	6
Totale	388

8.2 Generazione dei corpus sintetici

La validazione del rilevamento richiede documenti annotati — testo di cui si conosca in anticipo, con certezza, dove sono le PII e di che tipo. Non esistono *benchmark* pubblici adatti, e usare documenti reali sarebbe in contraddizione con l'oggetto stesso del lavoro (trattare PII reali per costruirne uno strumento di tutela); la soluzione, prevista in sede di analisi dei rischi (§4.1.1), è **generare corpus sintetici e riproducibili a partire da un seed**. Questa sezione ne descrive la realizzazione. In tutti i corpus il *gold* è **derivato dalla stessa sorgente** che produce il testo, così che annotazione e documento restino allineati per costruzione.

Annotazione in-line e *gold* Ogni PII è marcata nel testo con `{{pii_type:value}}` — per esempio `{{iban:IT60X05...}}`. Un unico *loader* rimuove i marcatori e restituisce al tempo stesso il testo pulito su cui girano i rilevatori e gli *span* di *ground truth* nelle coordinate di quel testo. Poiché entrambi sono calcolati dal medesimo documento in un solo passaggio, gli *offset* non richiedono alcun conteggio manuale e il *gold* resta allineato al testo. Su questa convenzione poggiano tutti i corpus descritti sotto.

Valori realistici e validi al *checksum* Il generatore (`corpus_generator`) produce documenti annotati con valori **realistici** — nomi, indirizzi, condizioni sanitarie, prodotti da *Faker* — e, dove la categoria prevede una cifra di controllo, **aritmeticamente validi**: IBAN (regola *mod-97* ISO 7064), numero di carta (algoritmo di Luhn), numero AVS svizzero (EAN-13), codice fiscale italiano (generato con la libreria `python-codicefiscale`). La validità al *checksum* è una condizione necessaria alla correttezza della misura: i riconoscitori di Presidio *verificano* quelle cifre di controllo, e scarterebbero in silenzio un valore formalmente plausibile ma incoerente. Il generatore diverrebbe così la causa di un mancato rilevamento, gonfiando artificiosamente i falsi negativi (§6.4 rimanda a questa proprietà già in fase di architettura). Per non dipendere da Presidio nel garantirla, ogni valore è **ri-verificato** con algoritmi puri e

indipendenti prima di entrare nel corpus. La generazione è governata da un **seme**: a parità di seme il corpus è identico, quindi ogni misura è riproducibile.

Resa in documenti reali Per valutare la pipeline *end-to-end* — estrazione compresa — il testo generato è reso in file veri (*render*): i marcatori sono rimossi, il testo pulito è scritto in **PDF/DOCX** e, accanto ai file, è emesso un *gold* per **valore** (*gold.jsonl*, coppie (*pII_type*, *value*)). La granularità passa dagli *span* ai valori, perché l'estrazione sposta gli *offset* e la posizione originale non è più recuperabile; la sorgente resta comunque unica, e i file e il *gold* sono **derivati nello stesso passaggio**. La resa è deliberatamente semplice (colonna singola, *born-digital*) e serve a esercitare il collegamento estrazione → rilevamento su documenti nativi digitali. La complessità tipografica reale (multi-colonna, tabelle, scansioni) richiederebbe OCR e analisi di *layout* ed è dichiarata fuori perimetro (§10.3).

Il corpus aziendale (scala e falsi positivi) Accanto ai due corpus «piatti», un terzo generatore costruisce da un seme un intero **albero di cartelle** aziendale (dell'ordine delle centinaia di documenti, da poche righe a decine di pagine), pensato per la valutazione a scala e per lo stress della scansione ricorsiva. Tre sue proprietà sono metodologicamente rilevanti. Anzitutto è **pianificato come valore**: una funzione pura (*plan_corpus*) decide in anticipo quali file esistono, con quale contenuto, lunghezza e data, senza toccare il disco, sicché le proprietà che contano (riproducibilità, mix di lunghezze, quota di documenti senza PII, copertura delle categorie) sono verificabili in millisecondi. In secondo luogo contiene una **quota deliberata di documenti privi di qualsiasi PII** (verbali, listini, manuali) con elementi **distrattori** (protocolli, importi, codici articolo): su un corpus in cui ogni documento contiene PII i **falsi positivi** non sarebbero osservabili (§4.2.2). Infine il **rumore è dichiarato**: ogni file volutamente problematico (formati non supportati, PDF vuoto o troncato, file di lock) porta con sé nel *manifest* il comportamento atteso, sicché il test ne asserisce l'esito esatto.

8.3 Metodo di valutazione

Corpus e granularità del confronto La qualità del rilevamento è misurata sul corpus **sintetico aziendale** descritto sopra (§8.2). Il corpus è riproducibile da seed, il suo *gold* è certo e non comporta il trattamento di PII reali. È lo stesso corpus per tutte le misure del capitolo, così che ogni tabella sia confrontabile con le altre. Due sue proprietà lo rendono adatto al ruolo. La prima è la scala: alcune centinaia di documenti e alcune migliaia di occorrenze annotate danno stime sufficientemente stabili anche disaggregando per categoria. La seconda è la presenza deliberata di documenti *privi* di PII, in assenza dei quali i falsi positivi non sarebbero osservabili (§4.2.2).

La **granularità del confronto** varia invece a seconda che l'estrazione sia o meno in gioco. Dove i rilevatori operano sul testo pulito il *gold* è *posizionale*: ogni PII porta il proprio

`pII_type` e i propri estremi nel testo, e il confronto avviene per sovrapposizione di *span*. È il caso di tutte le tabelle per detector. Dove invece il testo è stato prima estratto da un file, gli *offset* originali non sono più disponibili e il confronto avviene per **valore**. Le due granularità sono chiamate nel seguito **Tier 1** e **Tier 2**. Il loro scarto misura, per costruzione, il degrado imputabile alla sola estrazione (§8.4.4).

Metrica e regola di *matching* In Tier 1 un rilevamento conta come vero positivo quando condivide il `pII_type` con uno *span* di *ground truth* e vi si sovrappone con *Intersection over Union* ≥ 0.5 ; ogni rilevamento e ogni *span* sono appaiati al più una volta (*matching greedy*, le coppie a sovrapposizione maggiore per prime). In Tier 2, dove l'estrazione ha già spostato gli *offset*, il confronto è per **valore normalizzato** (multiinsieme, spazi e maiuscole normalizzati) anziché per posizione. In entrambi i casi si derivano *precision*, *recall* e **F1 micro-mediati** — le tre grandezze definite in §3.1.4 — riportati sia complessivamente sia **disaggregati per categoria**, perché una categoria scoperta è un difetto anche quando l'indicatore complessivo è buono (§4.2.1). Nelle tabelle che seguono le colonne **P**, **R** e **F1** riportano queste metriche, mentre **TP**, **FP** e **FN** sono i conteggi assoluti da cui esse discendono: veri positivi, falsi positivi e falsi negativi contati come **occorrenze di PII**, non come documenti. Ogni configurazione è incapsulata dietro il contratto `PIIDetector` e misurata sullo stesso corpus, così da confrontarle sullo stesso metro.

Estensione della misura e campionamento I rilevatori tradizionali sono misurati sull'intero corpus: una passata di pattern e NER su tutti i documenti richiede tempi dell'ordine dell'ora, compatibili con l'impiego della scala completa. Per il rilevatore generativo i tempi sono di altro ordine: l'inferenza su CPU procede a decine di secondi per documento, e alle latenze misurate una passata sull'intero corpus richiederebbe alcune ore per ciascuna configurazione. Poiché le configurazioni sono due per modello, misurare l'intero corpus per i modelli confrontati comporterebbe oltre venti ore di tempo macchina continuativo. Le configurazioni generative sono perciò valutate su un **campione casuale** di 20 documenti (315 occorrenze annotate), estratto con seme fisso e quindi riproducibile come il resto dell'apparato.

Due precisazioni consentono di verificare la scelta. La prima riguarda il *modo* in cui il campione è estratto, ossia per sorteggio. Un corpus organizzato ad albero è infatti **ordinato**, e le sue prime cartelle contengono un solo genere di documenti: prelevarne la parte iniziale escluderebbe intere categorie, e con esse i documenti privi di PII, sui quali soltanto i falsi positivi sono osservabili. Il sorteggio con seme fisso soddisfa insieme le due esigenze, rappresentatività e riproducibilità.

La seconda precisazione riguarda l'accertamento di tale rappresentatività, condotto per due vie indipendenti. La prima è interna al *benchmark* generativo: ogni sua esecuzione ricalcola sul campione anche la riga della pipeline tradizionale, sicché due corse condotte in momenti distinti forniscono due misure indipendenti della stessa configurazione sullo

stesso sottoinsieme. Il loro accordo — $F1$ 0.79 e 0.80 — indica che la misura è stabile. La seconda via è esterna: gli stessi rilevatori tradizionali sono misurati sia sull'intero corpus sia sul campione, e lo scarto fra le due cifre quantifica l'eventuale distorsione introdotta dal sottoinsieme.

Il confronto è stato eseguito e la distorsione risulta misurabile. Sull'unione di pattern e NER il campione dà $F1$ 0.77 contro lo 0.73 del corpus completo, con una **sovrastima di** 0.04 concentrata interamente sulla precisione (0.63 contro 0.57), mentre la recall coincide (0.98 contro 0.99). La causa risiede nella composizione del sottoinsieme, dichiarata più avanti: contenendo una quota minore di documenti privi di PII, esso offre meno occasioni di produrre falsi positivi. La distorsione ha però verso variabile. Presi separatamente, il livello a pattern risulta sovrastimato dal campione ($F1$ 0.71 contro 0.63) e il NER *sottostimato* ($F1$ 0.49 contro 0.53); si tratta perciò di rumore campionario, che nella fusione si compensa in parte, più che di un errore sistematico.

Ne discende il criterio di lettura delle cifre generative. Esse valgono come **confronto fra configurazioni misurate sullo stesso terreno**, che è la domanda a cui il capitolo deve rispondere; trasferite all'intero corpus tenderebbero invece a risultare inferiori di qualche centesimo. La recall, che per questo dominio è la dimensione più rilevante, resta comunque inalterata dal campionamento.

Due limiti del campionamento vanno infine dichiarati. Anzitutto le categorie meno frequenti vi hanno un supporto esiguo, e una può mancare del tutto: nel campione non compare alcun numero di carta di credito, categoria che nel corpus completo conta 38 occorrenze. Per questo le tabelle riportano accanto alle metriche i **conteggi assoluti**, che indicano su quante occorrenze poggia ciascuna riga. In secondo luogo il campione contiene 3 documenti privi di PII su 20, contro i 96 su 300 del corpus completo: è la sproporzione che spiega lo scarto di precisione appena riportato.

Le grandezze di **costo** sono esenti da questa cautela, in quanto proprietà del modello e della macchina anziché del corpus: i secondi e i wattora per finestra di testo non dipendono da quante finestre si misurino. Le cifre di costo per documento e per migliaia di documenti sono quindi ottenute per estrapolazione dalla misura per finestra, e vanno lette come tali.

Assi di costo (solo per l'AI) Per lo studio sui modelli generativi (§8.5) alla qualità si affiancano tre grandezze di costo: i **secondi per documento** (il tempo effettivamente trascorso durante la passata di rilevamento, ripartito sui documenti del corpus), i **wattora per documento** (stimati con `codecarbon` [91]) e, da questi, una **stima monetaria** del costo energetico (§8.5), così da riprendere sul sistema reale il confronto costo-per-inferenza già impostato in letteratura (§5.1). Le cifre energetiche hanno valore **comparativo**: dove manca il conteggio hardware (RAPL, non disponibile nella VM) `codecarbon` ricade su una stima basata sul TDP, e l'inferenza gira nel *container* Ollama adiacente. Vanno perciò confrontate *fra loro* e non assunte come consumo reale.

8.3.1 Ambiente e condizioni di prova

Poiché i tempi e i consumi misurati dipendono strettamente dall'hardware e dalla configurazione, se ne dà conto per intero nella Tabella 8.2: senza questi parametri le cifre di §8.5 non sarebbero riproducibili né interpretabili. Tre condizioni meritano di essere segnalate, perché determinano l'*ordine di grandezza* dei risultati.

Tabella 8.2: Ambiente di prova: hardware, risorse del *container* e parametri di inferenza.

Componente	Configurazione
Host	MacBook Pro (Mac14,9), Apple M2 Pro
CPU	10 core (6 <i>performance</i> + 4 <i>efficiency</i>)
Memoria host	16 GB unificata
Runtime	Docker Desktop — VM Linux (<code>docker compose</code>)
Risorse assegnate alla VM	10 vCPU, ~11.7 GiB RAM (12.5 GB)
Accelerazione	nessuna: inferenza solo CPU (GPU non esposta alla VM)
Motore LLM	Ollama (<code>llama.cpp</code>), modelli GGUF quantizzati
Modelli residenti	1 alla volta (<code>OLLAMA_MAX_LOADED_MODELS=1</code>)
Motore NER	GLiNER (<i>encoder</i>), su CPU
Tetto di generazione	1024 token per richiesta (<code>num_predict</code>)
Stima energetica	codecarbon (OfflineEmissionsTracker, paese ITA)

Inferenza esclusivamente su CPU Docker Desktop su macOS **non espone la GPU integrata** dell'Apple M2 Pro alla VM Linux in cui girano i *container*: né l'*encoder* NER né i modelli generativi vengono accelerati, e ogni inferenza è svolta dai soli 10 core CPU. La condizione è *voluta*: coincide con il vincolo di progetto di un modello «piccolo, eseguibile su CPU, su hardware *commodity*» (§5.4) e riproduce lo scenario di *deployment* atteso. Da essa dipende anche l'ordine di grandezza delle latenze generative, che si misurano in *decine di secondi* a documento; su GPU si ridurrebbero di circa un ordine di grandezza (§8.5.1), ma quello non è l'ambiente bersaglio.

Memoria e un solo modello per volta Alla VM sono assegnati ~11.7 GiB dei 16 dell'host: sufficienti per la fascia ~4B, insufficienti per quella da 12B (§8.5). La risorsa critica in questo ambiente è dunque la memoria. Ollama è inoltre configurato per tenere **un solo modello residente** (`OLLAMA_MAX_LOADED_MODELS=1`): il benchmark ne carica più d'uno in sequenza e, senza questo vincolo, i modelli si accumulerebbero in memoria fino a saturarla.

Tetto di generazione Il budget di inferenza è fissato da un **tetto di 1024 token generati** per richiesta (`num_predict`) anziché da un limite di tempo. La motivazione è duplice. Anzitutto *semantica*: l'output atteso è una breve lista JSON di valori, per la quale poche centinaia di token sono sufficienti, sicché un tetto generoso lascia intatta ogni risposta legittima. In secondo luogo *di costo*: senza tetto un modello *reasoning* può emettere migliaia di token

di catena-di-pensiero prima di rispondere, con latenza fuori scala (nel corso delle prove un singolo documento ha richiesto oltre 4500 token, ~13 minuti su CPU) e uscita spesso non interpretabile. Un tetto espresso in token è inoltre **deterministico** — indipendente dal carico istantaneo della macchina, quindi riproducibile — e impone le stesse condizioni a ogni modello, mentre un limite a tempo varierebbe con il carico. Il prezzo è che un modello incapace di rispondere entro il budget viene troncato e non produce nulla, come mostra il caso di qwen3:4b (§8.5). Si noti infine che il campionamento dei documenti descritto sopra è una scelta *sperimentale*, distinta e indipendente dal campionamento 1-su- N previsto in esercizio (§7.5), che è invece una politica di *deployment*.

8.4 Il sistema di rilevamento nel suo insieme

Questa sezione misura il livello di rilevamento (blocco B4) e chiude su base empirica le scelte lasciate aperte in fase di implementazione: costruire il livello regex/NER sopra Presidio (§5.3) e adottare GLiNER come motore di NER (§5.4). La lettura procede per gradi: prima il contributo di **ciascun rilevatore** preso da solo (§8.4.1), poi il confronto fra l'approccio **tradizionale e quello agentico** (§8.4.2), quindi il **sistema completo** con tutti i metodi fusi (§8.4.3), e infine la validazione end-to-end con il **degrado da estrazione** (§8.4.4).

8.4.1 Contributo dei singoli rilevatori

Le tre tabelle che seguono riportano, per ciascun rilevatore preso isolatamente, che cosa riconosce e con quali statistiche, disaggregate per categoria. I due rilevatori tradizionali sono misurati sull'intero corpus, quello generativo sul campione, per le ragioni di costo esposte sopra (§8.3). Ne emergono tre strumenti dalle competenze in larga misura complementari.

Il **rilevatore a pattern** (Tabella 8.3) è *selettivo*: la sua precisione complessiva è 0.88, mentre la recall si ferma a 0.49. Sulle categorie a formato fisso il comportamento è netto: indirizzo di posta elettronica, IBAN, codice fiscale e numero AVS svizzero sono riconosciuti tutti con precisione e recall pari a 1.00, la carta di credito con precisione 1.00 e recall 0.74. Sulle categorie prive di forma regolare il rilevatore non dispone invece di alcun criterio di riconoscimento, e nome di persona, indirizzo postale e dato sanitario restano interamente fuori dalla sua portata.

Due categorie strutturate, che ci si attenderebbe di trovare qui, sono rilevate solo in parte: il numero di *telefono* nel 16% dei casi e l'indirizzo *IP* nel 36%, poiché il corpus adotta convenzioni di scrittura che le espressioni del riconoscitore non prevedono. Vi si aggiunge la *data di nascita*, sulla quale il rilevatore produce 137 segnalazioni, nessuna delle quali corretta: ne riconosce la forma generica, mai quella attesa. I tre casi illustrano la fragilità propria degli approcci a regole, la cui copertura dipende dall'aver previsto in anticipo la variante corretta (§4.2.1).

Il **rilevatore NER** (Tabella 8.4) presenta il profilo speculare. Copre le categorie contestuali che sfuggono ai pattern — dato sanitario con F1 1.00, data di nascita 0.99, indirizzo postale 0.72 — e recupera proprio il telefono e l'indirizzo IP, dove raggiunge 0.86 e 0.79. Sugli identificatori a formato rigido risulta invece inaffidabile: posta elettronica 0.07, IBAN 0.28, codice fiscale 0.39, numero AVS nullo. Introduce inoltre falsi positivi in misura preponderante sui nomi di persona, dove ne concentra 1316 dei 1733 complessivi, ossia il 76%; sulla stessa categoria la sua recall è però 1.00, valore che nessun altro rilevatore raggiunge.

Il **rilevatore generativo** (Tabella 8.5) si colloca in posizione intermedia, con una copertura estesa a quasi tutte le categorie ma parziale su ciascuna. Va letto tenendo presente che il campione su cui è misurato non contiene numeri di carta di credito: le due segnalazioni della riga `credit_card` sono quindi prive di riscontro nel *gold* e non riguardano occorrenze reali.

Nessuno dei tre rilevatori è sufficiente da solo: le recall complessive valgono 0.49 per i pattern, 0.57 per il NER e 0.85 per il modello generativo. Pattern e NER, in particolare, coprono insieme di categorie in larga parte disgiunti — là dove l'uno raggiunge 1.00 l'altro è quasi sempre prossimo a zero, e viceversa — ed è la loro unione a portare la recall a 0.99, come mostra il confronto che segue (§8.4.2). Si osservi infine che una sola categoria, il numero AVS, è coperta dal solo livello a pattern, per mezzo di una regola aggiunta in configurazione: è l'identificatore più caratteristico del contesto normativo svizzero, e gli altri due rilevatori non lo riconoscono.

Tabella 8.3: Rilevatore a pattern (riconoscitori di Presidio più le regole di `custom_patterns.yaml`), per categoria, sull'intero corpus: 300 documenti, 2918 occorrenze annotate.

Categoria	P	R	F1	TP	FP	FN
address	0.00	0.00	0.00	0	0	253
credit_card	1.00	0.74	0.85	28	0	10
date_of_birth	0.00	0.00	0.00	0	137	247
email	1.00	1.00	1.00	532	0	0
health_data	0.00	0.00	0.00	0	0	18
iban	1.00	1.00	1.00	355	0	0
ip_address	1.00	0.36	0.53	45	0	80
italian_id	1.00	1.00	1.00	402	0	0
person_name	0.00	0.00	0.00	0	0	533
phone	0.56	0.16	0.24	63	50	339
swiss_avs	1.00	1.00	1.00	13	0	0
OVERALL	0.88	0.49	0.63	1438	187	1480

Tabella 8.4: Rilevatore NER (GLiNER, `gliner_multi_pii`), per categoria, sull'intero corpus: 300 documenti, 2918 occorrenze annotate.

Categoria	P	R	F1	TP	FP	FN
address	0.57	0.98	0.72	249	187	4
credit_card	0.83	0.26	0.40	10	2	28
date_of_birth	0.97	1.00	0.99	247	7	0
email	0.58	0.04	0.07	21	15	511
health_data	1.00	1.00	1.00	18	0	0
iban	0.87	0.17	0.28	59	9	296
ip_address	0.97	0.66	0.79	83	3	42
italian_id	0.43	0.35	0.39	141	188	261
person_name	0.29	1.00	0.45	533	1316	0
phone	0.98	0.77	0.86	310	6	92
swiss_avs	0.00	0.00	0.00	0	0	13
OVERALL	0.49	0.57	0.53	1671	1733	1247

Tabella 8.5: Rilevatore generativo (gemma3:4b), per categoria, sul campione (§8.3).

Categoria	P	R	F1	TP	FP	FN
address	0.89	0.84	0.86	16	2	3
credit_card	0.00	0.00	0.00	0	2	0
date_of_birth	0.69	0.75	0.72	9	4	3
email	0.92	0.85	0.89	70	6	12
health_data	0.00	0.00	0.00	0	0	2
iban	1.00	0.92	0.96	60	0	5
ip_address	1.00	1.00	1.00	5	0	0
italian_id	0.71	0.94	0.81	17	7	1
person_name	0.41	0.70	0.52	21	30	9
phone	1.00	0.86	0.92	66	0	11
swiss_avs	1.00	1.00	1.00	5	0	0
OVERALL	0.84	0.85	0.85	269	51	46

8.4.2 Rilevamento tradizionale contro agentic

Riuniti i rilevatori nella pipeline di fusione (MergeEngine), la Tabella 8.6 confronta l'approccio **tradizionale**, ossia Presidio come somma di pattern e NER, con la sua estensione **agentic**, in cui il rilevatore generativo interviene come secondo parere. L'unione di pattern e NER conferma la complementarità appena descritta: la pipeline tradizionale raggiunge F1 0.79 con una recall di 0.99, superiore a quella di ciascuna componente presa separatamente, poiché ognuna copre le categorie che sfuggono all'altra. L'esito motiva l'adozione di Presidio con GLiNER (§5.3, §5.4).

Il confronto con le righe agentiche costituisce il dato di maggiore rilievo per il presente lavoro: **l'aggiunta del parere generativo non migliora la pipeline**, con nessuno dei due modelli. Le due fusioni ottengono F1 0.76 e 0.79, a fronte dello 0.79 della baseline. Il caso di *gemma3:4b* ne mostra il meccanismo: la fusione porta la recall a 1.00 e individua ogni occorrenza annotata, mentre la precisione scende da 0.66 a 0.61, perché ai falsi positivi della componente tradizionale si sommano quelli della componente generativa. Sul versante della *qualità* la domanda «l'AI generativa vale il suo costo?» riceve qui una prima risposta; il versante del *costo* la rende più netta (§8.5).

Tabella 8.6: Rilevamento tradizionale contro agentico: metriche complessive sul campione (P, R, F1).

Configurazione	P	R	F1
Presidio — solo pattern/checksum	0.94	0.57	0.71
Presidio — solo NER (GLiNER)	0.50	0.48	0.49
Presidio (pattern + NER) — tradizionale	0.66	0.99	0.79
AI generativa (<i>gemma3:4b</i>) da sola	0.84	0.85	0.85
AI generativa (<i>phi4-mini</i>) da sola	0.95	0.50	0.66
Presidio + AI (<i>gemma3:4b</i>) — <i>agentico</i>	0.61	1.00	0.76
Presidio + AI (<i>phi4-mini</i>) — <i>agentico</i>	0.66	0.99	0.79

8.4.3 Il sistema completo

La Tabella 8.7 riporta il sistema nella configurazione in cui verrebbe eseguito in esercizio, con pattern, NER e AI (*gemma3:4b*) fusi, letto per categoria. La copertura è **completa**: la recall è 1.00 su ogni categoria e nessuna occorrenza annotata sfugge al sistema. Il costo di tale copertura è la precisione, che si ferma a 0.61. Il quadro per categoria mostra un difetto concentrato: *person_name* contribuisce da sola 139 dei 199 falsi positivi complessivi, con una precisione di 0.18, e *italian_id* ne aggiunge altri 24; insieme a *address* e *date_of_birth* queste quattro categorie raccolgono il 91% dei falsi positivi. Le sette categorie restanti ne totalizzano 17 in tutto, con precisioni comprese fra 0.83 e 1.00; fa eccezione *credit_card*, la cui precisione nulla poggia su due sole segnalazioni in un campione privo di numeri di carta. La sovra-segnalazione dei nomi di persona è del resto un comportamento ricorrente dei riconoscitori di entità, che privilegiano il richiamo: un NER a *transformer* dedicato ai nomi raggiunge richiamo 0.87 a precisione 0.55 [92]. Anche in questa configurazione l'esito resta inferiore a quello della pipeline tradizionale (§8.4.2): su documenti *born-digital* e ben formati il livello generativo non aggiunge copertura, mentre i falsi positivi che introduce si sommano a quelli già presenti.

Tabella 8.7: Sistema completo (pattern + NER + Al gemma3:4b, fusi), per categoria, sul campione.

Categoria	P	R	F1	TP	FP	FN
address	0.63	1.00	0.78	19	11	0
credit_card	0.00	0.00	0.00	0	2	0
date_of_birth	0.60	1.00	0.75	12	8	0
email	0.94	1.00	0.97	82	5	0
health_data	1.00	1.00	1.00	2	0	0
iban	0.93	1.00	0.96	65	5	0
ip_address	0.83	1.00	0.91	5	1	0
italian_id	0.43	1.00	0.60	18	24	0
person_name	0.18	1.00	0.30	30	139	0
phone	0.95	1.00	0.97	77	4	0
swiss_avs	1.00	1.00	1.00	5	0	0
OVERALL	0.61	1.00	0.76	315	199	0

8.4.4 Validazione end-to-end e degrado da estrazione

Le misure viste finora isolano il *rilevamento*, applicandolo a testo già disponibile in forma pulita. In esercizio, però, il testo arriva ai rilevatori solo dopo essere stato estratto da un file, e quello stadio può perdere informazione che il rilevatore avrebbe altrimenti individuato. Questa sezione isola il **costo introdotto dall'estrazione** (§6.4): la stessa pipeline è valutata in *Tier 1*, sul testo pulito, e in *Tier 2*, sul testo che il blocco B3 estrae dai file veri del corpus — PDF, DOCX e TXT insieme, dalle poche righe alle decine di pagine. Lo scarto fra le due F1 misura il **degrado da estrazione**, ossia la quota di prestazione persa nel solo passaggio attraverso quello stadio; un file che B3 non riesca a leggere vi contribuisce come mancato rilevamento, secondo il criterio di contabilizzazione adottato.

Una precisazione precede la lettura dei numeri. Poiché l'estrazione sposta gli *offset*, il confronto avviene qui per **valore** e non per posizione (§8.3). Il criterio è più severo di quello adottato nelle tabelle precedenti: là un rilevamento che copra almeno metà dello *span* atteso conta come corretto, qui il valore restituito deve coincidere con quello annotato. Le due righe della Tabella 8.8 vanno perciò confrontate soltanto *fra loro*, che è il confronto qui rilevante; lo scarto rispetto alle tabelle per rilevatore si concentra sulle categorie dai confini incerti, di cui l'indirizzo postale — che il NER delimita spesso in modo diverso dall'annotazione — è l'esempio principale.

L'esito è che l'estrazione **non comporta alcun degrado misurabile**: le due righe coincidono su tutte le metriche e il degrado vale -0.001 . Il segno negativo indica che il testo estratto produce qualche falso positivo in meno di quello pulito (1999 contro 2012), poiché l'estrazione normalizza spaziature e interruzioni di riga e fa cadere con esse alcune segnalazioni spurie. La variazione è dell'ordine del millesimo e rientra nel rumore: sui documenti *born-digital*

lo stadio di estrazione risulta quindi sostanzialmente trasparente. Ne discende inoltre che nessun file del corpus è risultato illeggibile in misura rilevante; in caso contrario le occorrenze contenute comparirebbero come mancati rilevamenti e la riga Tier 2 mostrerebbe una recall inferiore.

Tabella 8.8: Validazione end-to-end sull'intero corpus (300 documenti, 2918 occorrenze): Tier 1 (testo pulito) contro Tier 2 (testo estratto dai file). Confronto per valore.

Configurazione	P	R	F1	TP	FP	FN
Tier 1 — testo pulito (solo rilevamento)	0.57	0.92	0.71	2694	2012	224
Tier 2 — testo estratto dai file (B3 + rilevamento)	0.57	0.92	0.71	2695	1999	223
<i>Degrado da estrazione</i> (F1 Tier 1 – F1 Tier 2): –0.001						

Limiti Il corpus è sintetico e riproducibile da seme: più ordinato del reale, fornisce un **limite superiore** delle prestazioni attese sul campo, e i valori vanno letti come direzionali (§4.1.1). Le configurazioni generative, inoltre, sono misurate su un campione e non sull'intero corpus (§8.3): le loro cifre portano l'incertezza propria di una stima campionaria, tanto più ampia quanto meno frequente è la categoria, e servono a *confrontare* configurazioni sullo stesso terreno più che come stime assolute. Resta infine l'assenza, per costruzione, di documenti scansionati, che metterebbero alla prova l'estrazione in un regime che questo corpus non rappresenta (§10.3).

8.5 Studio sui modelli generativi: qualità contro costo

La sezione precedente ha mostrato che, sul piano della sola qualità, il secondo parere generativo non migliora la pipeline tradizionale. Al criterio di qualità si affianca quello di **costo**, che il pilastro energetico della tesi impone di considerare. Questa sezione lo misura riportando, accanto a P/R/F1, i secondi e i wattora per documento. Per ciascun modello si riportano la riga del modello **da solo** (*ai:*) e quella della **fusione** con i rilevatori tradizionali (*union:*); la baseline tradizionale (senza AI) è la riga di riferimento.

Selezione dei modelli Le domande a cui lo studio deve rispondere sono due, e vanno tenute distinte: (i) un modello generativo aggiunge qualità alla pipeline tradizionale, e a quale prezzo? (ii) una taglia maggiore è una via percorribile per migliorare l'esito? Il numero e la scelta dei modelli misurati discendono da queste due domande.

Alla prima domanda un solo modello non basterebbe, perché un esito negativo sarebbe indistinguibile dalla scelta di un modello inadatto al compito. Il confronto è perciò condotto su **due famiglie indipendenti** della medesima fascia di taglia (~4B) — *gemma3:4b* e *phi4-mini* — sviluppate separatamente, con dati e procedure di addestramento diversi. Se convergono sullo stesso esito, la conclusione riguarda l'approccio generativo su questo compito e non il singolo modello; se divergono, la divergenza stessa costituisce il dato da riportare.

Alla seconda domanda si è tentato di rispondere sul piano della qualità, mediante un confronto **controllato** fra `gemma3:4b` e `gemma3:12b`. I due modelli appartengono alla stessa famiglia e alla stessa quantizzazione e differiscono per il solo numero di parametri; un eventuale divario sarebbe perciò attribuibile alla taglia, attribuzione che un confronto fra famiglie diverse non consentirebbe. Il tentativo si è arrestato prima di produrre una misura, poiché nella configurazione di riferimento il modello da 12B **non è eseguibile**; il paragrafo dedicato ne documenta la ragione, che costituisce essa stessa la risposta alla domanda. La questione della taglia riceve dunque risposta sull'asse della *praticabilità*, e la fascia superiore esce dal confronto quantitativo.

Il confine di questa conclusione va delimitato con precisione: **non si afferma che un modello da 12B produrrebbe risultati peggiori**. Si constata che nel perimetro che il progetto si è dato — esecuzione locale su hardware aziendale ordinario, imposta dalla minimizzazione del dato e non da una preferenza (§2.3.1, §2.3.2) — quel modello non è disponibile. Una macchina dotata di maggiore memoria sposterebbe il perimetro, lasciando invariato il ragionamento.

Perimetro del confronto Restano fuori dal confronto quantitativo la fascia intermedia (~7–8B) e i modelli *reasoning*. Misurare la prima avrebbe documentato un punto isolato fra due estremi di cui uno non è raggiungibile, senza con ciò ricostruire l'andamento della qualità al crescere della taglia. Sui secondi, una verifica preliminare condotta in fase di messa a punto dell'apparato (`qwen3:4b`) ne aveva mostrato l'incompatibilità con il tetto di generazione adottato (§8.3.1), e se ne dà conto più avanti come osservazione autonoma, esterna alla tabella.

Tabella 8.9: Studio multi-modello: qualità (P/R/F1) contro costo (s/doc, Wh/doc, CHF/1000 doc). Due famiglie indipendenti della fascia ~4B; la fascia superiore è esclusa perché non eseguibile nella configurazione di riferimento (Tabella 8.10). Baseline = sistema senza AI.

Configurazione	P	R	F1	s/doc	Wh/doc	CHF/1k doc
Presidio (pattern + NER) — baseline	0.66	0.99	0.79	11.8	0.066	0.018
<code>ai:gemma3:4b</code>	0.84	0.85	0.85	79.1	0.763	0.211
<code>union:gemma3:4b</code>	0.61	1.00	0.76	82.7	0.759	0.210
<code>ai:phi4-mini</code>	0.95	0.50	0.66	40.4	0.405	0.112
<code>union:phi4-mini</code>	0.66	0.99	0.79	50.1	0.477	0.132

Finestre di testo perse I run riportati nella Tabella 8.9 hanno registrato alcune finestre di testo la cui analisi è fallita per terminazione del processo di inferenza, e che il rilevatore salta proseguendo la scansione: cinque per `gemma3:4b` e due per `phi4-mini`, su circa sessantasei finestre per passata. Le righe generative sottostimano quindi lievemente la copertura dei rispettivi modelli, nell'ordine di qualche punto percentuale. L'effetto è sfavorevole al modello e lascia invariate le conclusioni, che poggiano sulle righe di fusione. Per questa ragione il

rilevatore espone un contatore esplicito delle finestre perse, che segnala quando una misura sia stata raccolta su una copertura parziale.

Ordine di grandezza del costo Il primo dato è comune a entrambi i modelli: la baseline tradizionale rileva a 11.8 s/doc e 0.066 Wh/doc, mentre ogni passata generativa richiede fra 40 e 83 s/doc e fra 0.41 e 0.76 Wh/doc. Il divario è di un ordine di grandezza — da ~ 3 a $\sim 7\times$ in tempo e da ~ 6 a $\sim 12\times$ in energia — e va letto tenendo presente che entrambi i termini crescono con la lunghezza del documento: su documenti reali anche il rilevamento tradizionale, che esegue l'*encoder* NER sull'intero testo, costa sensibilmente più che su testi brevi.

Conversione del consumo in costo Per riprendere in sede di conclusioni il confronto economico impostato in letteratura (§5.1), i wattora sono tradotti in una stima monetaria. La conversione è immediata: l'energia per 1000 documenti in kWh coincide numericamente con i Wh per documento, e basta moltiplicarla per il prezzo dell'energia. Assumendo la tariffa mediana svizzera del 2026, pari a 27.7 ct./kWh per un'economia domestica tipo secondo la Commissione federale dell'energia elettrica [93]¹, si ottiene l'ultima colonna della Tabella 8.9: il rilevamento tradizionale costa ~ 0.018 CHF ogni 1000 documenti, la passata generativa fra ~ 0.11 e ~ 0.21 CHF — lo stesso rapporto visto sull'energia. Due precisazioni sono necessarie. La prima riguarda la natura della grandezza: si tratta del costo *marginale di sola energia* su hardware *posseduto*, complementare e non assimilabile al prezzo per interrogazione di un'API *cloud* (§5.1), che incorpora anche l'ammortamento dell'hardware e il margine del fornitore. La seconda riguarda la scala: in valore assoluto il costo energetico è *piccolo*, pochi centesimi di franco per migliaia di documenti, in quanto tale è l'energia per documento, e a questa scala il vincolo stringente rimane il **tempo** (alle latenze misurate una scansione di decine di migliaia di file richiede giorni, contro i minuti della via tradizionale). L'argomento di §5.1 moltiplica però il *rapporto* di costo per l'intero patrimonio documentale e per la frequenza di riesecuzione: su un archivio esteso, un fattore compreso fra 6 e 12 incide in misura determinante sulla sostenibilità di riesecuzioni periodiche. Il capitolo dei risultati (§9) riprende questo punto in sede di discussione.

Qualità a fronte del costo Il confronto si articola su due piani: le righe *ai*: quantificano il rilevatore generativo *da solo*, le righe *union*: l'effetto della sua aggiunta alla pipeline tradizionale. Quest'ultima è la domanda operativamente rilevante, poiché nel sistema il rilevatore generativo affianca quelli tradizionali anziché sostituirli.

¹Si tratta della tariffa mediana del servizio universale per un'economia domestica con consumo annuo di 4500 kWh (profilo H4). Per i clienti commerciali ElCom indica per il 2026 una flessione dello stesso ordine (profili C3 e C4, -3% e -4%), senza però un valore mediano di riferimento altrettanto consolidato: si adotta perciò la tariffa domestica, che è il prezzo al dettaglio meglio documentato.

Sul primo piano le due famiglie **divergono**: *gemma3:4b* da sola ottiene F1 0.85, superiore alla baseline (0.79), mentre *phi4-mini* si ferma a 0.66. Da due osservazioni discordi si ricava che l'esito dipende dal modello, non una proprietà generale dell'approccio generativo. Il costo resta invece superiore di un ordine di grandezza per entrambi. Va inoltre osservato il *modo* in cui *gemma3:4b* ottiene il proprio vantaggio: la sua recall è 0.85 contro lo 0.99 della baseline, ossia manca all'incirca una PII su sette a fronte di una su cento, e la F1 le risulta favorevole grazie alla sola precisione (0.84 contro 0.66). Su un compito di conformità le due grandezze non hanno però lo stesso peso: una PII non rilevata resta un rischio non visibile, mentre un falso positivo si traduce in lavoro di revisione (§4.2.2). Il sistema riconosce tale asimmetria adottando un *merge* a impostazione *recall-oriented* (§6.5.3); sulla dimensione qui prioritaria il rilevatore generativo resta perciò indietro.

Sul secondo piano il quadro è concorde: **nessuno dei due modelli migliora la pipeline**. Le fusioni ottengono 0.76 e 0.79 contro lo 0.79 della baseline, che al più eguagliano. I conteggi ne mostrano il meccanismo: *union:gemma3:4b* porta la recall a 1.00 e trova tutte e 315 le occorrenze del campione, mentre la precisione scende a 0.61 perché i falsi positivi salgono da 159 a 199. Il *merge* opera infatti come unione senza arbitraggio (§6.5.3): al guadagno di copertura si somma l'errore di ciascuna sorgente.

Eseguibilità della fascia da 12B Nella configurazione di riferimento — *container* su CPU e macchina *commodity* da 16 GB — il modello da 12B **non è eseguibile in modo affidabile**, ed è un esito non previsto che costituisce esso stesso un risultato. La memoria che il modello occupa una volta caricato e quella dell'*encoder* NER, residente accanto ad esso, eccedono insieme quanto la macchina può garantire con memoria fisica; il sistema operativo termina perciò il processo di inferenza durante la scansione. Il rilevatore degrada in modo controllato: una finestra di testo la cui analisi fallisce viene saltata e la scansione prosegue, comportamento corretto in esercizio. In sede di misura, tuttavia, l'esito si manifesta come risultato *silenziosamente parziale*, ossia metriche dall'aspetto ordinario calcolate su una frazione del corpus.

Sono state provate le tre configurazioni riportate nella Tabella 8.10, nessuna delle quali produce una misura utilizzabile. Il loro andamento esclude che si tratti di una taratura da correggere: ampliare la memoria della macchina virtuale *peggiora* l'esito, perché su un host da 16 GB quella memoria non è coperta da memoria fisica e il sistema operativo pagina su disco le pagine del modello stesso. Al di sotto di una certa soglia il modello non è caricabile, al di sopra la macchina virtuale eccede la capacità dell'host: fra le due condizioni non esiste un intervallo utile.

Tabella 8.10: Tentativi di misura di `gemma3:12b` nella configurazione di riferimento: finestre di testo perse per terminazione del processo di inferenza.

Configurazione	Memoria VM	NER residente	Finestre perse
iniziale	11.7 GB	sì	27%
memoria ampliata	12.7 GB	sì	40%
senza rilevatori tradizionali	10.7 GB	no	30%

Il rilievo ha portata più generale del singolo modello e riguarda direttamente la premessa di questo lavoro: su hardware ordinario il limite all'impiego di un modello generativo di taglia media è la **memoria** disponibile, requisito che una macchina soddisfa o non soddisfa, mentre il tempo di inferenza ammette attesa. Si tratta di una conferma sperimentale del vincolo che il progetto si era dato in sede di scelta del modello (§5.4). Si è deliberatamente rinunciato a misurare la qualità del modello grande in una configurazione diversa, per esempio sfruttando l'accelerazione hardware dell'host, che ne consentirebbe il caricamento: una misura simile descriverebbe un contesto d'uso estraneo al sistema, e il suo accostamento a righe di costo tutte riferite al bersaglio di *deployment* risulterebbe improprio. Per questo lavoro il dato rilevante è l'indisponibilità di quella fascia nell'ambiente considerato.

Effetto della taglia del modello Per la ragione appena documentata la questione resta priva di una misura di qualità: nella configurazione di riferimento la fascia superiore non giunge a un'esecuzione completa e non produce alcun dato. L'affermazione possibile è circoscritta, ma non trascurabile: sull'hardware assunto come bersaglio l'aumento della taglia del modello non costituisce un'opzione praticabile, poiché il presupposto di esecuzione viene meno prima che se ne possa valutare il rendimento. Per un sistema destinato a operare nell'infrastruttura in cui i dati già risiedono, la questione si sposta dunque dalla convenienza di un modello più grande alla sua disponibilità; non essendo soddisfatta la seconda condizione, la prima perde rilevanza pratica.

Osservazione preliminare sui modelli *reasoning* Il confronto principale non include modelli a catena-di-pensiero. L'esclusione è motivata da una verifica preliminare condotta su `qwen3:4b` in fase di messa a punto dell'apparato: il modello risultò al tempo stesso il più costoso fra quelli provati e incapace di produrre alcun rilevamento, poiché con il tetto di 1024 token (§8.3.1) la generazione si interrompe durante la catena di ragionamento, prima che la risposta venga formulata. La combinazione di costo massimo e assenza di risultato indica l'inadeguatezza di un modello di questo tipo a un compito estrattivo con budget di inferenza limitato. L'osservazione poggia però su una misura preliminare, raccolta con un protocollo diverso da quello delle righe della Tabella 8.9, e vale come motivazione di una scelta di perimetro.

8.5.1 Verifica dell'impatto dell'inferenza su sola CPU

Poiché le latenze osservate dipendono dall'inferenza su sola CPU (§8.3.1), se ne è verificato il peso effettivo. Il controllo è stato condotto in fase di messa a punto dell'apparato, su un corpus preliminare e su un solo modello (*phi4-mini*): lo si è eseguito su Ollama in esecuzione *nativa* sull'host, dove la GPU integrata dell'Apple M2 Pro (Metal) è disponibile, confrontandolo con la misura containerizzata su CPU. Le cifre assolute non sono quindi confrontabili con quelle delle tabelle precedenti, mentre il rapporto che ne emerge è informativo. La latenza del solo passo generativo si riduce di circa nove volte, mentre **la qualità non cambia**.

Il risultato è atteso: la GPU esegue il medesimo calcolo del modello a velocità maggiore, e la qualità di una risposta dipende dal modello e dal *prompt* anziché dall'hardware su cui l'inferenza gira. La lieve oscillazione di F1 osservata fra i due *backend* è rumore numerico, dovuto al calcolo in virgola mobile non bit-identico fra CPU e Metal e a versioni diverse del *runtime*.

La verifica ha quindi un duplice esito. Da un lato *delimita* il vincolo: l'esecuzione su CPU penalizza la velocità e lascia intatta l'accuratezza, e non costituisce perciò un limite bloccante sul piano dei risultati. Dall'altro lascia invariata la conclusione del capitolo, poiché anche nel caso migliore l'inferenza generativa resta di gran lunga più lenta del rilevamento tradizionale e l'accelerazione GPU non è comunque disponibile nel *target* di *deployment* containerizzato, dove la GPU non è esposta alla *VM* (§8.3.1).

Conclusione Sul corpus in esame le misure convergono su un esito univoco: il rilevamento generativo, impiegato come secondo parere, non compensa il proprio costo. La pipeline tradizionale rileva già la quasi totalità delle occorrenze presenti, con una recall di 0.99 su documenti *born-digital* e ben formati, e in tale regime l'apporto del modello generativo non giustifica un costo superiore di un ordine di grandezza in tempo ed energia. La portata della conclusione va però delimitata, poiché vale *su questo tipo di dato*. Le condizioni nelle quali il parere generativo potrebbe risultare vantaggioso sono testo rumoroso o proveniente da OCR, formati inusuali, PII non strutturate che né i pattern né il NER prevedono: precisamente quelle che un corpus sintetico e regolare non riproduce. La misura descrive dunque con precisione il regime favorevole, nel quale il modello generativo è ridondante, mentre il regime sfavorevole resta una questione aperta (§10.5). Su questa base si giustifica la scelta architetturale di mantenere l'AI come stadio **opzionale e campionato** anziché sempre attivo (§7.5), ossia come costo da sostenere in modo selettivo. Si ricorda infine che le cifre energetiche sono stime *comparative* e che le configurazioni generative sono misurate su un campione (§8.3): i valori indicano l'ordine di grandezza e la direzione del fenomeno più che una misura definitiva.

Capitolo 9

Risultati e discussione

Questo capitolo raccoglie le considerazioni maturate nei vari capitoli di questo lavoro e rilegge le misure del Capitolo 8 alla luce delle domande poste all'inizio, traendone due conclusioni. La prima riguarda ciò che è stato costruito: un sistema funzionante e misurato, che risponde a un bisogno concreto e potrebbe essere impiegato così com'è. La seconda, che costituisce il baricentro della tesi, riguarda l'AI generativa: un lavoro nato come esplorazione del suo apporto al rilevamento delle PII ne ridimensiona il ruolo sulla base dei dati raccolti durante le sperimentazioni.

9.1 Il sistema realizzato

Al netto della questione generativa, il lavoro ha prodotto un **prototipo funzionante e misurato** che copre l'intera catena dal documento all'esito di conformità. A partire da una cartella di documenti non strutturati, il sistema ne estrae il testo (§7.3), vi rileva le categorie di dati personali fondendo pattern e NER (§7.5), registra le PII trovate come *riferimenti* minimizzati — tipo, posizione, provenienza, mai il valore (§7.6) —, associa i documenti alle attività dichiarate nel registro dei trattamenti (§6.7) e restituisce un esito di conformità leggibile dal DPO: categorie orfane, coperte, mancanti e superamento dei termini di conservazione (§7.8). È esattamente l'insieme degli obiettivi enunciati all'inizio (§1.5), realizzato end-to-end e operabile da un'interfaccia web unica (§6.9).

Sul metro sperimentale, la parte di rilevamento coglie quasi tutto ciò che deve trovare. La pipeline tradizionale raggiunge una **recall di 0.99** sul corpus aziendale, misurata sull'insieme completo dei 300 documenti e delle 2918 occorrenze annotate (Tabella 8.3, Tabella 8.4). Il degrado misurato fra testo pulito e testo estratto dai file reali è nullo (Tabella 8.8), sicché su documenti *born-digital* lo stadio di estrazione si può considerare trasparente. La precisione è invece più bassa (0.66 sul campione su cui è tabulata la pipeline fusa, Tabella 8.6), con una causa circoscritta a una sola categoria: il riconoscimento dei *nomi di persona*, dove l'*encoder* NER etichetta come tali molte espressioni che non lo sono, raccoglie da solo circa

due terzi di tutti i falsi positivi, proporzione che si conferma sia sul campione sia sull'intero corpus. Il difetto è reale ma di natura benigna per l'uso previsto, poiché produce lavoro di revisione per il DPO anziché rischi di conformità non visti, ed è coerente con l'impostazione *recall-oriented* del *merge* (§6.5.3), che preferisce deliberatamente un falso positivo a una PII mancata. Trattandosi di numeri ottenuti su corpus sintetici, essi vanno letti come *limite superiore* (§8.4.4) più che come garanzia sul campo; bastano però a stabilire che l'impianto scelto — Presidio con GLiNER, fusi da un motore di *merge recall-oriented* — è adeguato al compito per cui è stato adottato, e a indicare con precisione dove andrebbe migliorato.

Oltre all'accuratezza, il sistema risponde a un **bisogno normativo concreto e mal coperto dal mercato**. L'obbligo di sapere dove stanno i dati personali, a quale trattamento si riferiscono e se sono ancora entro i termini di conservazione discende direttamente dalla nLPD (art. 6(4)) e dal GDPR (art. 5(1)(e), art. 30), mentre nella pratica gran parte di quei dati vive dispersa in documenti non strutturati, fuori da ogni visibilità sistematica (§1.2). Gli strumenti commerciali esistenti sono tarati su GDPR o CCPA e non trattano nativamente le categorie della nLPD svizzera, e solo i prodotti *enterprise* più avanzati integrano il registro dei trattamenti come riferimento dichiarativo. Il differenziale del sistema qui realizzato consiste proprio in quel collegamento: le PII rilevate sono *confrontate con il dichiarato* e ne deriva un esito di conformità, che è il valore aggiunto per il DPO.

Il sistema è inoltre **impiegabile già oggi**, senza prerequisiti onerosi. Gira su hardware *commodity* in un *container* portatile (§7.10), non richiede né GPU né addestramento su dati dell'organizzazione (§5.2), è coerente con la minimizzazione del dato per requisito — i valori delle PII non vengono mai persistiti (§7.6) — e rileva a un costo contenuto in tempo ed energia (~ 11.8 s e ~ 0.066 Wh a documento, Tabella 8.9), il che lo rende eseguibile con continuità sull'intero patrimonio documentale. Restano naturalmente i limiti dichiarati — assenza di OCR e di analisi di *layout*, un solo profilo utente, corpus di valutazione sintetici (Capitolo 10) — nessuno dei quali ostacola un impiego reale nello scenario per cui il sistema è stato pensato: documenti nativi digitali di un'organizzazione soggetta alla normativa sulla privacy.

9.2 Costo e apporto dell'AI generativa

Il secondo risultato riguarda l'apporto dell'AI generativa, e ne ridimensiona il ruolo rispetto a un'aspettativa largamente condivisa. Il modello generativo è oggi presentato, ben oltre l'ambito tecnico, come soluzione pressoché universale ai problemi di trattamento del testo, da preferire per principio agli approcci tradizionali. Applicata al rilevamento sistematico delle PII, tale aspettativa non trova riscontro nei dati raccolti: sul compito qui studiato l'impiego generativo comporta un costo superiore di un ordine di grandezza, in tempo e in energia, senza un corrispondente guadagno di qualità.

La conclusione arriva da due direzioni convergenti. Da un lato la letteratura la anticipa in termini generali (§5.1): a parità di compito, un modello discriminativo consuma circa 0.002 kWh ogni 1000 inferenze contro le ~ 0.047 kWh di un generativo, e sul piano economico il divario fra un *encoder* e un modello generativo di frontiera arriva a rapporti dell'ordine di 300:1. Dall'altro — ed è il contributo proprio di questo lavoro — la misura diretta sul sistema reale conferma il divario e ne fissa l'entità sul *nostro* hardware e sul *nostro* compito. Il rilevamento tradizionale opera a ~ 11.8 s e ~ 0.066 Wh per documento; ogni passata generativa costa fra 40 e 83 s e fra 0.41 e 0.76 Wh per documento (Tabella 8.9): da ~ 3 a $\sim 7\times$ più lenta, da ~ 6 a $\sim 12\times$ più energivora. Va osservato che il divario, misurato su documenti reali, risulta inferiore a quello che la letteratura riporta su singole inferenze: entrambi i termini crescono con la lunghezza del testo, e su documenti estesi anche il rilevamento tradizionale sostiene il costo dell'*encoder* NER sull'intero contenuto.

A tale costo non corrisponde un guadagno di qualità *per il sistema*. La pipeline tradizionale raggiunge F1 0.79 sul corpus aziendale, e l'aggiunta del parere generativo la lascia invariata: le configurazioni di fusione — le uniche operative, poiché il rilevatore generativo affianca i rilevatori tradizionali anziché sostituirli — si collocano fra 0.76 e 0.79, ossia al più eguagliano la baseline (Tabella 8.6). Il meccanismo è quello atteso: la fusione recupera occorrenze e porta la recall a 1.00, introducendo però falsi positivi che riportano la precisione a 0.61 e ne riassorbono il vantaggio.

Preso isolatamente, il rilevatore generativo restituisce un quadro meno netto, che va riportato con esattezza. Dei due modelli misurati uno ottiene una F1 superiore a quella della baseline, l'altro le resta sensibilmente al di sotto: da due osservazioni discordi si ricava che l'esito **dipende dal modello**, non una proprietà generale dell'approccio generativo, mentre il costo resta superiore di un ordine di grandezza per entrambi. Va inoltre osservato il modo in cui il primo ottiene il proprio vantaggio: la sua recall è 0.85 contro lo 0.99 della baseline, ossia rileva *meno* PII, e la F1 gli risulta favorevole soltanto perché quell'indicatore pesa allo stesso modo precisione e recall. La pesatura non corrisponde alle priorità del dominio: una PII non rilevata è un rischio di conformità che resta invisibile, mentre un falso positivo si traduce in lavoro di revisione per il DPO. Il sistema riconosce tale asimmetria adottando un *merge* a impostazione *recall-oriented* (§6.5.3); sulla dimensione che per questo compito conta di più il rilevatore generativo resta perciò indietro.

Neppure la taglia del modello incide favorevolmente, e in questo caso il vincolo si manifesta prima ancora della qualità: il modello da 12B della stessa famiglia **non è risultato eseguibile** nel bersaglio di *deployment* dichiarato. La memoria che esso occupa e quella dell'*encoder* NER eccedono insieme quanto una macchina *commodity* da 16 GB può garantire, e tre configurazioni successive hanno perso fra il 27% e il 40% delle finestre di testo per interruzione del processo di inferenza (§8.5). La fascia superiore è perciò esclusa dal confronto quantitativo, con una precisazione che ne delimita la portata: non si afferma che un modello da 12B produrrebbe risultati peggiori, bensì che nel perimetro che il progetto si è dato —

esecuzione locale su hardware ordinario, imposta dalla minimizzazione del dato e non da una preferenza — quel modello non è disponibile. Analoga esclusione vale per il modello *reasoning*, che presenta la combinazione meno favorevole fra costo e risultato (§8.5).

È stata infine verificata l'ipotesi che a penalizzare l'AI sia la sola esecuzione su CPU, e il controllo l'ha esclusa: su GPU la medesima inferenza è $\sim 9\times$ più veloce e di qualità identica (§8.5.1), poiché l'accelerazione riguarda la velocità del calcolo e lascia intatta la sua accuratezza. Quell'accelerazione non è d'altronde disponibile nel bersaglio di *deployment* containerizzato, dove la GPU non è esposta alla macchina virtuale.

La **portata** di questa conclusione va delimitata con precisione. Non si tratta di un rifiuto dell'AI generativa in quanto tale, bensì della constatazione che, *su questo compito e su questo tipo di dato*, il suo costo non risulta compensato. Il compito è il rilevamento *sistematico e ricorrente* — ogni documento, ripetutamente — ed è la ricorrenza a moltiplicare il costo unitario fino a renderlo decisivo (§5.1). La grandezza determinante è il **tempo** più che la spesa in valore assoluto, che resta di pochi centesimi: a decine di secondi per documento, una scansione che per via tradizionale richiede minuti si estende a giorni. Il dato in esame è inoltre regolare e nativo digitale, ossia il regime più favorevole ai rilevatori tradizionali, nel quale pattern e NER sono già prossimi al massimo delle loro prestazioni e lasciano all'AI un margine di scoperta minimo. Le condizioni nelle quali il parere generativo potrebbe invece risultare vantaggioso — testo rumoroso o proveniente da OCR, formati inusuali, PII non strutturate che né i pattern né il NER prevedono — sono esattamente quelle che i corpus impiegati non riproducono, e restano una questione aperta (§10.5). La scelta architettuale adottata è coerente con questa lettura: l'AI è mantenuta e collocata dove il suo costo non si moltiplica per il corpus — uno slot **opzionale e campionato** nel rilevamento (§7.5) e i pochi compiti *selettivi* a bassa frequenza (§5.4) — riservandola ai casi in cui le interrogazioni sono poche e la comprensione semantica apporta un valore che gli altri approcci non possono dare.

La lezione metodologica va oltre il singolo sistema. Di fronte a un problema di trattamento del testo la domanda pertinente non è se l'AI generativa lo risolva, poiché quasi sempre la risposta è affermativa, ma se lo risolva *meglio* degli approcci più semplici, in misura sufficiente a giustificare il costo alla scala e alla frequenza d'uso previste. Per il rilevamento sistematico delle PII, misure alla mano, la risposta è negativa: la via tradizionale è sufficientemente accurata, più veloce di un ordine di grandezza e sostenibile con continuità.

9.3 Considerazioni conclusive

I due risultati si intersecano. Il sistema realizzato dimostra che la conformità alla nLPD sul contenuto documentale reale è affrontabile immediatamente, con mezzi leggeri e riproducibili; lo studio sull'AI generativa spiega *perché* quei mezzi sono leggeri: la leggerezza è un esito misurato e non una rinuncia, poiché la via più costosa non avrebbe reso di più. Il contributo

di questo lavoro consiste dunque in un prototipo funzionante e, insieme, in una risposta quantitativa e documentata a una domanda che spesso si dà per scontata: sul rilevamento sistematico dei dati personali l'approccio tradizionale è la scelta migliore.

Entrambi i risultati poggiano su un artefatto che il lettore può esaminare: il codice del sistema, i generatori dei corpus e la procedura che produce le misure discusse in questi due capitoli sono pubblicati in un repository ad accesso aperto [94], alla versione consegnata insieme a questa tesi. La scelta è coerente con il criterio che il Capitolo 3 ha usato per giudicare gli strumenti esistenti (§3.6.1): un lavoro che rimprovera alle suite commerciali di non essere ispezionabili deve, per primo, esporre il proprio funzionamento alla verifica.

Capitolo 10

Sviluppi futuri

Alcune funzionalità concepite nel corso del lavoro non sono state realizzate. Non si tratta di omissioni: ciascuna è stata valutata e accantonata per ragioni che questo capitolo espone, così da rendere esplicito il perimetro di ciò che è stato costruito e distinguere le funzionalità deliberatamente escluse da quelle semplicemente non affrontate. Le prime quattro sezioni ne trattano una ciascuna — che cosa avrebbe portato, che cosa se ne è valutato, perché non è stata realizzata e a quali condizioni potrebbe essere ripresa; l'ultima raccoglie i limiti già riconosciuti altrove nel documento, che restano aperti come naturale prosecuzione del lavoro svolto.

10.1 Analisi contestuale della struttura del file system

L'idea, formulata fra i requisiti funzionali (cfr. 2.1.10) e prevista in architettura come blocco autonomo (§6.3), era di trattare l'organizzazione delle cartelle come un segnale: i nomi dei rami, la profondità e il raggruppamento dei documenti dicono qualcosa sulla finalità per cui quei documenti sono tenuti insieme. Sottoposta a un modello linguistico, la resa testuale dell'albero avrebbe proposto l'associazione fra rami e attività di trattamento e segnalato le anomalie posizionali, ossia i documenti il cui contenuto non è coerente con la cartella che li ospita.

Interpretare una struttura di cartelle è però un'euristica, e valutarne l'affidabilità richiederebbe un corpus di alberi documentali annotati con l'attribuzione corretta di ciascun ramo: non esiste, andrebbe costruito sinteticamente e raddoppierebbe l'apparato di valutazione (cfr. 4.1.1). Un'attribuzione inferita dalla macchina, inoltre, diventerebbe il presupposto dell'esito di conformità, che verrebbe così a poggiare su una supposizione anziché su una dichiarazione. La parte utile della funzionalità è stata infine ottenuta per altra via: le regole che associano un prefisso di percorso a una o più attività (cfr. 7.7) associano un intero ramo in un solo gesto, inclusi i documenti che vi compariranno in seguito, con la differenza che a stabilire il significato della collocazione è il DPO. Resterebbe da realizzare la sola *proposta* di quelle

10.2 Canale di segnalazione dai dipendenti

```
graph TD; DA[Documenti aziendali] -- "consultati nel lavoro quotidiano" --> DP[Dipendenti dell'azienda]; DA -- "analizza" --> SRP[Sistema di rilevamento PII]; DP -- "segnalano le PII in cui si imbattono" --> DPO[DPO <br/> (analisi segnalazioni)]; DPO -- "valuta e arricchisce" --> SRP; DPO -- "valuta e arricchisce" --> PS[Parametri del sistema <br/> (catalogo PII, regole)]; PS --> SRP; SRP <--> ROPA[(ROPA <br/> (trattamenti, categorie))];
```

Il punto che il canale tocca è il più debole del sistema. I falsi negativi sono per definizione invisibili (cfr. 4.2.3): nessuna misura interna può rivelare un dato personale che i detector non vedono, e chi lavora sui documenti nel loro contesto d'uso è l'unico in grado di segnalare queste mancanze.

Realizzarlo significherebbe però passare da un unico profilo di utilizzatore — il DPO, premessa su cui poggia l'intera restituzione — a un insieme di identità da censire e mantenere allineato all'organico, con ruoli, permessi differenziati, un'interfaccia distinta per chi non deve vedere la console e la moderazione delle segnalazioni: ciò che è oneroso non è *autenticare*

(una schermata di accesso è cosa dovuta e figura fra gli interventi residui, §10.5) ma *autorizzare*. Un secondo argomento sconsiglia la forma prevista: il registro non contiene i valori dei dati personali ma ne costituisce la mappa (cfr. 4.2.9), e affacciarvi tutto il personale ne amplierebbe la superficie di accesso dal solo DPO all'intera organizzazione, in uno strumento che esiste per ridurre l'esposizione. Se ripreso, il canale andrebbe perciò realizzato in sola immissione, separato dalla console del DPO e privo di qualsiasi accesso in lettura al registro.

10.3 Estensione oltre i documenti nativi digitali

Il requisito di estrazione (cfr. 2.1.2) è soddisfatto sui formati *born-digital* dell'ufficio — documenti impaginati, fogli di calcolo e testo semplice. Restano fuori i documenti che sono immagini, ossia le scansioni, e con essi il riconoscimento ottico dei caratteri, i layout complessi a più colonne e le tabelle, oltre ai formati meno diffusi come *.xls* e *.rtf*.

È la limitazione che più circoscrive la portata dei risultati: le prestazioni di rilevamento riportate, e con esse il degrado da estrazione nullo (Capitolo 8), valgono per documenti nativi digitali e non si estendono a un archivio di scansioni. Il riconoscimento ottico non sarebbe infatti un lettore in più, ma una sorgente di errore collocata prima del rilevamento: il testo che ne esce contiene errori di trascrizione, e ogni PII non rilevata diverrebbe attribuibile indifferentemente all'estrattore o al rilevatore. Estendere l'insieme dei formati ha valore e costo lineari: ogni formato aggiunto rende leggibile un'altra sezione dell'insieme documentale, senza cambiare ciò che il sistema analizza e produce.

10.4 Tracciamento temporale delle PII rilevate

Le linee guida di progetto prospettavano che il registro sapesse riconoscere, fra due scansioni successive dello stesso documento, che una determinata PII è stata cancellata o modificata: non solo le PII presenti in un certo istante, ma anche le variazioni dei dati sensibili presenti nel tempo. Il registro realizzato si limita allo stato corrente (§6.6): ogni scansione sostituisce le PII registrate di un documento con quelle che vi trova al momento. Il disegno del tracciamento temporale è stato svolto ma non realizzato.

Tre approcci sono stati confrontati. Salvare a ogni esecuzione un'istantanea completa delle PII rilevate è il più semplice, ma non lega fra loro le scansioni consecutive: ricostruire che cosa sia cambiato resterebbe un confronto da calcolare. L'*event sourcing* puro registra invece i soli eventi e ricava lo stato ripercorrendo la timeline, con una storia completa e immutabile per costruzione; lo studio empirico di Overeem et al. su venticinque adozioni reali ne documenta però le difficoltà ricorrenti [95], e poiché lo stato corrente non è memorizzato anche l'interrogazione più semplice richiederebbe di ripercorrere gli eventi, o di mantenere in parallelo un secondo modello di lettura [96]. Il terzo approccio, che si sarebbe adottato, tiene insieme i due: lo stato corrente resta salvato, interrogabile direttamente e senza ricostruzioni,

e gli si affianca un log *append-only* secondario che registra ogni variazione legandola alla scansione precedente dello stesso documento. In concreto occorrerebbe aggiungere allo schema una sola entità, che per ciascuna istanza registri il tipo di variazione (NEW, CONFIRMED, MOVED, REMOVED) e la scansione che l'ha prodotta; restando riferimenti e non valori, il log resterebbe coerente con la minimizzazione del dato (§2.3.5).

10.5 Limiti dichiarati e direzioni residue

Oltre alle funzionalità sopra discusse, restano aperti alcuni limiti già riconosciuti nel corso del documento, che costituiscono la naturale prosecuzione del lavoro:

- **Autenticazione della console.** L'interfaccia del DPO è oggi raggiungibile da chiunque abbia accesso di rete alla porta su cui è servita. Anche per un applicativo a utilizzatore singolo, e per quanto collocato in una rete interna, l'accesso va protetto da una schermata di autenticazione (cfr. 4.2.9). L'intervento è circoscritto e va tenuto distinto dalla gestione di più profili discussa in 10.2: qui si tratta di verificare *un'unica* identità, non di amministrarne molte.
- **Parallelizzazione dell'analisi.** L'elaborazione incrementale riduce il costo delle esecuzioni successive alla prima, ma non quello della passata iniziale, che resta sequenziale (cfr. 4.2.6).
- **Validazione su corpus reale.** Le misure disponibili provengono da corpus sintetici, più ordinati del reale, e vanno intese come limite superiore; la verifica sul patrimonio documentale effettivo di un'organizzazione resta un passaggio non eludibile prima dell'adozione (cfr. 4.1.1, 4.2.4). Il limite è più stringente di quanto il solo ordine dei documenti suggerisca: un corpus generato a partire dal catalogo delle categorie contiene per costruzione i soli tipi di dato che il catalogo prevede, sicché i dati personali che né i pattern né il riconoscimento di entità sono attrezzati a riconoscere non vi compaiono affatto, e restano non misurati.
- **Fascia superiore dei modelli generativi.** Il modello da 12B non è risultato eseguibile nel bersaglio di *deployment* dichiarato (§8.5); se una taglia maggiore modifichi il bilancio fra costo e qualità resta perciò una domanda aperta, da affrontare fuori da quel vincolo di memoria e dichiarando i costi non confrontabili.
- **Datazione dal contenuto.** La data di riferimento di un documento è oggi stimata dai metadati e dal file system; ricavarla dal corpo del testo — dove spesso compare in chiaro — renderebbe la verifica della conservazione assai più solida, ma richiede un riconoscimento delle date e una disambiguazione fra le molte presenti in un documento.

Elenco delle figure

6.1	Architettura a macro-blocchi del sistema	64
6.2	Dettaglio del blocco B4: pipeline ibrida di rilevamento PII	65
7.1	Blocco B1: catena di ingestione del ROPA e mappatura assistita	77
7.2	Blocco B4: i tre rilevatori dietro il contratto PIIDetector	82
7.3	Blocco B6: associazione documento–trattamento e legame fra i due registri	89
7.4	Blocco B7: il confronto fra dichiarato e rilevato	91
7.5	Deployment containerizzato: servizi, volumi e configurazione	95
10.1	Meccanismo di <i>human-in-the-loop</i> : nel normale lavoro con i documenti aziendali i dipendenti segnalano al DPO le PII in cui si imbattono; il DPO analizza le segnalazioni e arricchisce il ROPA o i parametri del sistema, migliorando il rilevamento. Il processo di analisi resta trasparente ai dipendenti, per i quali i file sono semplici documenti.	122

Elenco delle tabelle

3.1	Prestazioni di rilevamento degli approcci di PII detection secondo le evidenze di letteratura. Precisione e recall sono espressi sulla scala ordinale riportata nella legenda; i valori misurati da cui ciascun livello discende sono richiamati nelle note.	30
3.2	Copertura funzionale e condizioni di adozione dei sistemi esaminati. Le righe sono formulate come feature richieste nel contesto d'impiego considerato (Capitolo 2); le caselle riprendono quanto documentato nel §3.5. L'ultima colonna non riporta misure, ma le proprietà <i>richieste</i> al sistema da realizzare.	34
4.1	Valutazione dei rischi identificati: probabilità, impatto e priorità residua.	47
5.1	Costo energetico ed economico degli approcci di PII detection, secondo le evidenze di letteratura.	53
5.2	Criteri di selezione dello SLM e relative fonti.	56
5.3	Throughput (token/s), 4-bit GGUF su CPU [13].	57
5.4	Occupazione di memoria: FP16 vs 4-bit (Q4) e RAM minima [13].	57
5.5	Raccomandazione per task (fascia CPU).	60
8.1	Distribuzione dei test di unità per area del sistema (388 casi, tutti superati). . .	98
8.2	Ambiente di prova: hardware, risorse del <i>container</i> e parametri di inferenza. .	102
8.3	Rilevatore a pattern (riconoscitori di Presidio più le regole di <code>custom_patterns.yaml</code>), per categoria, sull'intero corpus: 300 documenti, 2918 occorrenze annotate. .	104
8.4	Rilevatore NER (GLiNER, <code>gliner_multi_pii</code>), per categoria, sull'intero corpus: 300 documenti, 2918 occorrenze annotate.	105
8.5	Rilevatore generativo (<code>gemma3:4b</code>), per categoria, sul campione (§8.3).	105
8.6	Rilevamento tradizionale contro agentico: metriche complessive sul campione (P, R, F1).	106
8.7	Sistema completo (pattern + NER + AI <code>gemma3:4b</code> , fusi), per categoria, sul campione.	107
8.8	Validazione end-to-end sull'intero corpus (300 documenti, 2918 occorrenze): Tier 1 (testo pulito) contro Tier 2 (testo estratto dai file). Confronto per valore.	108

8.9	Studio multi-modello: qualità (P/R/F1) contro costo (s/doc, Wh/doc, CHF/1000 doc). Due famiglie indipendenti della fascia $\sim 4B$; la fascia superiore è esclusa perché non eseguibile nella configurazione di riferimento (Tabella 8.10). Baseline = sistema senza AI.	109
8.10	Tentativi di misura di <code>gemma3</code> : 12b nella configurazione di riferimento: finestre di testo perse per terminazione del processo di inferenza.	112

Bibliografia

- [1] Amir Gandomi e Murtaza Haider. «Beyond the Hype: Big Data Concepts, Methods, and Analytics». In: *International Journal of Information Management* 35.2 (apr. 2015), pp. 137–144. ISSN: 02684012.
- [2] *Privacy Suite*. BigID. URL: <https://bigid.com/privacy-suite/>.
- [3] *Overview of Varonis AI-Powered Data Discovery and Classification*. URL: <https://www.varonis.com/blog/ai-data-classification>.
- [4] *PII Tools | Discover, Analyze & Remediate Sensitive Data Anywhere*. PII Tools. URL: <https://pii-tools.com/>.
- [5] Shirley Radack. «Protecting Personally Identifiable Information». In: *Guide to Protecting the Confidentiality of Personally Identifiable Information (PII)*. NIST Special Publication (apr. 2010).
- [6] Parlamento europeo e Consiglio dell'Unione europea. *Regolamento (UE) 2016/679*. 27 Apr. 2016.
- [7] *RS 235.1 - Legge Federale Del 25 Settembre 2020 Sulla Protezione Dei Dati (LPD) | Fedlex*. URL: <https://www.fedlex.admin.ch/eli/cc/2022/491/it>.
- [8] Pierre Lison et al. «Anonymisation Models for Text Data: State of the Art, Challenges and Future Directions». In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers). Online: Association for Computational Linguistics, 2021, pp. 4188–4203.
- [9] Ildikó Pilán et al. «The Text Anonymization Benchmark (TAB): A Dedicated Corpus and Evaluation Framework for Text Anonymization». In: *Computational Linguistics* 48.4 (1 dic. 2022), pp. 1053–1101. ISSN: 0891-2017, 1530-9312.
- [10] Rene Abraham, Johannes Schneider e Jan Vom Brocke. «Data Governance: A Conceptual Framework, Structured Review, and Research Agenda». In: *International Journal of Information Management* 49 (dic. 2019), pp. 424–438. ISSN: 02684012.

- [11] Orlando Amaral Cejas et al. «NLP-Based Automated Compliance Checking of Data Processing Agreements Against GDPR». In: *IEEE Transactions on Software Engineering* 49.9 (set. 2023), pp. 4282–4303. ISSN: 0098-5589, 1939-3520, 2326-3881.
- [12] Zhenyan Lu et al. *Small Language Models: Survey, Measurements, and Insights*. 26 Feb. 2025. arXiv: 2409.15790 [cs.CL]. URL: <http://arxiv.org/abs/2409.15790>. Pre-pubblicato.
- [13] Kyrylo S Malakhov. «Deploying LLMs on CPU-only Environments with Llama.Cpp Library Set: MedLocalGPT Project Case». In: ().
- [14] *ISO/IEC/IEEE 29148:2018*. ISO. URL: <https://www.iso.org/standard/72089.html>.
- [15] *IEEE-29148-SRS-LaTeX-Template/IEEE-29148-2018-SRS-Template.Tex at Main · Wxix/IEEE-29148-SRS-LaTeX-Template*. GitHub. URL: <https://github.com/wxix/IEEE-29148-SRS-LaTeX-Template/blob/main/IEEE-29148-2018-SRS-Template.tex>.
- [16] A. Makhambet e A. Moldagulova. «A Comparative Analysis of Machine Learning Methods for Personal Information Recognition (PII) in Unstructured Texts». In: *Computing & Engineering* 3.1 (31 mar. 2025), pp. 41–52. ISSN: 3080-3829.
- [17] Harshit Rajgarhia et al. *An Evaluation Study of Hybrid Methods for Multilingual PII Detection*. arXiv.org. 8 Ott. 2025. URL: <https://arxiv.org/abs/2510.07551v1>.
- [18] Nagaraju Velishetty. «Personal Identifiable Information (PII) Detection and Identification for Fintech with AI and Text Analytics». Tesi di laurea mag. Dublin, National College of Ireland, 10 ago. 2024. 23 pp.
- [19] *Le registre RGPD de la CNIL*. URL: <https://www.cnil.fr/fr/le-registre-rgpd-de-la-cnil>.
- [20] Jianliang Yang et al. «Exploring the Application of Large Language Models in Detecting and Protecting Personally Identifiable Information in Archival Data: A Comprehensive Study*». In: *2023 IEEE International Conference on Big Data (BigData)*. 2023 IEEE International Conference on Big Data (BigData). Sorrento, Italy: IEEE, 15 dic. 2023, pp. 2116–2123. ISBN: 979-8-3503-2445-7.
- [21] Sinan Gultekin et al. «An Energy-Based Comparative Analysis of Common Approaches to Text Classification in the Legal Domain». In: *AI, Machine Learning and Applications*. 27 Gen. 2024, pp. 31–41. arXiv: 2311.01256 [cs.CL].
- [22] Johannes Zschache e Tilman Hartwig. «Comparing Energy Consumption and Accuracy in Text Classification Inference». In: *Scientific Reports* 16.1 (17 apr. 2026), p. 12717. ISSN: 2045-2322. arXiv: 2508.14170 [cs.CL].
- [23] Ian Goodfellow, Yoshua Bengio e Aaron Courville. *Deep Learning*. MIT Press, 2016.

- [24] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Information Science and Statistics. New York: Springer, 2006. ISBN: 978-0-387-31073-2.
- [25] Trevor Hastie, Robert Tibshirani e Jerome H. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Second edition. Springer Series in Statistics. New York, NY: Springer, 2017. 1 p. ISBN: 978-0-387-84857-0 978-0-387-84858-7.
- [26] M. I. Jordan e T. M. Mitchell. «Machine Learning: Trends, Perspectives, and Prospects». In: *Science* 349.6245 (17 lug. 2015), pp. 255–260. ISSN: 0036-8075, 1095-9203.
- [27] Imed Keraghel, Stanislas Morbieu e Mohamed Nadif. *Recent Advances in Named Entity Recognition: A Comprehensive Survey and Comparative Study*. 20 Dic. 2024. arXiv: 2401.10825 [cs.CL]. URL: <http://arxiv.org/abs/2401.10825>. Pre-pubblicato.
- [28] Byamugisha Africano et al. «PII Detection in Low-Resource Languages Using Explainable Deep Learning Techniques». In: *Proceedings of the 2024 Sixteenth International Conference on Contemporary Computing*. IC3 2024: 2024 Sixteenth International Conference on Contemporary Computing. Noida India: ACM, 8 ago. 2024, pp. 94–103. ISBN: 979-8-4007-0972-2.
- [29] Jacob Devlin et al. «BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding». In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. NAACL-HLT 2019. A cura di Jill Burstein, Christy Doran e Tamar Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, giu. 2019, pp. 4171–4186.
- [30] James Kirkpatrick et al. «Overcoming Catastrophic Forgetting in Neural Networks». In: *Proceedings of the National Academy of Sciences* 114.13 (28 mar. 2017), pp. 3521–3526. ISSN: 0027-8424, 1091-6490.
- [31] Yongqin Xian et al. «Zero-Shot Learning—A Comprehensive Evaluation of the Good, the Bad and the Ugly». In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 41.9 (1 set. 2019), pp. 2251–2265. ISSN: 0162-8828, 2160-9292, 1939-3539.
- [32] Tom Brown et al. «Language Models Are Few-Shot Learners». In: *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901.
- [33] Urchade Zaratiana et al. «GLiNER: Generalist Model for Named Entity Recognition Using Bidirectional Transformer». In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human

- Language Technologies (Volume 1: Long Papers). Mexico City, Mexico: Association for Computational Linguistics, 2024, pp. 5364–5376.
- [34] Yuzhao Heng et al. «ProgGen: Generating Named Entity Recognition Datasets Step-by-step with Self-Reflexive Large Language Models». In: *Findings of the Association for Computational Linguistics ACL 2024*. Findings of the Association for Computational Linguistics ACL 2024. Bangkok, Thailand e virtual meeting: Association for Computational Linguistics, 2024, pp. 15992–16030.
- [35] Pritesh Jha. *PIIBench: A Unified Multi-Source Benchmark Corpus for Personally Identifiable Information Detection*. arXiv.org. 17 Apr. 2026. URL: <https://arxiv.org/abs/2604.15776v1>.
- [36] Yanai Elazar et al. «What's In My Big Data?» In: The Twelfth International Conference on Learning Representations. 13 Ott. 2023.
- [37] Lokesh Koli, Shubham Kalra e Karanpreet Singh. *Decoding Complexity: Intelligent Pattern Exploration with CHPDA (Context Aware Hybrid Pattern Detection Algorithm)*. 9 Feb. 2025. arXiv: 2502.07815 [cs.CR]. URL: <http://arxiv.org/abs/2502.07815>. Pre-pubblicato.
- [38] Luca Mainetti e Andrea Elia. «Detecting Personally Identifiable Information Through Natural Language Processing: A Step Forward». In: *ResearchGate* (20 mar. 2026).
- [39] Alexandra Sasha Luccioni, Yacine Jernite e Emma Strubell. «Power Hungry Processing: Watts Driving the Cost of AI Deployment?» In: *The 2024 ACM Conference on Fairness Accountability and Transparency*. 3 Giu. 2024, pp. 85–99. arXiv: 2311.16863 [cs.LG].
- [40] Andres Felipe Zambrano et al. «De-Identifying Student Personally Identifying Information in Discussion Forum Posts with Large Language Models». In: *Information and Learning Sciences* 126.5–6 (7 apr. 2025), pp. 401–424. ISSN: 2398-5348.
- [41] Hao Yu et al. *Open, Closed, or Small Language Models for Text Classification?* 19 Ago. 2023. arXiv: 2308.10092 [cs.CL]. URL: <http://arxiv.org/abs/2308.10092>. Pre-pubblicato.
- [42] Erik Johannes Husom et al. *The Price of Prompting: Profiling Energy Use in Large Language Models Inference*. 3 Mar. 2026. arXiv: 2407.16893 [cs.CY]. URL: <http://arxiv.org/abs/2407.16893>. Pre-pubblicato.
- [43] Agatsya Yadav e Renta Chintala Bhargavi. «Optimizing LLMs Using Quantization for Mobile Execution». In: vol. 1654. 2026, pp. 330–339. arXiv: 2512.06490 [cs.LG].
- [44] «Enterprise-Scale PII De-Identification with Microsoft Presidio Anonymizer: Architecture, Use Cases, and Best Practices». In: *ResearchGate* (3 mar. 2026). ISSN: 3050-9416.

- [45] Minqi Jiang et al. *Generative Data Refinement: Just Ask for Better Data*. 11 Set. 2025. arXiv: 2509.08653 [cs.LG]. URL: <http://arxiv.org/abs/2509.08653>. Pre-pubblicato.
- [46] Noopur Zambare et al. «Towards Fair and Efficient De-identification: Quantifying the Efficiency and Generalizability of De-identification Approaches». In: *Findings of the Association for Computational Linguistics: EACL 2026*. Findings of the Association for Computational Linguistics: EACL 2026. Rabat, Morocco: Association for Computational Linguistics, 2026, pp. 4242–4257.
- [47] Ben. *Best Open Source Models for PII Redaction*. Grepture. 25 Mar. 2026. URL: <https://grepture.com/blog/best-open-source-models-pii-redaction>.
- [48] Kushagra Mishra, Harsh Pagare e Kanhaiya Sharma. «A Hybrid Rule-Based NLP and Machine Learning Approach for PII Detection and Anonymization in Financial Documents». In: *Scientific Reports* 15.1 (2 lug. 2025), p. 22729. ISSN: 2045-2322.
- [49] *Comparing Best NER Models For PII Identification*. 18 Nov. 2025. URL: <https://www.protecto.ai/blog/best-ner-models-for-pii-identification/>.
- [50] *Knowledgator/Gliner-Pii-Base-v1.0 · Hugging Face*. 29 Gen. 2026. URL: <https://huggingface.co/knowledgator/gliner-pii-base-v1.0>.
- [51] *Open Source PHI De-Identification: A Technical Review*. IntuitionLabs. URL: <https://intuitionlabs.ai/articles/open-source-phi-de-identification-tools>.
- [52] Hao Shen et al. «PII-Bench: Evaluating Query-Aware Privacy Protection Systems». In: *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). San Diego, California, United States: Association for Computational Linguistics, 2026, pp. 4991–5026.
- [53] *Microsoft Presidio*. URL: <https://microsoft.github.io/presidio/>.
- [54] Urchade Zaratiana, Ash Lewis e George Hurn-Maloney. *GLiNER2-PII: A Multilingual Model for Personally Identifiable Information Extraction*. arXiv.org. 11 Mag. 2026. URL: <https://arxiv.org/abs/2605.09973v1>.
- [55] *Scrubadub*. 22 Giu. 2026. URL: <https://github.com/LeapBeyond/scrubadub>.
- [56] *Data Classification Buyer's Guide: How to Choose a Data Classification Solution*. URL: <https://www.varonis.com/blog/data-classification-buyers-guide>.
- [57] *Varonis Software Pricing & Plans 2025: See Your Cost*. URL: <https://www.vendr.com/marketplace/varonis>.
- [58] *BigID Data Sheet: GDPR Compliance*. Slideshare. URL: <https://www.slideshare.net/slideshow/bigid-data-sheet-gdpr-compliance-120471407/120471407>.

- [59] *Detecting Personal Names in Texts: Technical Whitepaper*. Technical whitepaper. Limassol, Cyprus: PII-Tools Ltd., 2024.
- [60] *BigID Introduces HyperscanTM for Speeding Unstructured File Scans at Scale*. URL: <https://www.businesswire.com/news/home/20200820005174/en/BigID-Introduces-Hyperscan-for-Speeding-Unstructured-File-Scans-at-Scale>.
- [61] *BigID: Details, Reviews, Pricing, & Features*. CheckThat.ai. URL: <https://checkthat.ai/brands/bigid>.
- [62] Anthony Yazdani, Ihor Stepanov e Douglas Teodoro. «GLiNER-BioMed: A Suite of Efficient Models for Open Biomedical Named Entity Recognition». In: *Bioinformatics* 42.6 (1 giu. 2026). A cura di Björn Grüning, bttag322. ISSN: 1367-4803, 1367-4811.
- [63] Saadullah Amin et al. «Few-Shot Cross-lingual Transfer for Coarse-grained De-identification of Code-Mixed Clinical Texts». In: *Proceedings of the 21st Workshop on Biomedical Language Processing*. Proceedings of the 21st Workshop on Biomedical Language Processing. Dublin, Ireland: Association for Computational Linguistics, 2022, pp. 200–211.
- [64] Joint Task Force Transformation Initiative. *Guide for Conducting Risk Assessments*. NIST SP 800-30r1. Gaithersburg, MD: National Institute of Standards and Technology, 2012, NIST SP 800–30r1. URL: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-30r1.pdf>.
- [65] *Facts & Figures · spaCy Usage Documentation*. Facts & Figures. URL: <https://spacy.io/usage/facts-figures/>.
- [66] *GPU Acceleration - Presidio*. URL: https://presidio.dataprivacystack.org/analyzer/nlp_engines/gpu_usage/.
- [67] Qingyu Song et al. *A Systematic Evaluation of On-Device LLMs: Quantization, Performance, and Resources*. 16 Mar. 2026. arXiv: 2505.15030 [cs.LG]. URL: <http://arxiv.org/abs/2505.15030>. Pre-pubblicato.
- [68] Uygar Kurt. *Which Quantization Should I Use? A Unified Evaluation of Llama.Cpp Quantization on Llama-3.1-8B-Instruct*. 11 Gen. 2026. arXiv: 2601.14277 [cs.LG]. URL: <http://arxiv.org/abs/2601.14277>. Pre-pubblicato.
- [69] Sönke Tenckhoff, Mario Koddenbrock e Erik Rodner. *LLMStructBench: Benchmarking Large Language Model Structured Data Extraction*. 16 Feb. 2026. arXiv: 2602.14743 [cs.CL]. URL: <http://arxiv.org/abs/2602.14743>. Pre-pubblicato.
- [70] Nikolai Skripko. *Instruction-Following Evaluation in Function Calling for Large Language Models*. 22 Set. 2025. arXiv: 2509.18420 [cs.AI]. URL: <http://arxiv.org/abs/2509.18420>. Pre-pubblicato.

- [71] *Ibm-Granite/Granite-3.0-Language-Models*. IBM Granite, 17 lug. 2026. URL: <https://github.com/ibm-granite/granite-3.0-language-models>.
- [72] Gemma Team et al. *Gemma 3 Technical Report*. 25 Mar. 2025. arXiv: 2503.19786 [cs.CL]. URL: <http://arxiv.org/abs/2503.19786>. Pre-pubblicato.
- [73] *Meta-Llama/Llama-3.2-1B-Instruct* · Hugging Face. 6 Dic. 2024. URL: <https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct>.
- [74] Josip Tomo Licardo et al. «Performance Trade-Offs of Optimizing Small Language Models for E-Commerce». In: *Big Data and Cognitive Computing* 10.5 (14 mag. 2026), p. 155. ISSN: 2504-2289.
- [75] Haolin Zhang e Jeff Huang. *Challenging GPU Dominance: When CPUs Outperform for On-Device LLM Inference*. 9 Mag. 2025. arXiv: 2505.06461 [cs.DC]. URL: <http://arxiv.org/abs/2505.06461>. Pre-pubblicato.
- [76] Dan Hendrycks et al. «Measuring Massive Multitask Language Understanding». In: arXiv, 2020.
- [77] Yubo Wang et al. «MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark». In: arXiv, 2024.
- [78] Karl Cobbe et al. *Training Verifiers to Solve Math Word Problems*. Ver. 2. 2021. URL: <https://arxiv.org/abs/2110.14168>. Pre-pubblicato.
- [79] Mark Chen et al. «Evaluating Large Language Models Trained on Code». In: arXiv, 2021.
- [80] Jeffrey Zhou et al. «Instruction-Following Evaluation for Large Language Models». In: arXiv, 2023.
- [81] Shishir G. Patil et al. «The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models». In: *Proceedings of the 42nd International Conference on Machine Learning*. International Conference on Machine Learning. PMLR, 6 ott. 2025, pp. 48371–48392.
- [82] Lianmin Zheng et al. «Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena». In: arXiv, 2023.
- [83] Qwen et al. *Qwen2.5 Technical Report*. 3 Gen. 2025. arXiv: 2412.15115 [cs.CL]. URL: <http://arxiv.org/abs/2412.15115>. Pre-pubblicato.
- [84] Marah Abdin et al. *Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone*. 30 Ago. 2024. arXiv: 2404.14219 [cs.CL]. URL: <http://arxiv.org/abs/2404.14219>. Pre-pubblicato.
- [85] Microsoft et al. *Phi-4-Mini Technical Report: Compact yet Powerful Multimodal Language Models via Mixture-of-LoRAs*. 7 Mar. 2025. arXiv: 2503.01743 [cs.CL]. URL: <http://arxiv.org/abs/2503.01743>. Pre-pubblicato.

- [86] Loubna Ben Allal et al. «SmolLM2: When Smol Goes Big — Data-Centric Training of a Fully Open Small Language Model». In: Second Conference on Language Modeling. 26 Ago. 2025.
- [87] Cosimo Galeone et al. *When Correct Isn't Usable: Improving Structured Output Reliability in Small Language Models*. 4 Mag. 2026. arXiv: 2605.02363 [cs.CL]. URL: <http://arxiv.org/abs/2605.02363>. Pre-pubblicato.
- [88] Md Romyull Islam et al. *Characterizing and Understanding Energy Footprint and Efficiency of Small Language Model on Edges*. 7 Nov. 2025. arXiv: 2511.11624 [cs.DC]. URL: <http://arxiv.org/abs/2511.11624>. Pre-pubblicato.
- [89] *Le registre des activités de traitement*. URL: <https://www.cnil.fr/fr/RGPD-le-registre-des-activites-de-traitement>.
- [90] Nikolaos Livathinos et al. *Docling: An Efficient Open-Source Toolkit for AI-driven Document Conversion*. 27 Gen. 2025. arXiv: 2501.17887 [cs.CL]. URL: <http://arxiv.org/abs/2501.17887>. Pre-pubblicato.
- [91] *Motivation - CodeCarbon*. URL: <https://docs.codecarbon.io/latest/explanation/why/>.
- [92] Eva-Lisa Meldau et al. «Automated Redaction of Names in Adverse Event Reports Using Transformer-Based Neural Networks». In: *BMC Medical Informatics and Decision Making* 24.1 (23 dic. 2024), p. 401. ISSN: 1472-6947.
- [93] *Prezzi dell'elettricità in leggero calo nel 2026*. URL: https://www.admin.ch/it/newsb/8nuE_fvwnfqCu8OHN0wKu.
- [94] Gabriele Algisi. *PII Discovery: Rilevamento Di Dati Personali in Documenti Non Strutturati Aziendali*. Ver. v1.0-thesis. 2026. URL: https://github.com/algiiii/PII_Discovery.
- [95] Michiel Overeem et al. «An Empirical Characterization of Event Sourced Systems and Their Schema Evolution – Lessons from Industry». In: *Journal of Systems and Software* 178 (ago. 2021), p. 110970. ISSN: 01641212. arXiv: 2104.01146 [cs.SE].
- [96] Greg Young. *CQRS Documents by Greg Young*. cqrsinfo.com, nov. 2010. URL: https://cqrs.wordpress.com/wp-content/uploads/2010/11/cqrs_documents.pdf.